

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Coherence

Instructor: Prof. Joshua San Miguel

Some slides adopted from Greg Steffan, Natalie Enright Jerger and Mikko Lipasti

Last Lecture

- Parallelism and cache locality
- Loop cost model
- Approximate caching

This Lecture

- Coherence definition
- Snooping-based coherence
- Valid-invalid protocol
- Write-update protocol
- MI and MSI protocols

The Coherence Problem

Is this outcome possible at the end of the program: $*X = 1$, $*Y = 1$?

```
// P0
```

```
*X = 1;
```

```
*Y = *Y + 1;
```

```
// P1
```

```
*Y = 1;
```

```
*X = *X + 1;
```

P0

P1

Main Memory: $X = 0$, $Y = 0$



The Coherence Problem

Is this outcome possible at the end of the program: $*X = 1$, $*Y = 1$? **No**

```
// P0
*X = 1;
*Y = *Y + 1;
```

```
// P1
*Y = 1;
*X = *X + 1;
```

P0

P1

Main Memory: $X = 0$, $Y = 0$



The Coherence Problem

Is this outcome possible at the end of the program: $*X = 1$, $*Y = 1$?

```
// P0
```

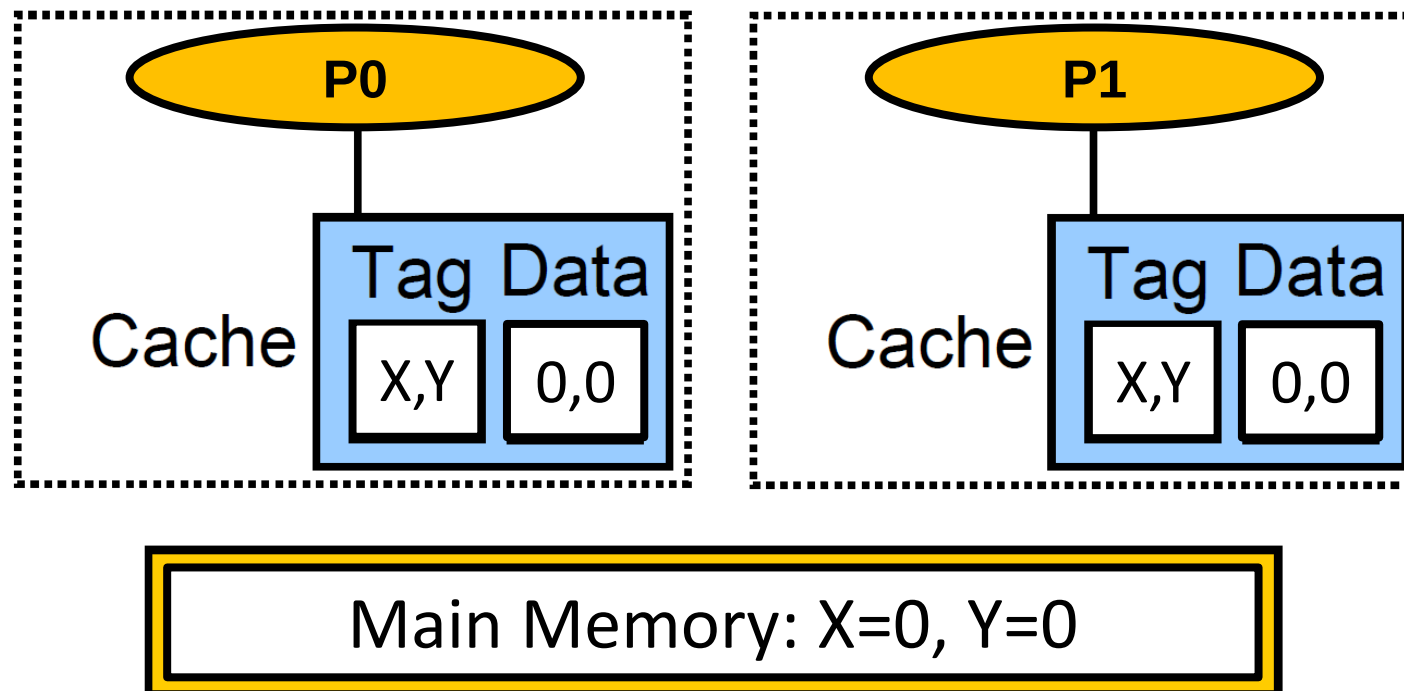
```
*X = 1;
```

```
*Y = *Y + 1;
```

```
// P1
```

```
*Y = 1;
```

```
*X = *X + 1;
```



The Coherence Problem

Is this outcome possible at the end of the program: $*X = 1$, $*Y = 1$?

// P0

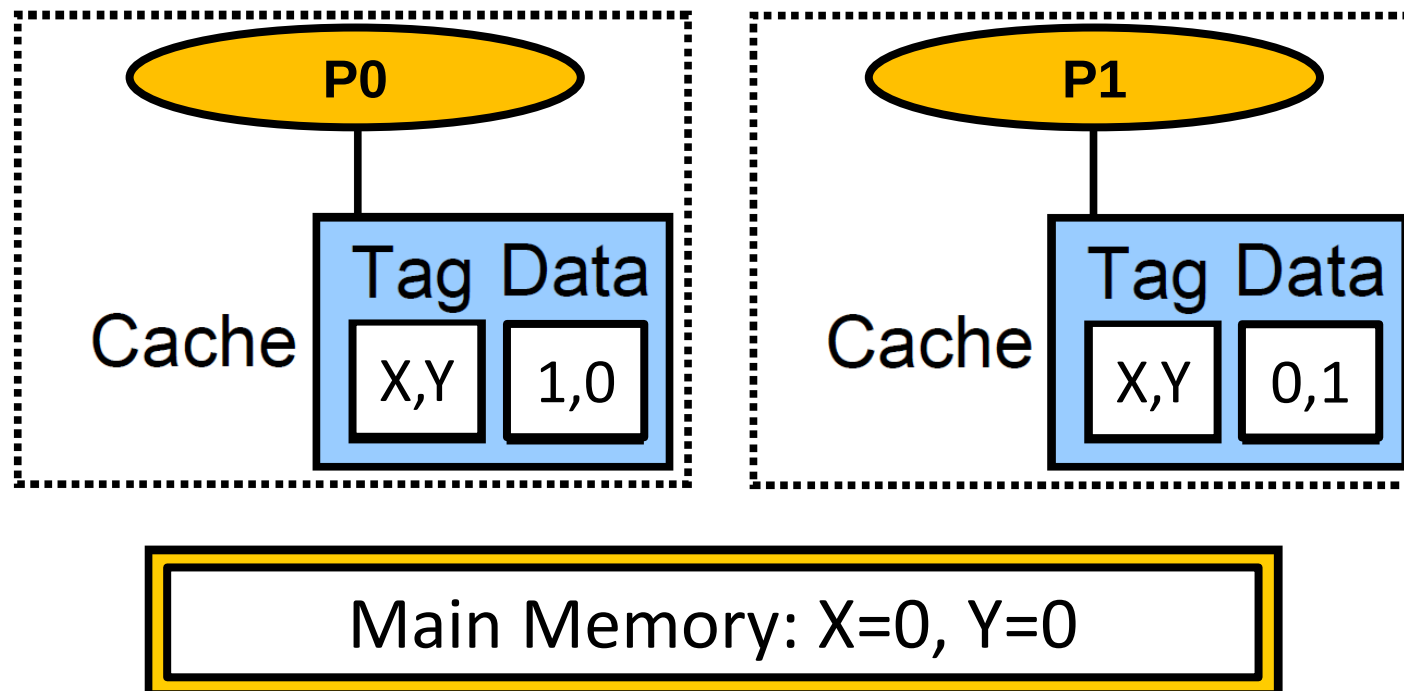
$*X = 1;$

$*Y = *Y + 1;$

// P1

$*Y = 1;$

$*X = *X + 1;$



The Coherence Problem

Is this outcome possible at the end of the program: $*X = 1$, $*Y = 1$?

// P0

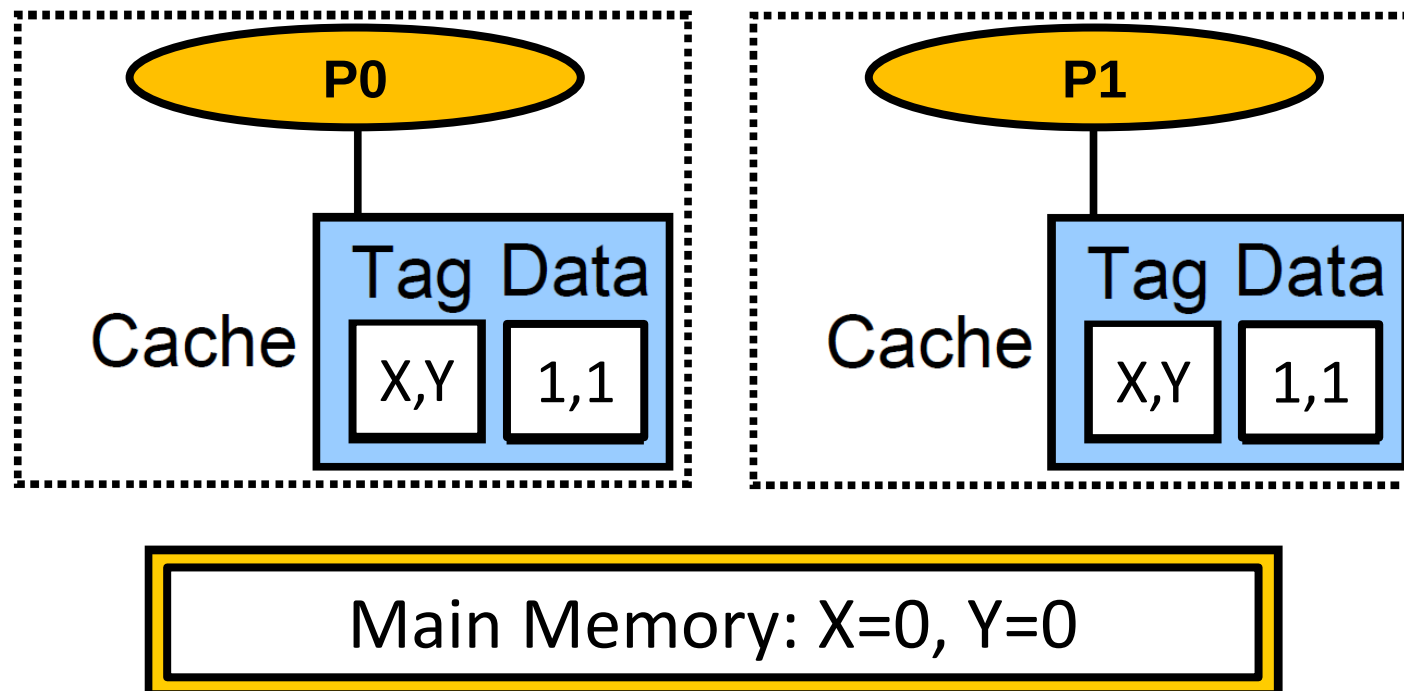
$*X = 1;$

$*Y = *Y + 1;$

// P1

$*Y = 1;$

$*X = *X + 1;$



The Coherence Problem

Is this outcome possible at the end of the program: $*X = 1$, $*Y = 1$? **Yes**

// P0

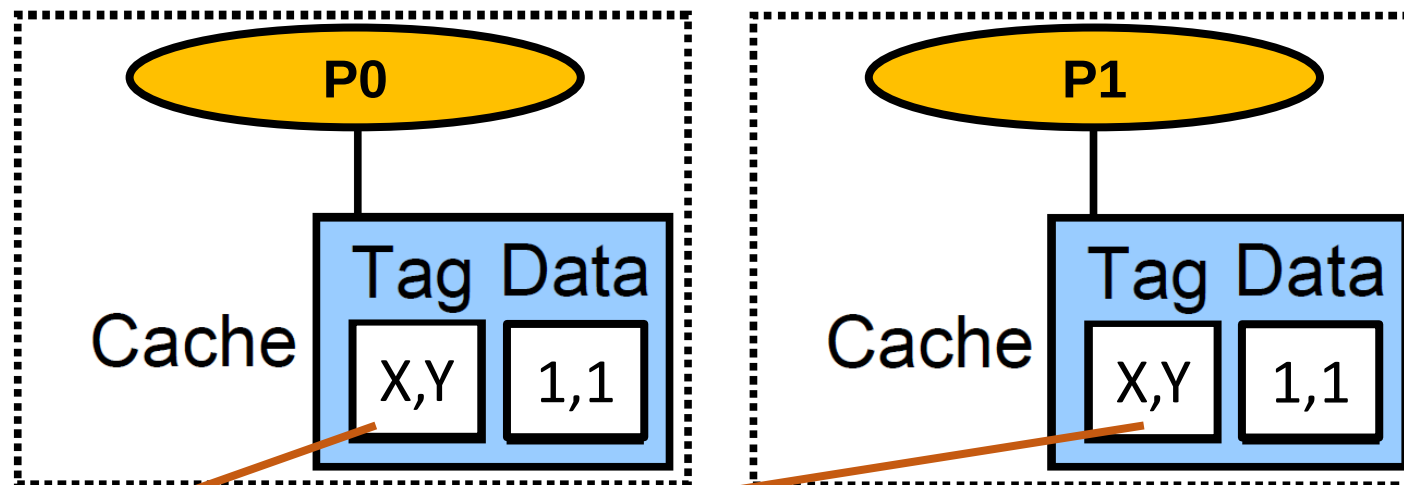
$*X = 1;$

$*Y = *Y + 1;$

// P1

$*Y = 1;$

$*X = *X + 1;$



culprit: replication
(due to caching of values)

Coherence Definition

Defined by two properties:

- **Write Serialization:** All modifications to a memory location's value are serialized, and all processors observe the same order in which these modifications take effect.
 - At any moment in time, only one processor can modify the value, and all processors agree on who it is (*single-writer, multiple-reader invariant*).
- **Write Propagation:** Any modification to a memory location's value is made visible to all readers simultaneously and instantaneously.
 - Any processors that read the memory location immediately after a modification must observe the new value (*data-value invariant*).

Coherence Definition

Another way to think about it:

- **A global *logical* time must be imposed upon all reads and writes to a memory location.**

Coherence Definition

Another way to think about it:

- **A global *logical* time must be imposed upon all reads and writes to a memory location.**
 - Can trivially achieve by serializing all accesses (whether read or write).

Coherence Definition

Another way to think about it:

- **A global *logical* time must be imposed upon all reads and writes to a memory location.**
 - Can trivially achieve by serializing all accesses (whether read or write).
 - Can do better by allowing reads to occur in parallel.

Coherence Definition

Another way to think about it:

- **A global *logical* time must be imposed upon all reads and writes to a memory location.**
 - Can trivially achieve by serializing all accesses (whether read or write).
 - Can do better by allowing reads to occur in parallel.
 - Can do even better by allowing commutative updates in parallel (Paper Review 3).

The Coherence Problem

What if we had no caches?

- No problem; serialize writes to memory location

The Coherence Problem

What if we had no caches?

- No problem; serialize writes to memory location

What if we had one shared cache?

- No problem; serialize writes to cache location

The Coherence Problem

What if we had no caches?

- No problem; serialize writes to memory location

What if we had one shared cache?

- No problem; serialize writes to cache location

What if we had multiple private caches with efficient broadcast (e.g., bus)?

- Snooping-based coherence

The Coherence Problem

What if we had no caches?

- No problem; serialize writes to memory location

What if we had one shared cache?

- No problem; serialize writes to cache location

What if we had multiple private caches with efficient broadcast (e.g., bus)?

- Snooping-based coherence

What if we had multiple private caches on an arbitrary network?

- Directory-based coherence

Cache Coherence

Snooping-based coherence

Caches *snoop* (i.e., observe) all read/write requests (and their responses) to memory and other caches via broadcasting.

Directory-based coherence

Read/write requests (and their responses) are point-to-point; a separate *directory* structure must be referenced first to see who has copies of the data.

Cache Coherence Protocols

Update-based protocol

Each cache is responsible for keeping its own blocks updated to the latest value.

Invalidation-based protocol

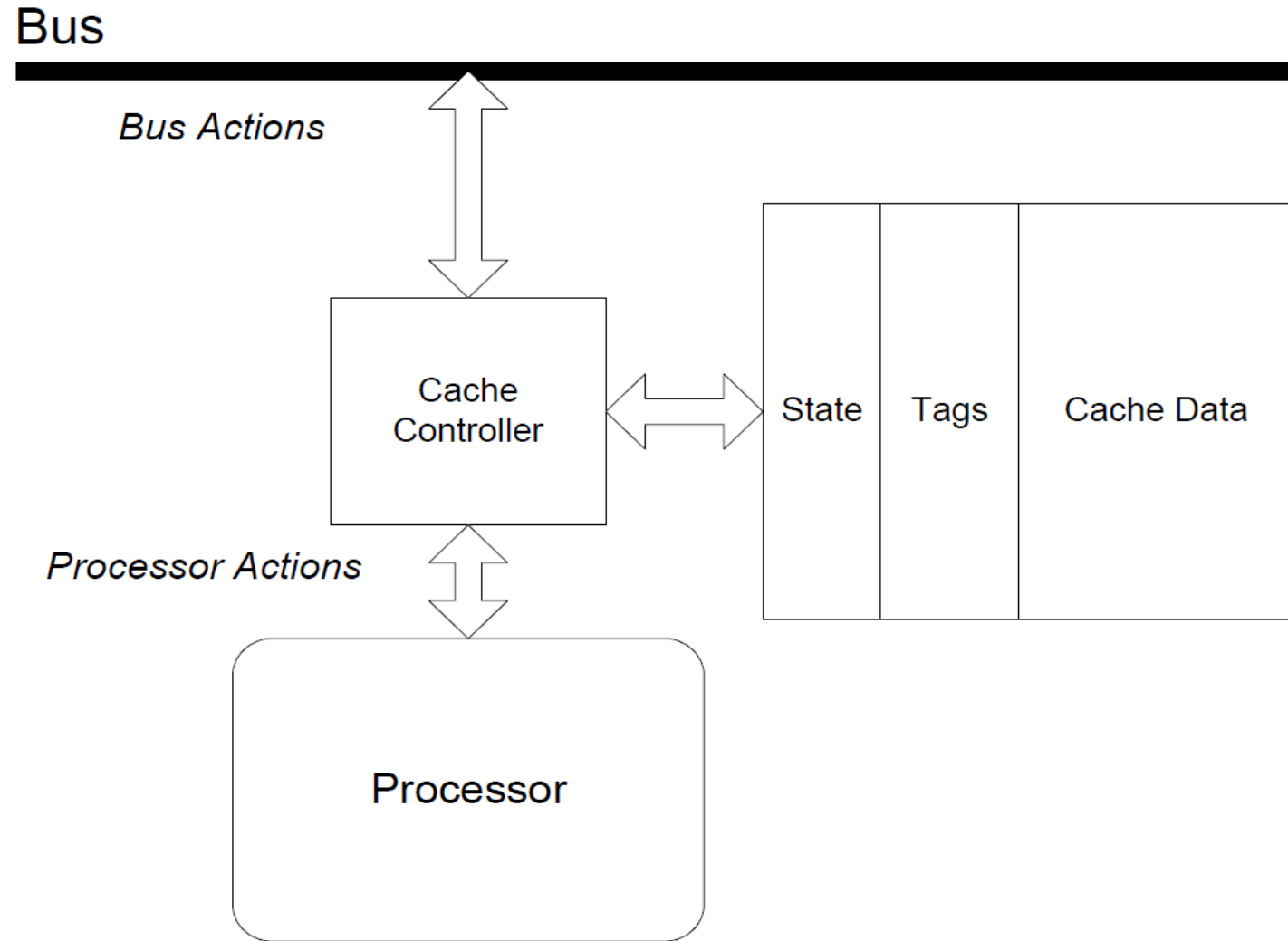
Each cache that modifies a block is responsible for notifying every other cache about the change (i.e., by invalidating the other caches' stale copies).

Cache Coherence Protocols

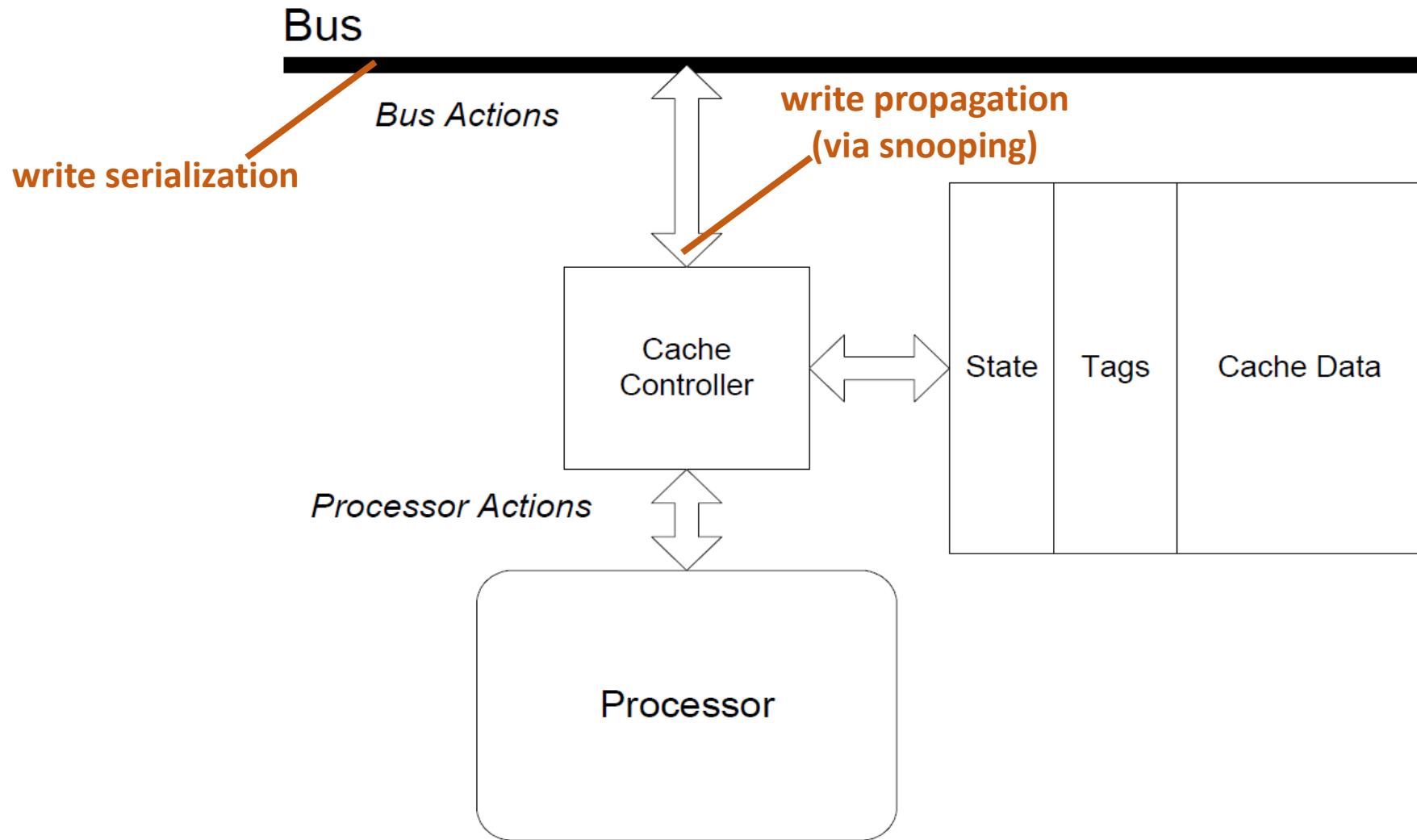
A cache coherence protocol is implemented as a system of FSMs:

- Each cache and memory employs a controller, which maintains a **state** for each block it contains and based on external **events** (e.g., requests, responses) from the upper and lower levels of the memory hierarchy, the controller **transitions** to a new state for the block and performs any **actions** associated with the transition (which may trigger new events).

Snooping-Based Coherence



Snooping-Based Coherence



Snooping-Based Coherence

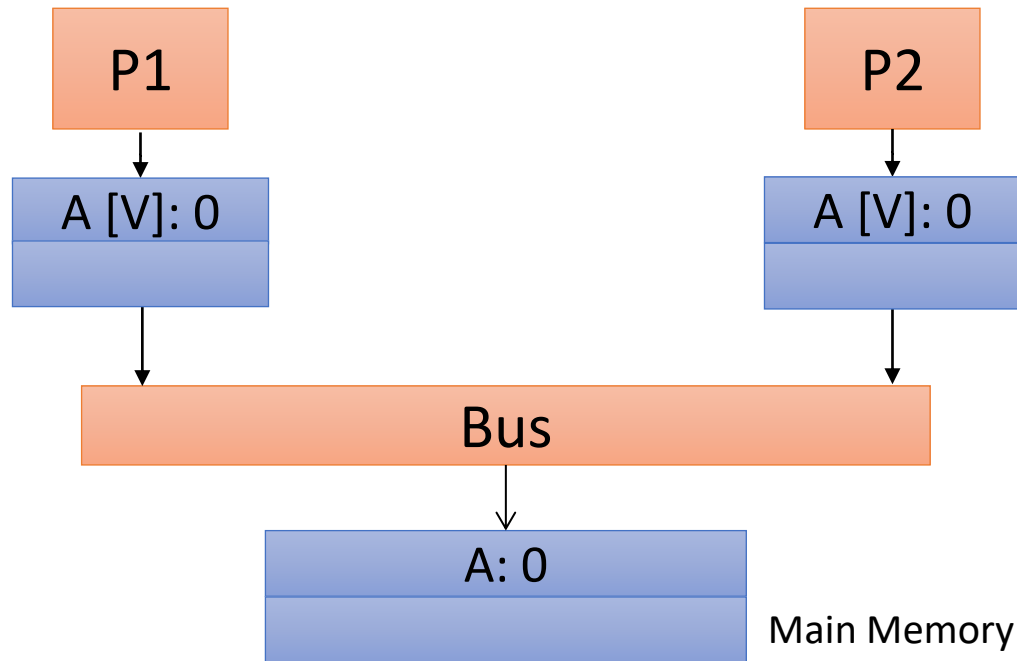
Valid-Invalid Protocol

- Write-through, write-no-allocate caches
- *Valid* state: Reads and writes allowed; block is cached
- *Invalid* state: Writes allowed; block is not cached

Snooping-Based Coherence

Valid-Invalid Protocol

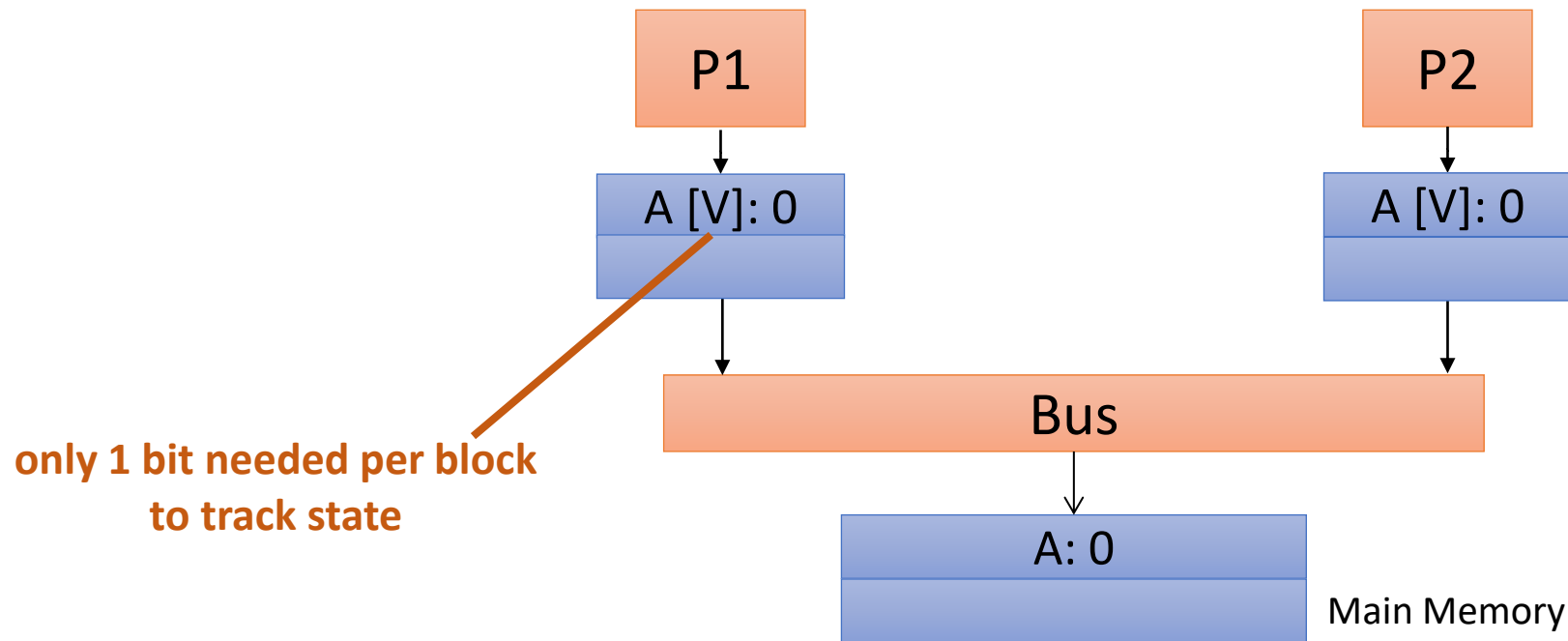
- Write-through, write-no-allocate caches
- *Valid* state: Reads and writes allowed; block is cached
- *Invalid* state: Writes allowed; block is not cached



Snooping-Based Coherence

Valid-Invalid Protocol

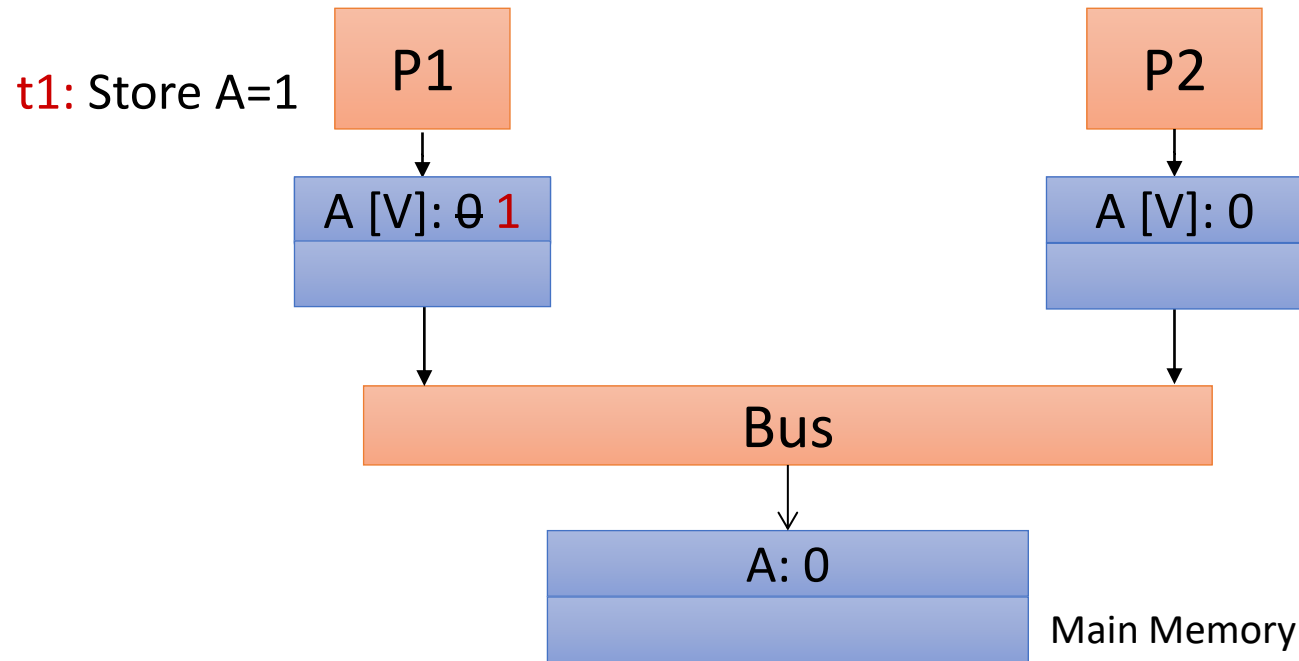
- Write-through, write-no-allocate caches
- *Valid* state: Reads and writes allowed; block is cached
- *Invalid* state: Writes allowed; block is not cached



Snooping-Based Coherence

Valid-Invalid Protocol

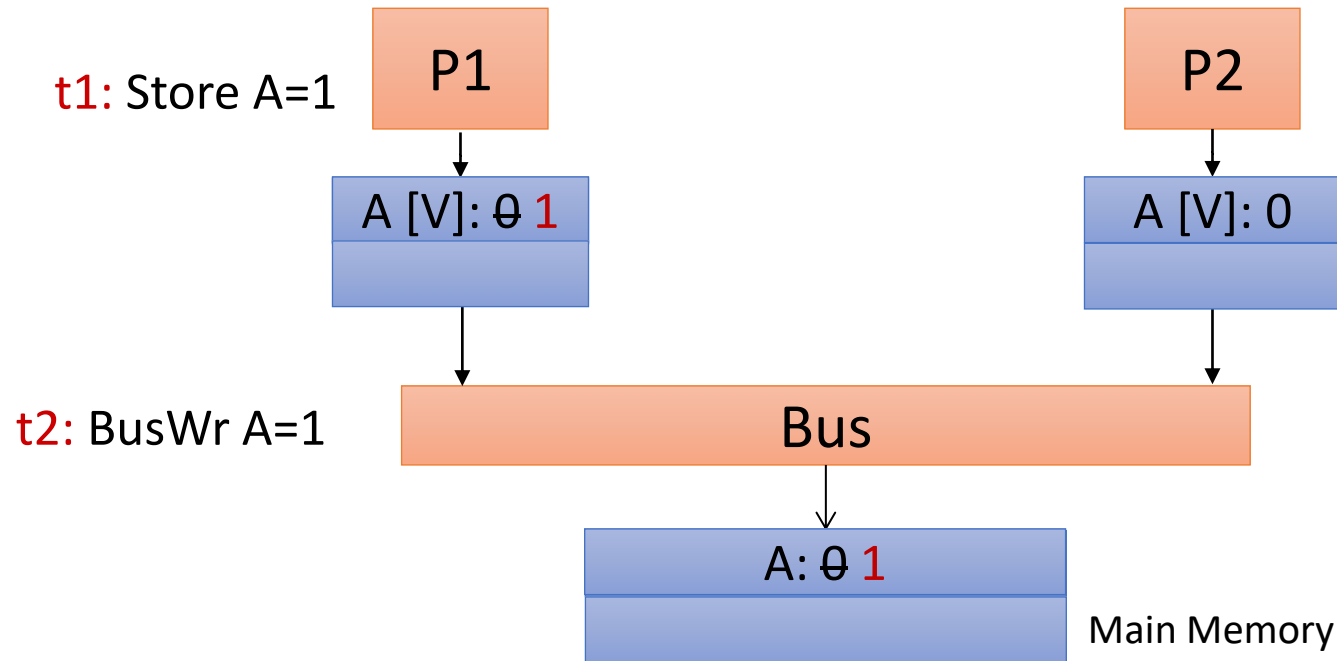
- Write-through, write-no-allocate caches
- *Valid* state: Reads and writes allowed; block is cached
- *Invalid* state: Writes allowed; block is not cached



Snooping-Based Coherence

Valid-Invalid Protocol

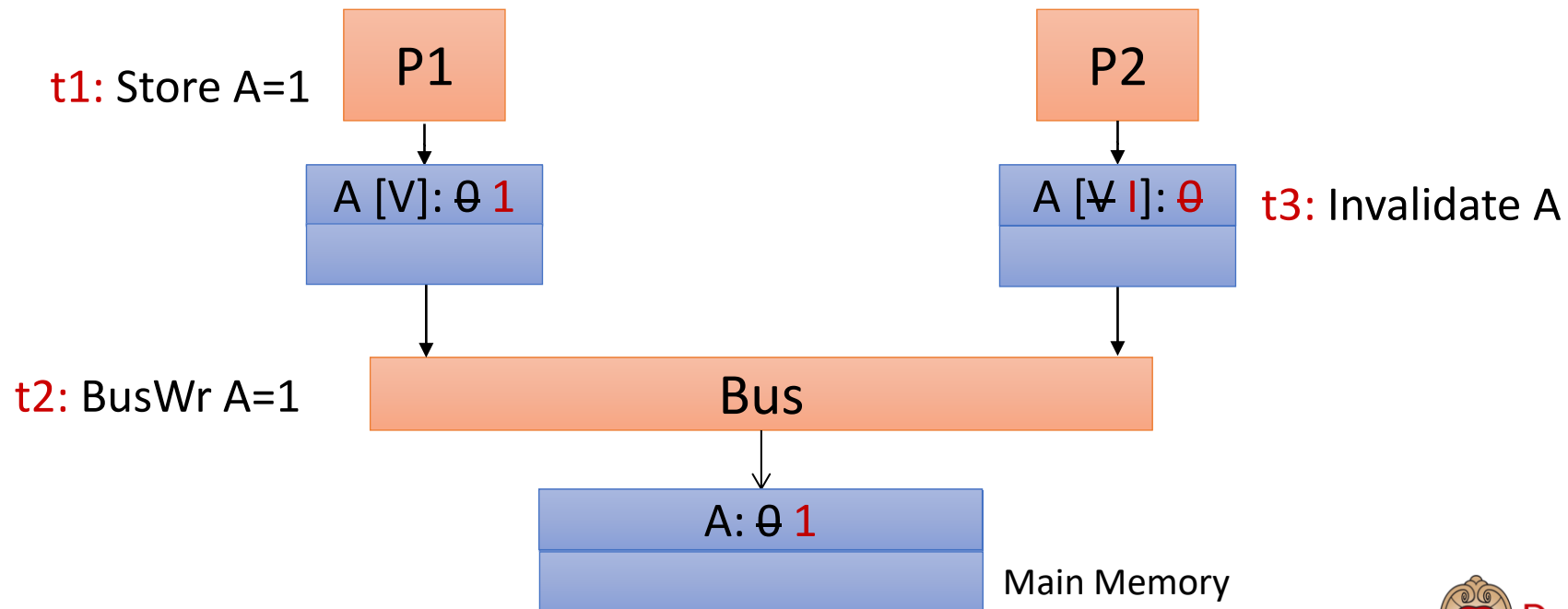
- Write-through, write-no-allocate caches
- *Valid* state: Reads and writes allowed; block is cached
- *Invalid* state: Writes allowed; block is not cached



Snooping-Based Coherence

Valid-Invalid Protocol

- Write-through, write-no-allocate caches
- *Valid* state: Reads and writes allowed; block is cached
- *Invalid* state: Writes allowed; block is not cached

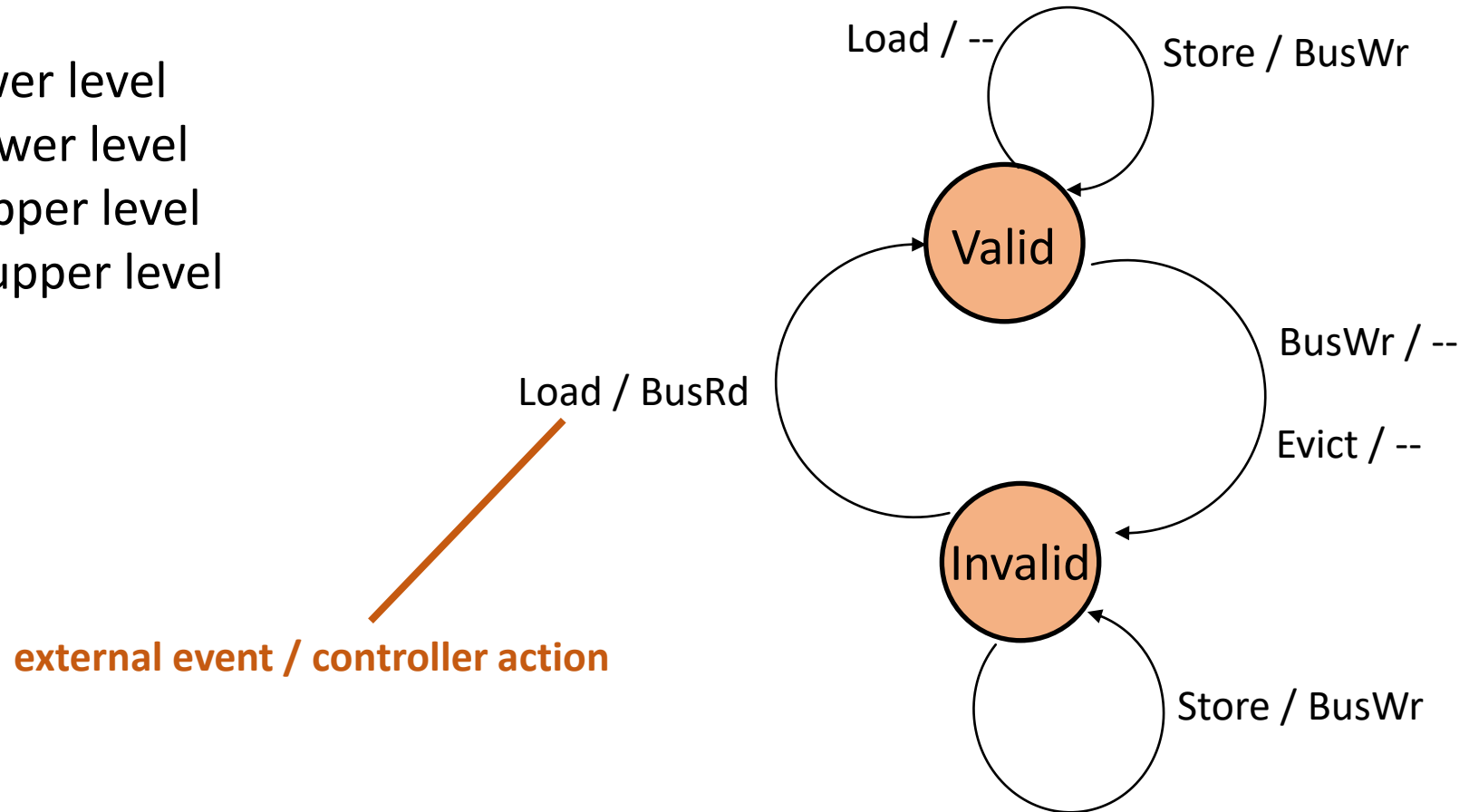


Snooping-Based Coherence

Valid-Invalid Protocol

- *Load*: read transaction at lower level
- *Store*: write transaction at lower level
- *BusRd*: read transaction at upper level
- *BusWr*: write transaction at upper level
- *Evict*: replacement in cache

Each transaction consists of a request and a response.

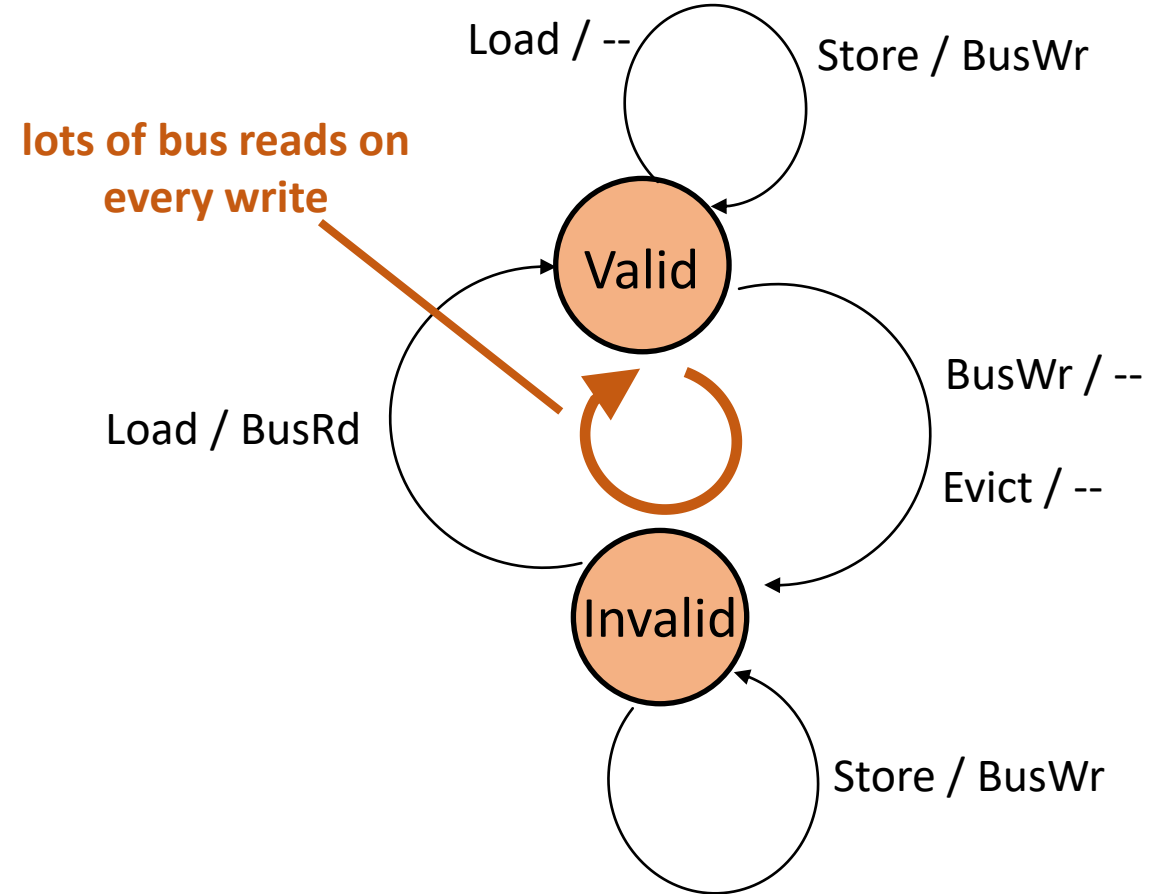


Snooping-Based Coherence

Valid-Invalid Protocol

- *Load*: read transaction at lower level
- *Store*: write transaction at lower level
- *BusRd*: read transaction at upper level
- *BusWr*: write transaction at upper level
- *Evict*: replacement in cache

Each transaction consists of a request and a response.



Snooping-Based Coherence

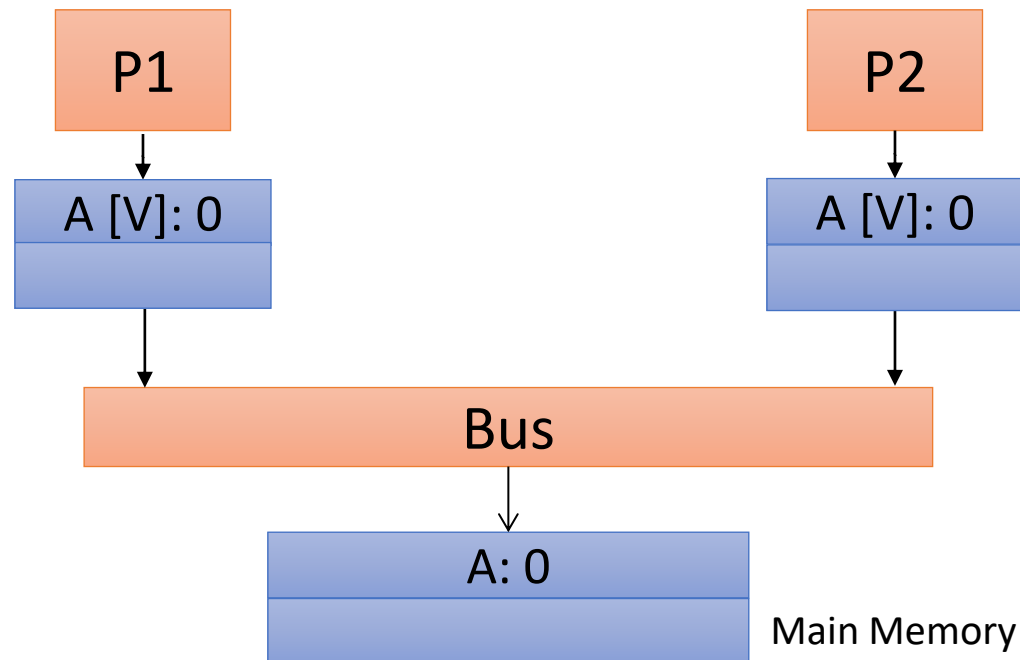
Write-Update Protocol

- Write-through, write-no-allocate caches
- Same as Valid-Invalid, except instead of invalidations, *snarf* new values from bus

Snooping-Based Coherence

Write-Update Protocol

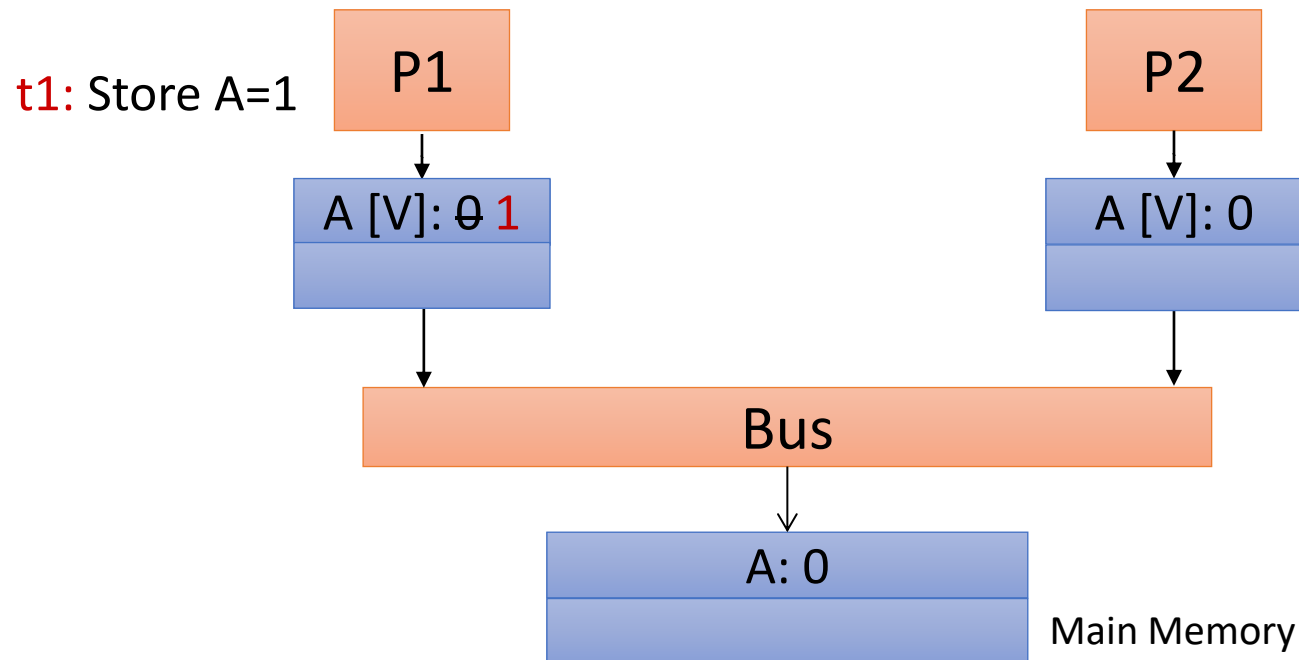
- Write-through, write-no-allocate caches
- Same as Valid-Invalid, except instead of invalidations, *snarf* new values from bus



Snooping-Based Coherence

Write-Update Protocol

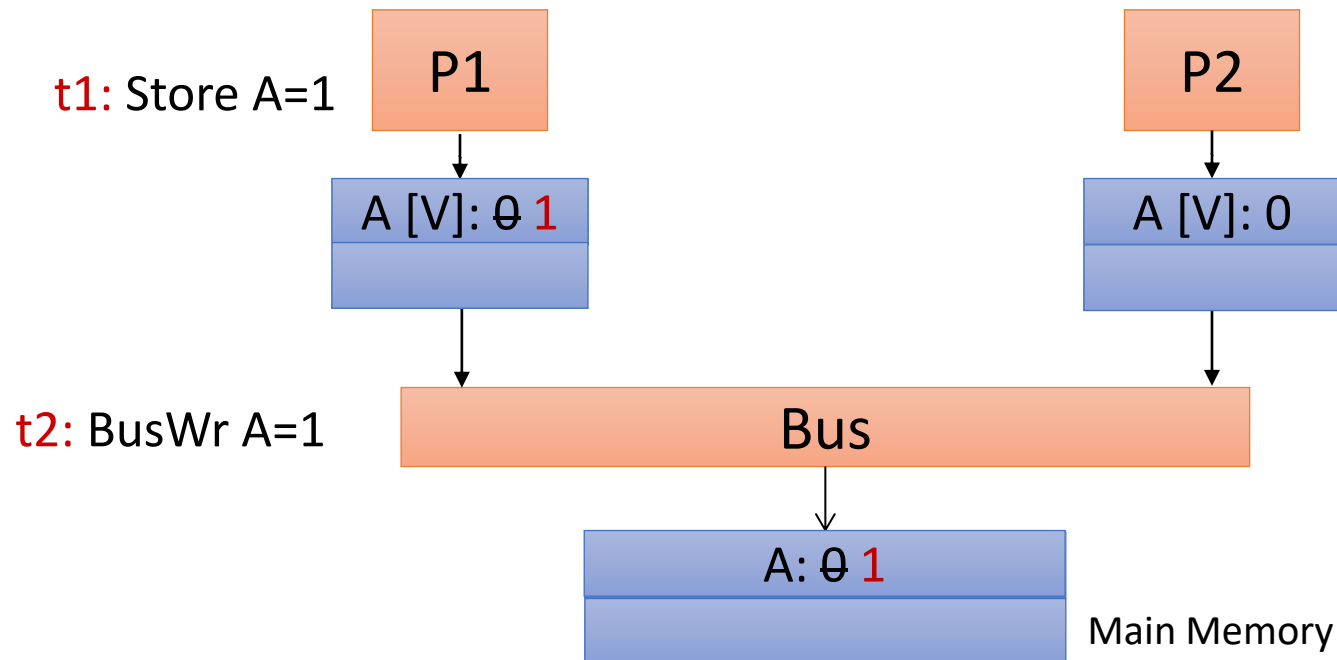
- Write-through, write-no-allocate caches
- Same as Valid-Invalid, except instead of invalidations, *snarf* new values from bus



Snooping-Based Coherence

Write-Update Protocol

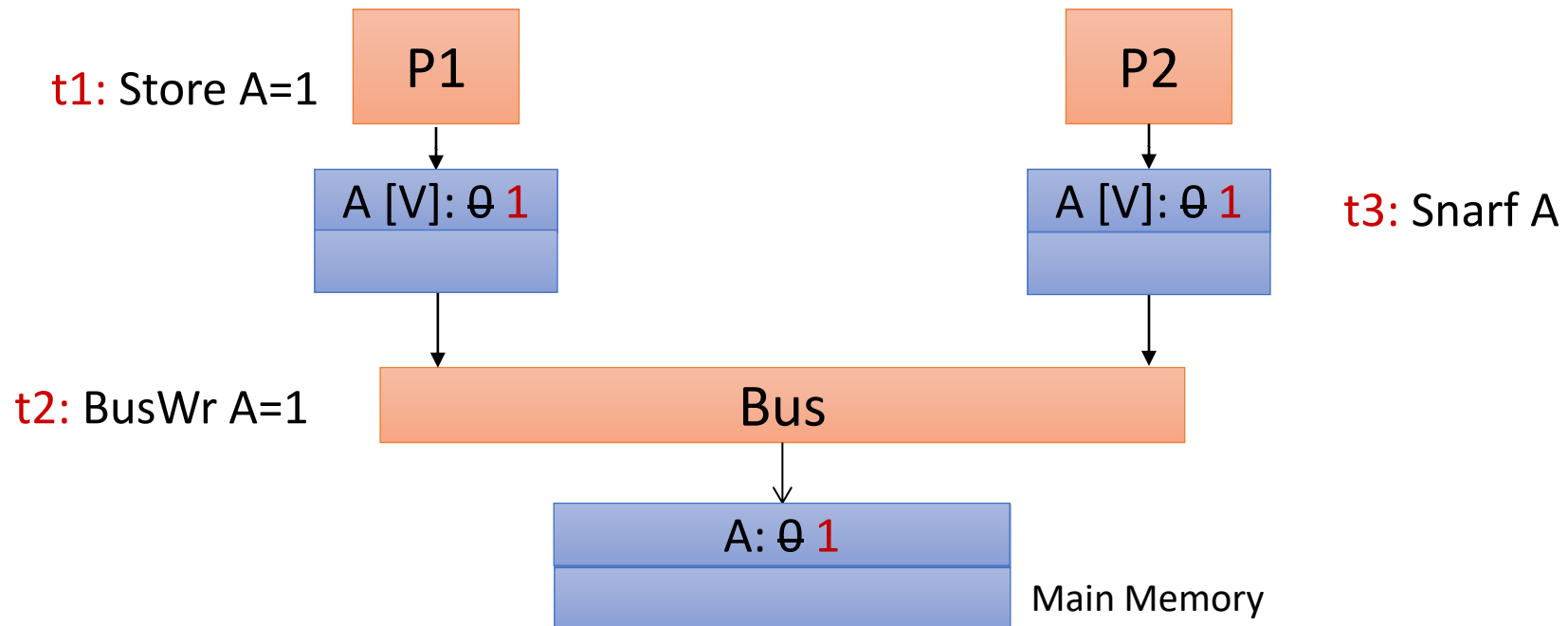
- Write-through, write-no-allocate caches
- Same as Valid-Invalid, except instead of invalidations, *snarf* new values from bus



Snooping-Based Coherence

Write-Update Protocol

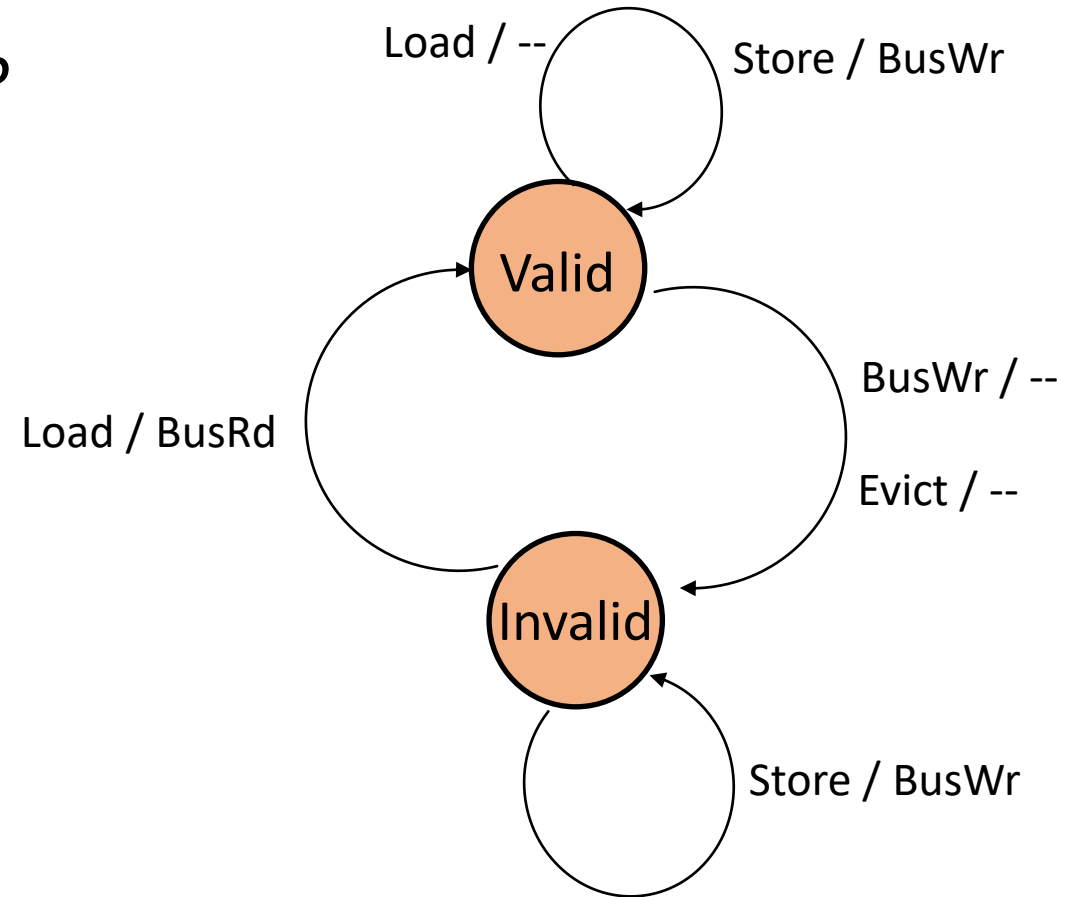
- Write-through, write-no-allocate caches
- Same as Valid-Invalid, except instead of invalidations, *snarf* new values from bus



Snooping-Based Coherence

Write-Update Protocol

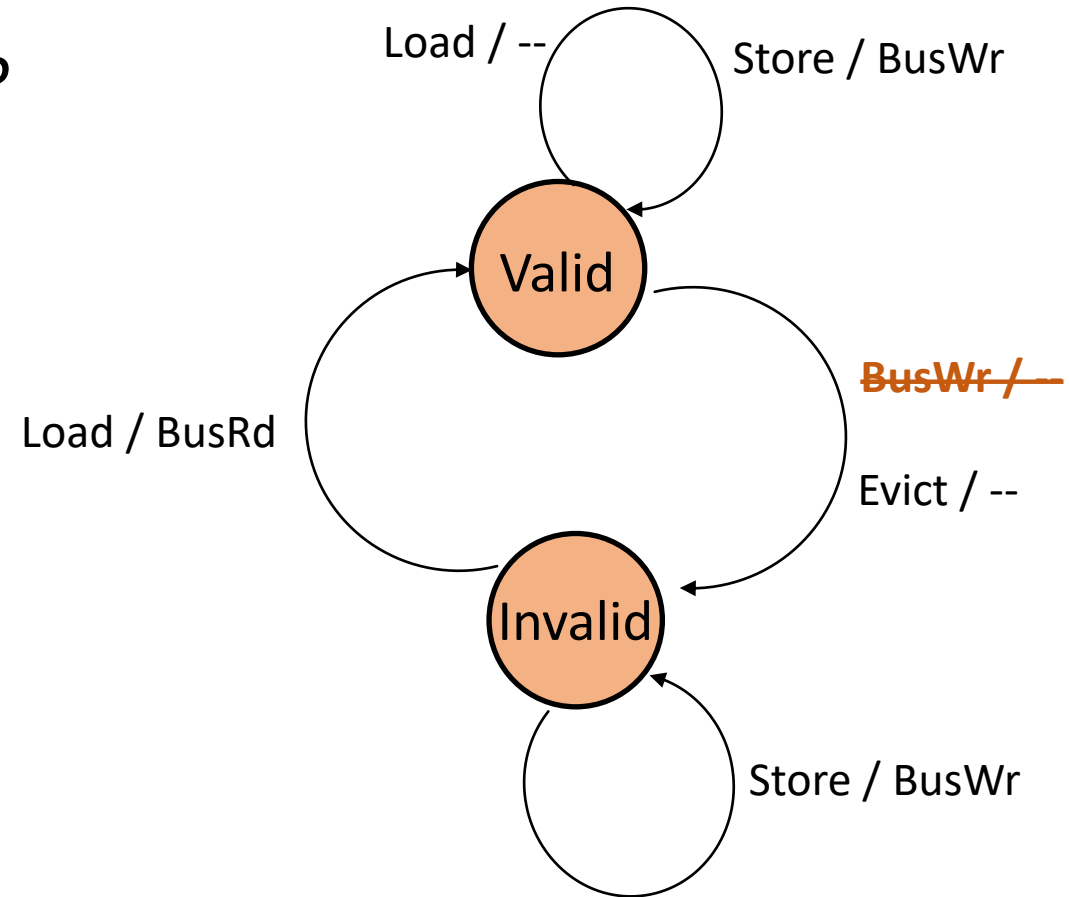
How does the state diagram change?



Snooping-Based Coherence

Write-Update Protocol

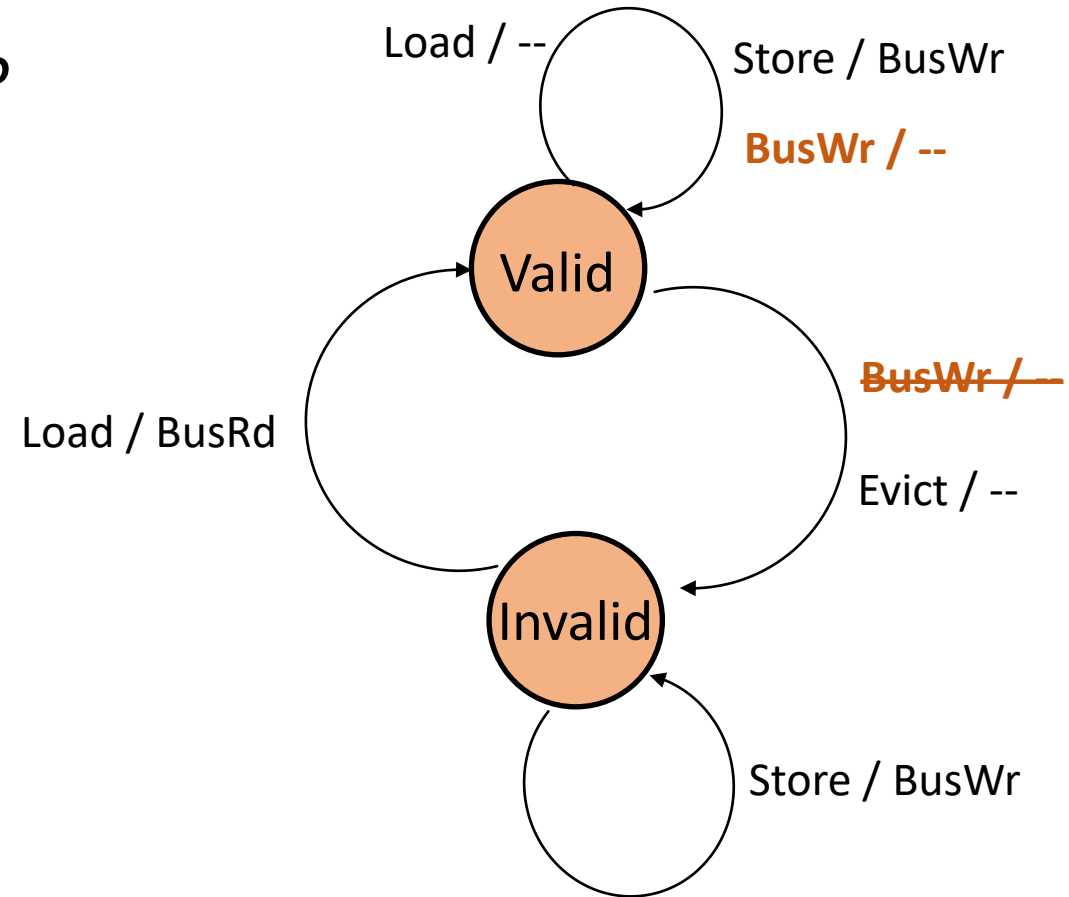
How does the state diagram change?



Snooping-Based Coherence

Write-Update Protocol

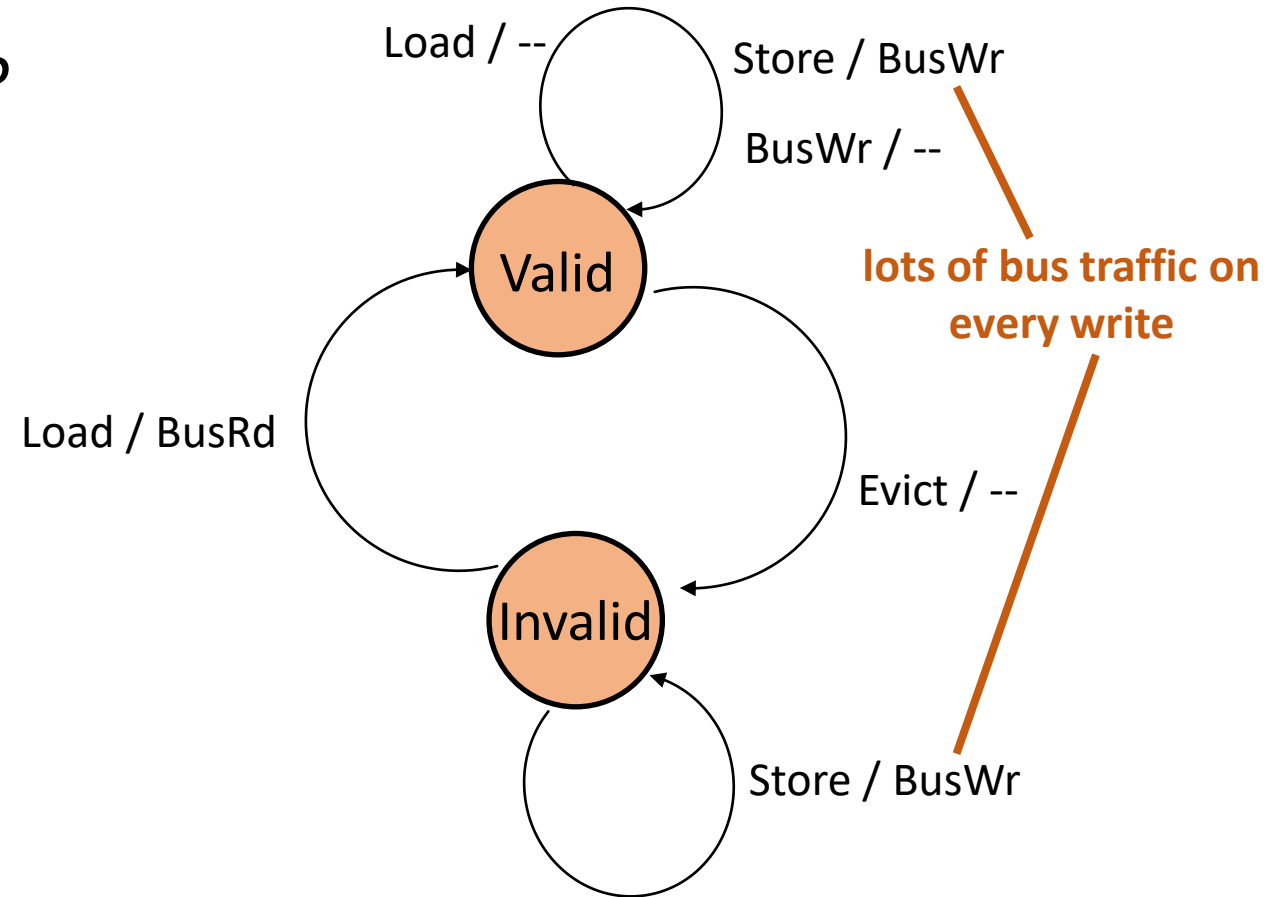
How does the state diagram change?



Snooping-Based Coherence

Write-Update Protocol

How does the state diagram change?



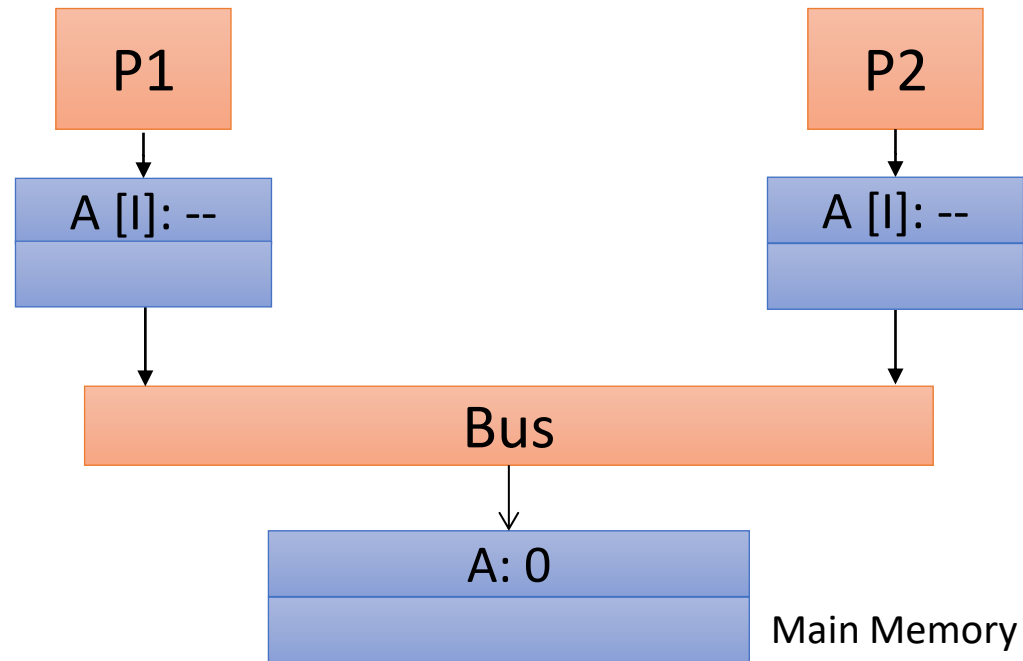
Snooping-Based Coherence

Modified-Invalid (MI) Protocol

- Write-back, write-allocate caches
- *Modified* state: Reads and writes allowed; cache has only copy of block
- *Invalid* state: Block is not cached
- *BusRdX*: Request exclusive access to block
- *BusReply*: Send block to requester
- *BusWB*: Send block to upper level

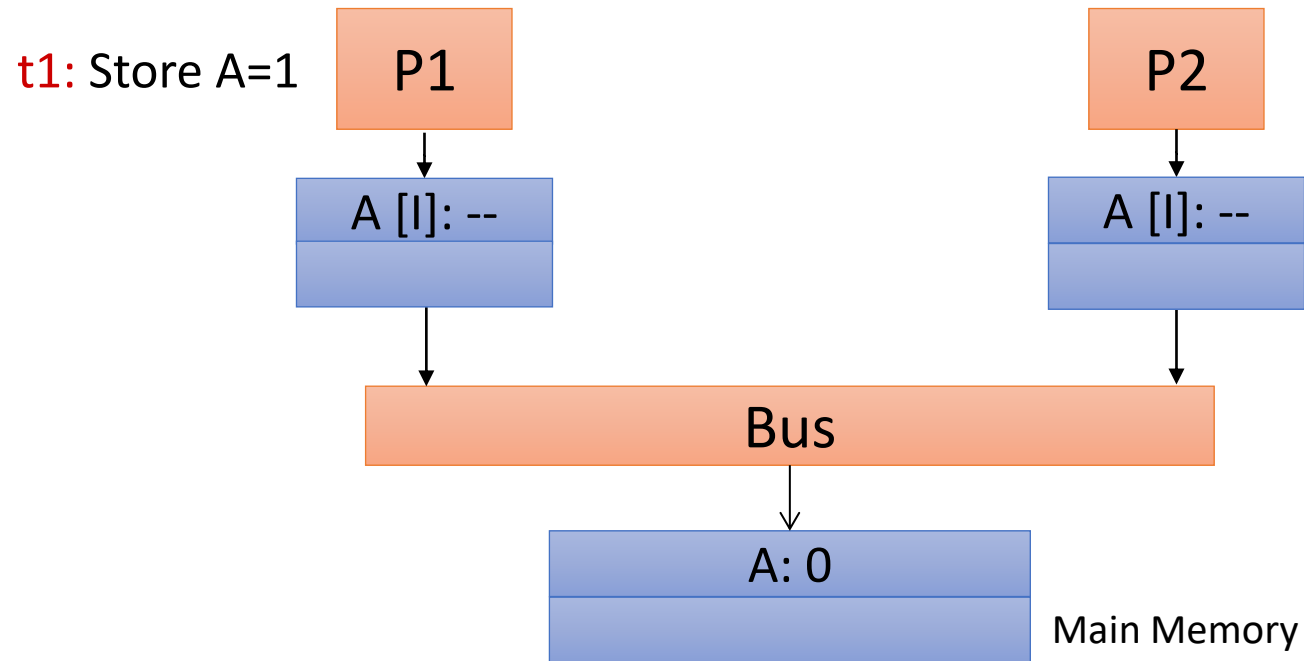
Snooping-Based Coherence

Modified-Invalid (MI) Protocol



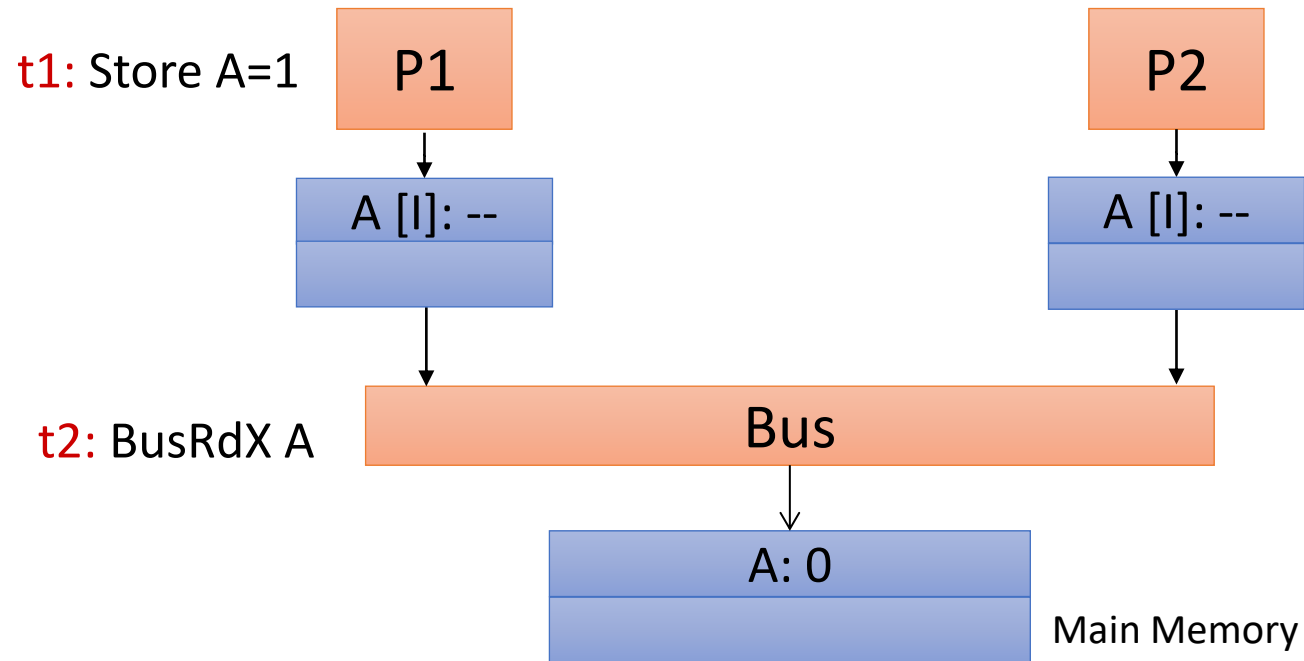
Snooping-Based Coherence

Modified-Invalid (MI) Protocol



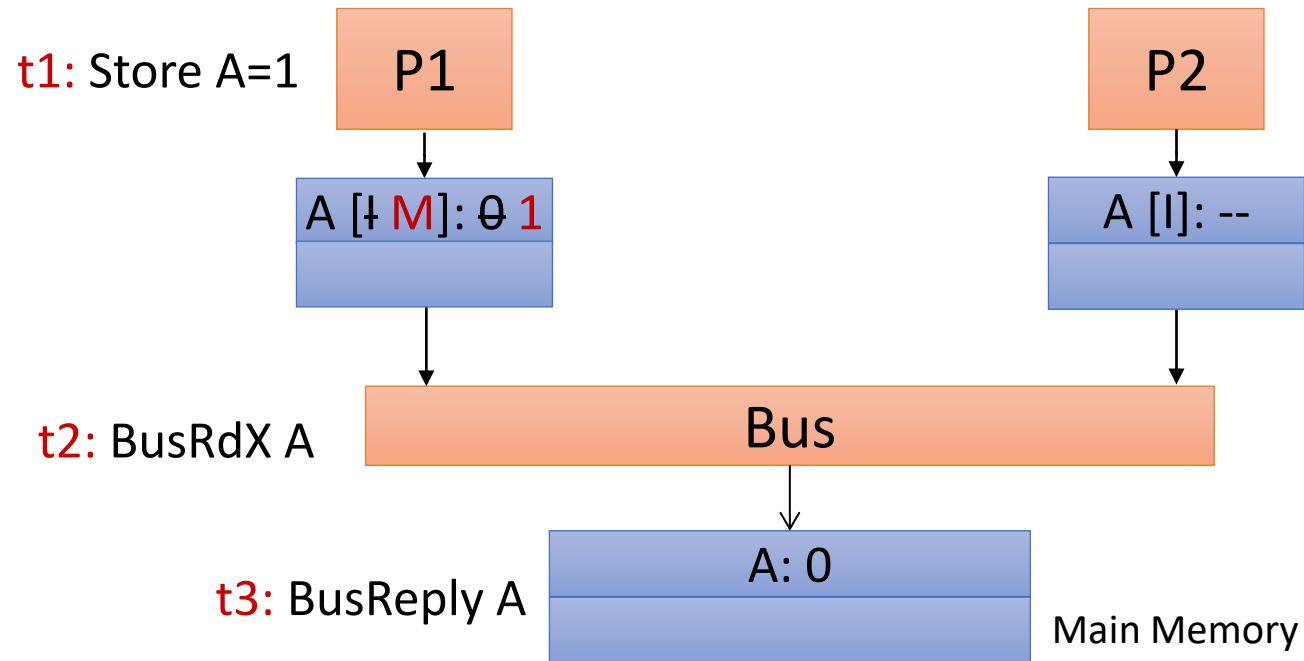
Snooping-Based Coherence

Modified-Invalid (MI) Protocol



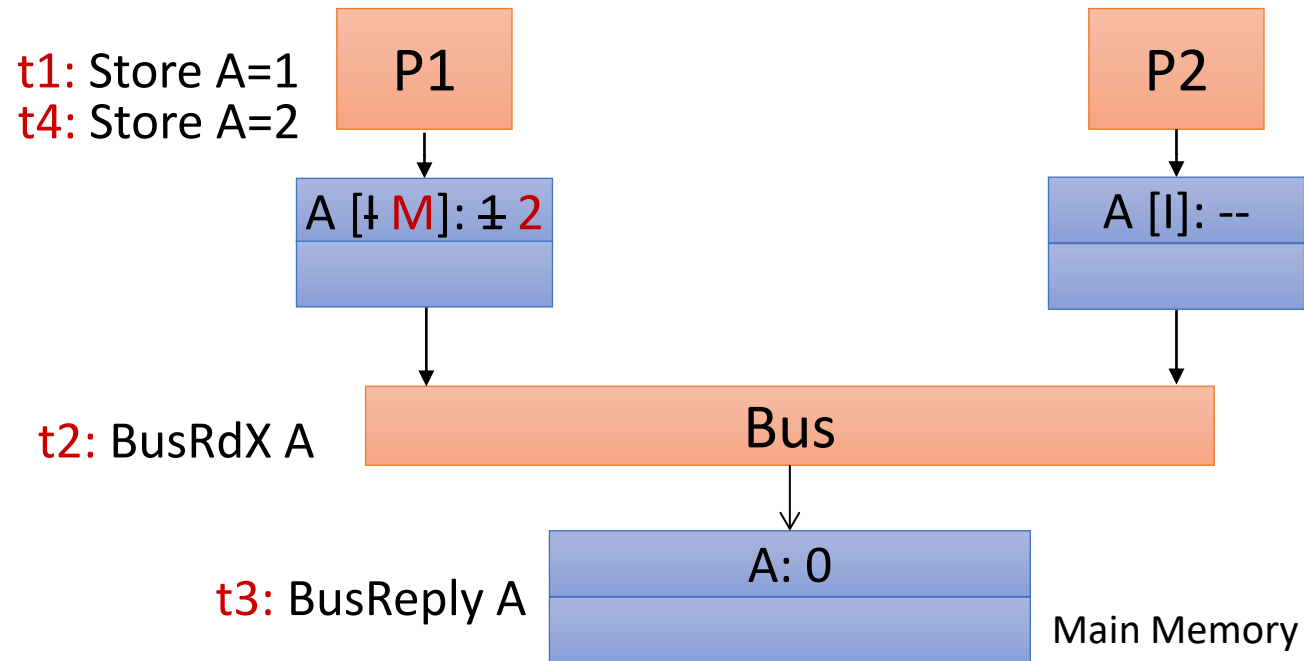
Snooping-Based Coherence

Modified-Invalid (MI) Protocol



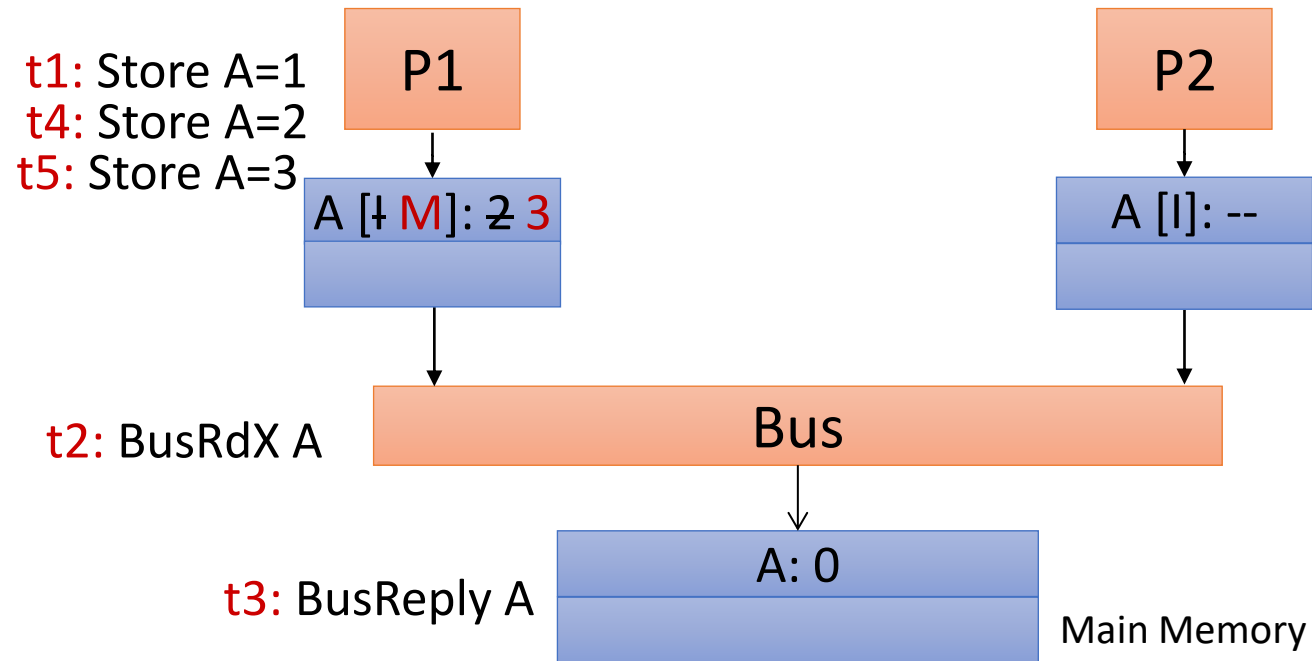
Snooping-Based Coherence

Modified-Invalid (MI) Protocol



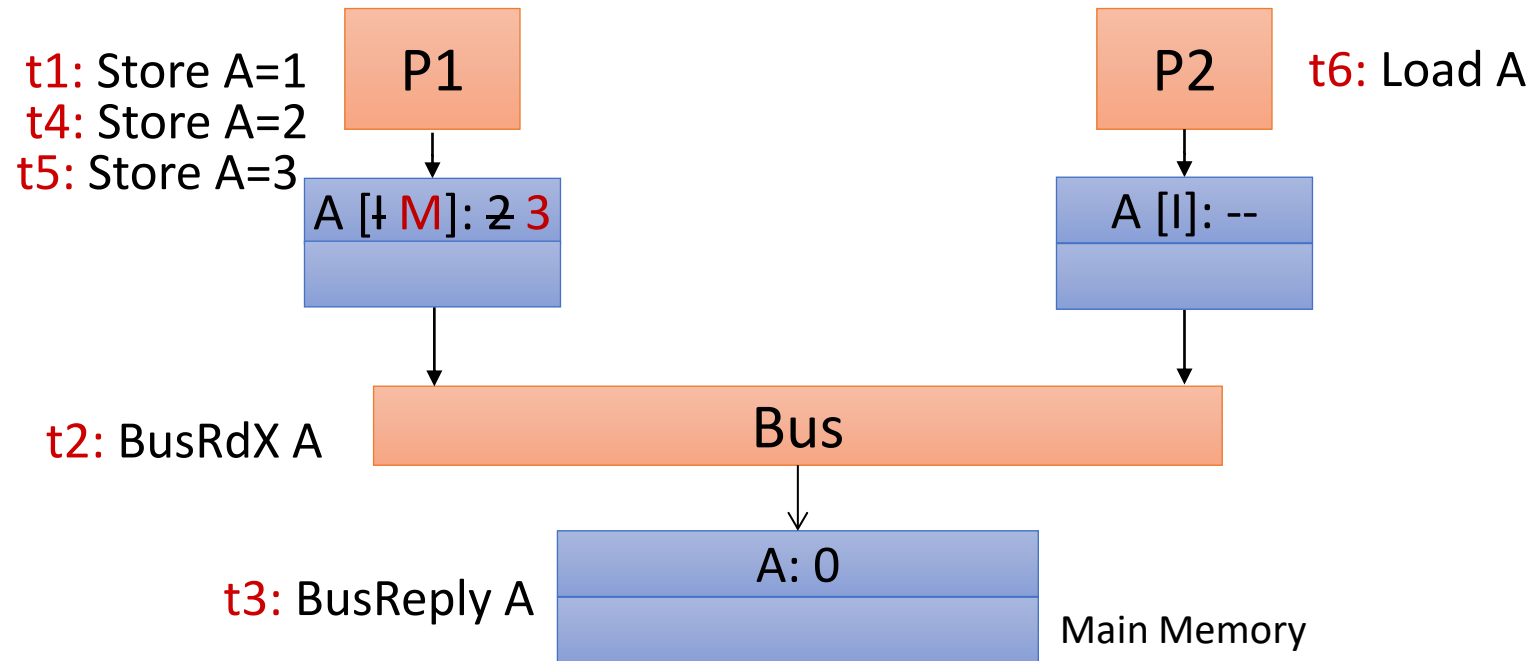
Snooping-Based Coherence

Modified-Invalid (MI) Protocol



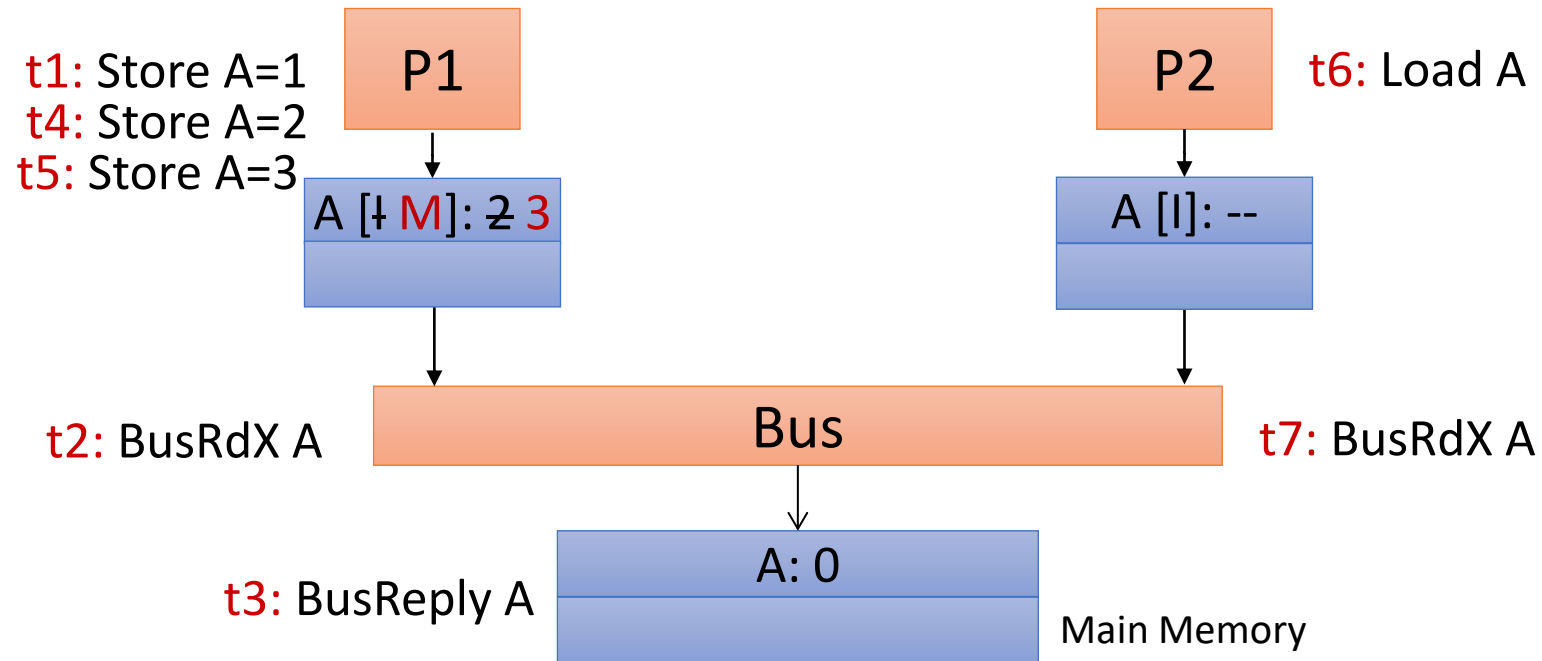
Snooping-Based Coherence

Modified-Invalid (MI) Protocol



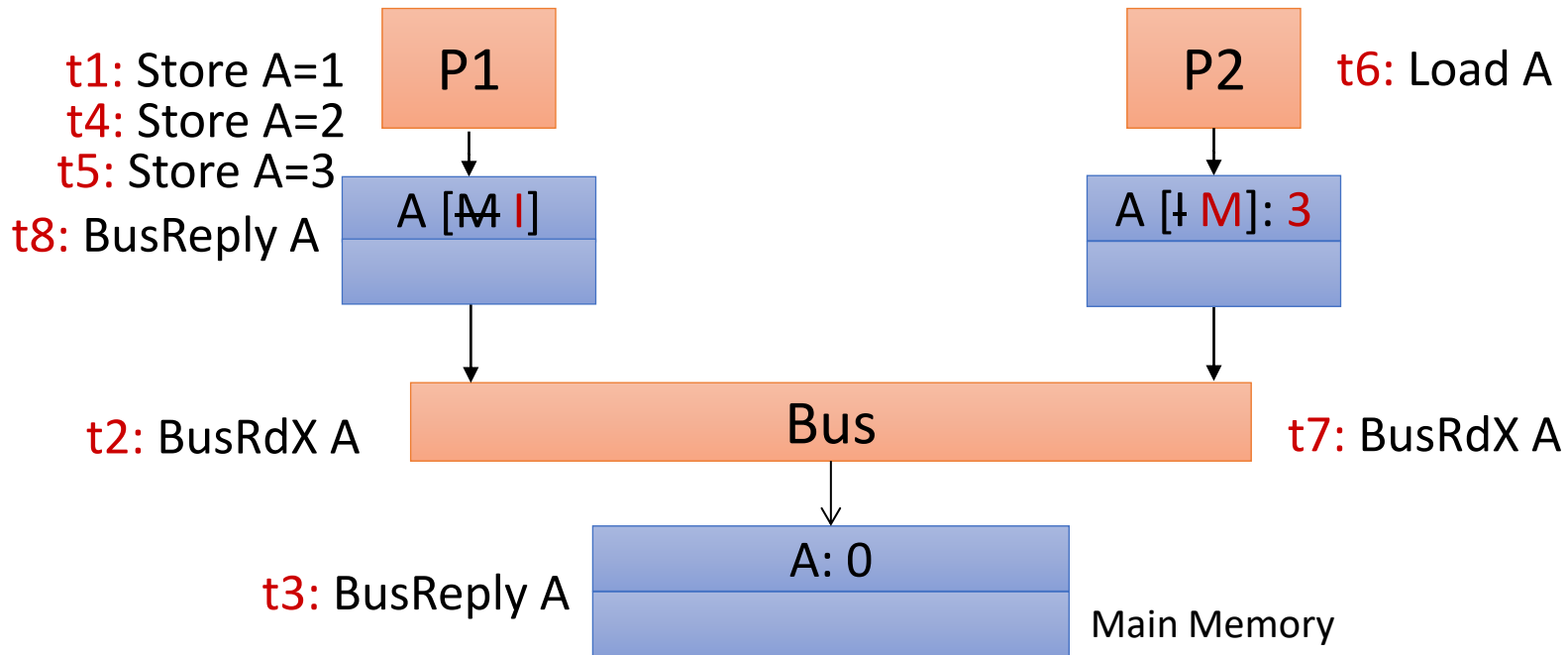
Snooping-Based Coherence

Modified-Invalid (MI) Protocol



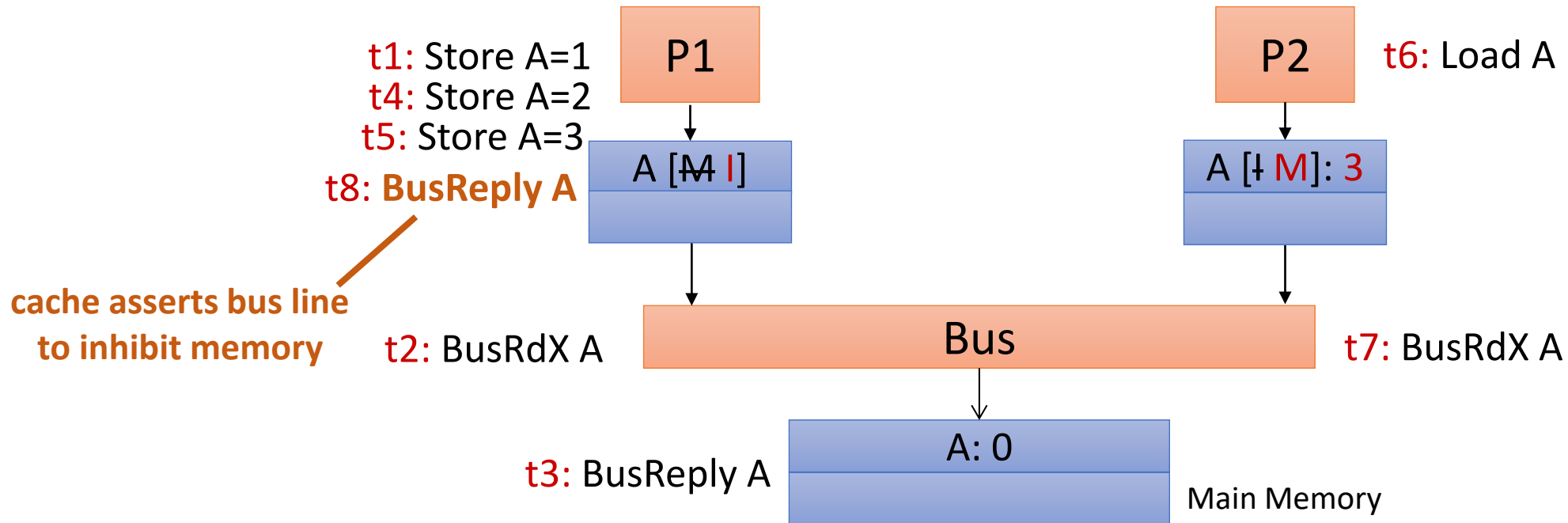
Snooping-Based Coherence

Modified-Invalid (MI) Protocol



Snooping-Based Coherence

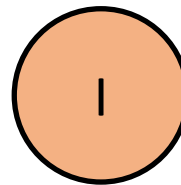
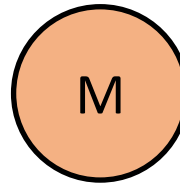
Modified-Invalid (MI) Protocol



Snooping-Based Coherence

Modified-Invalid (MI) Protocol

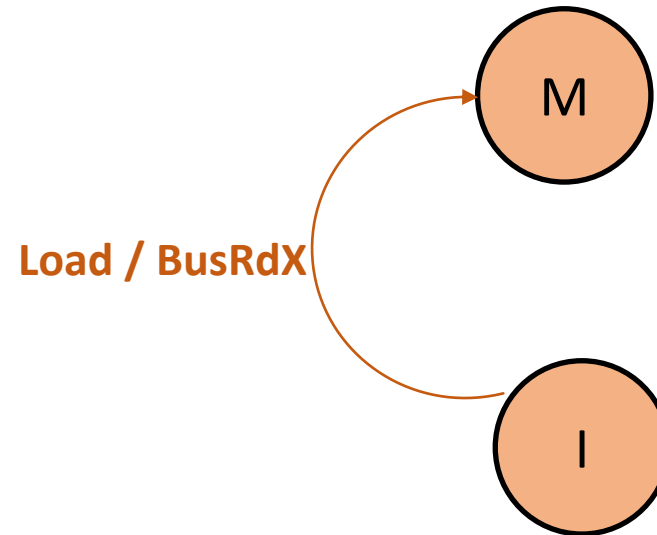
What happens on a Load?



Snooping-Based Coherence

Modified-Invalid (MI) Protocol

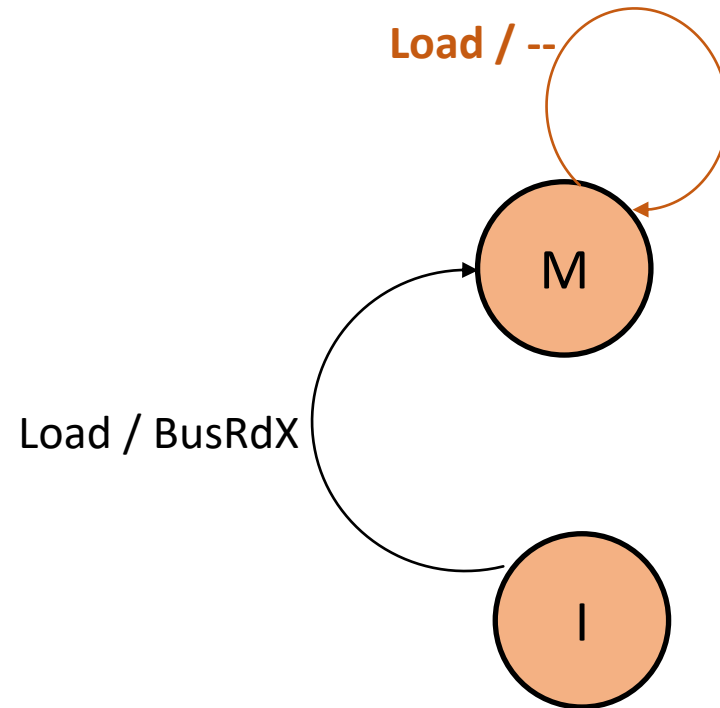
What happens on a Load?



Snooping-Based Coherence

Modified-Invalid (MI) Protocol

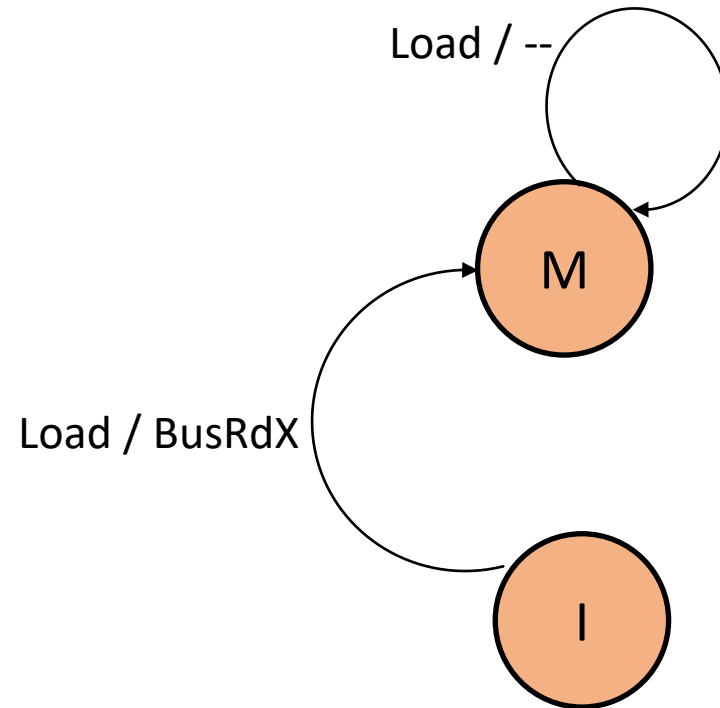
What happens on a Load?



Snooping-Based Coherence

Modified-Invalid (MI) Protocol

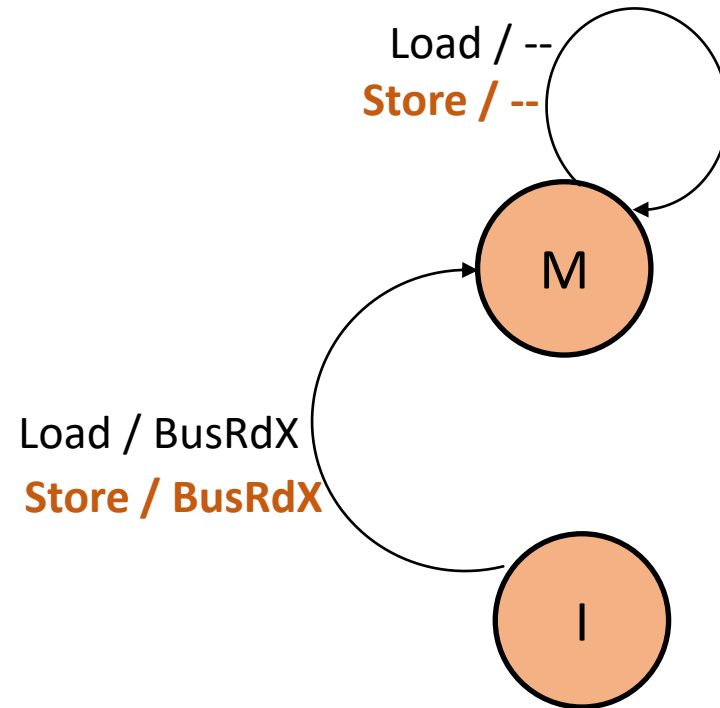
What happens on a Store?



Snooping-Based Coherence

Modified-Invalid (MI) Protocol

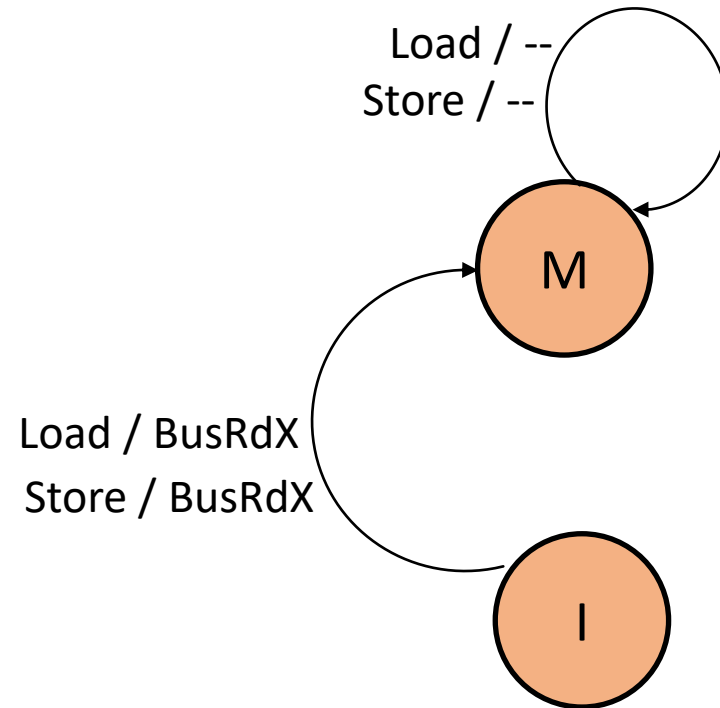
What happens on a Store?



Snooping-Based Coherence

Modified-Invalid (MI) Protocol

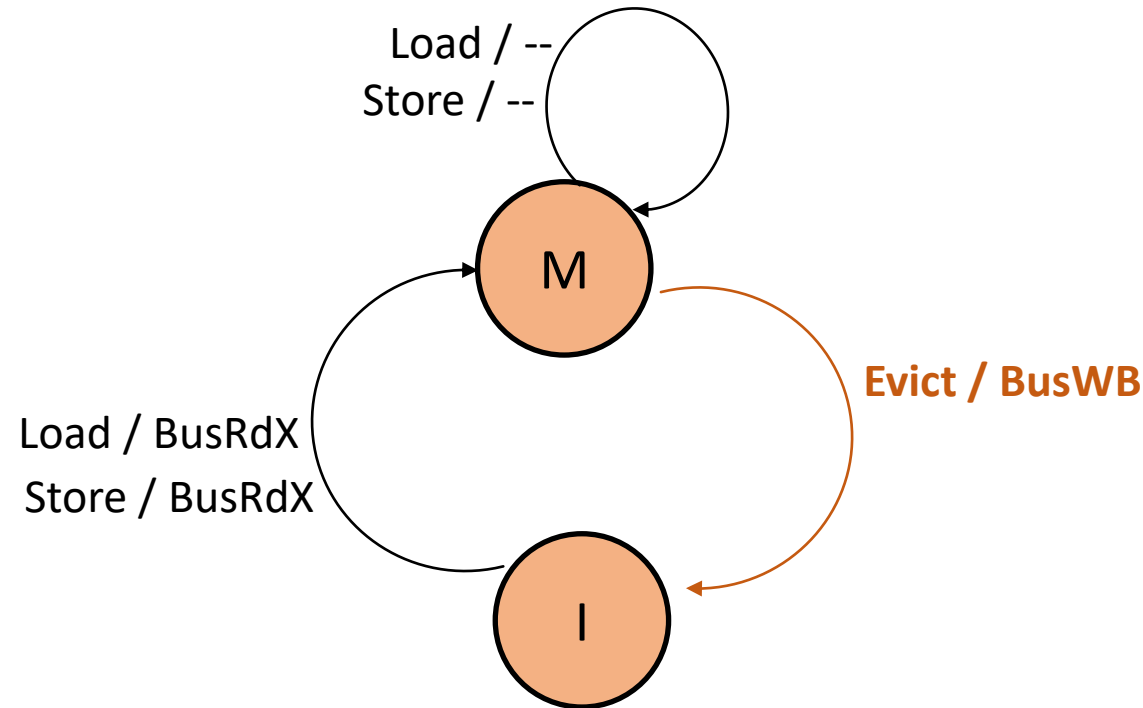
What happens on an Evict?



Snooping-Based Coherence

Modified-Invalid (MI) Protocol

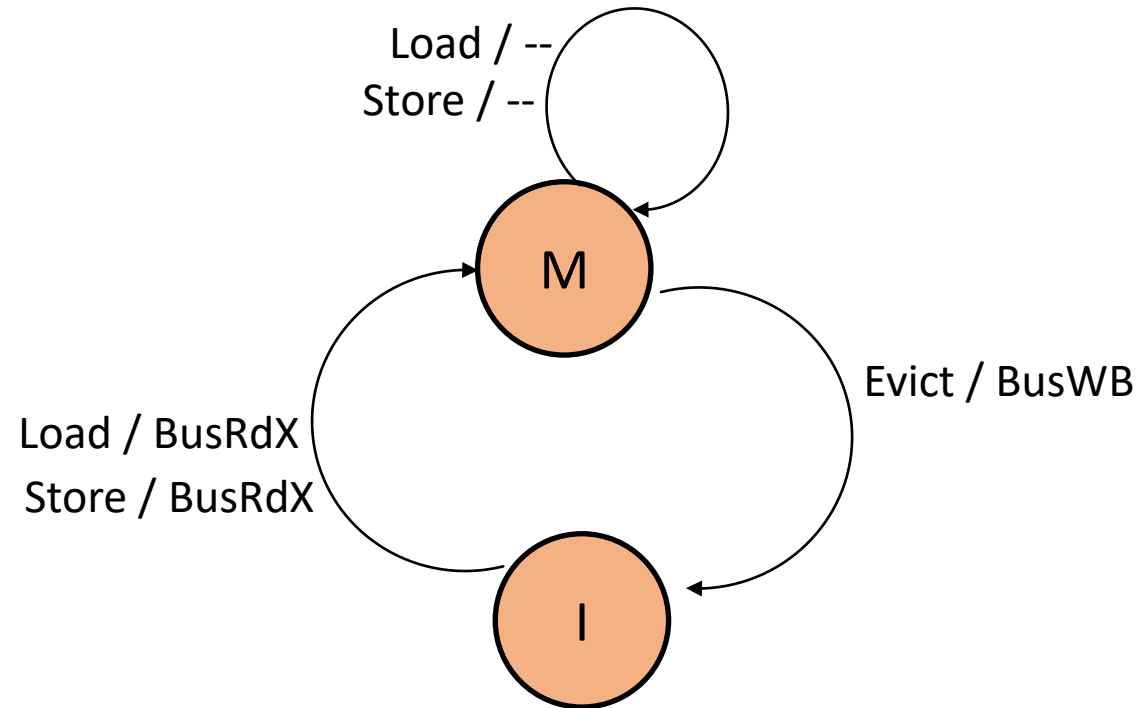
What happens on an Evict?



Snooping-Based Coherence

Modified-Invalid (MI) Protocol

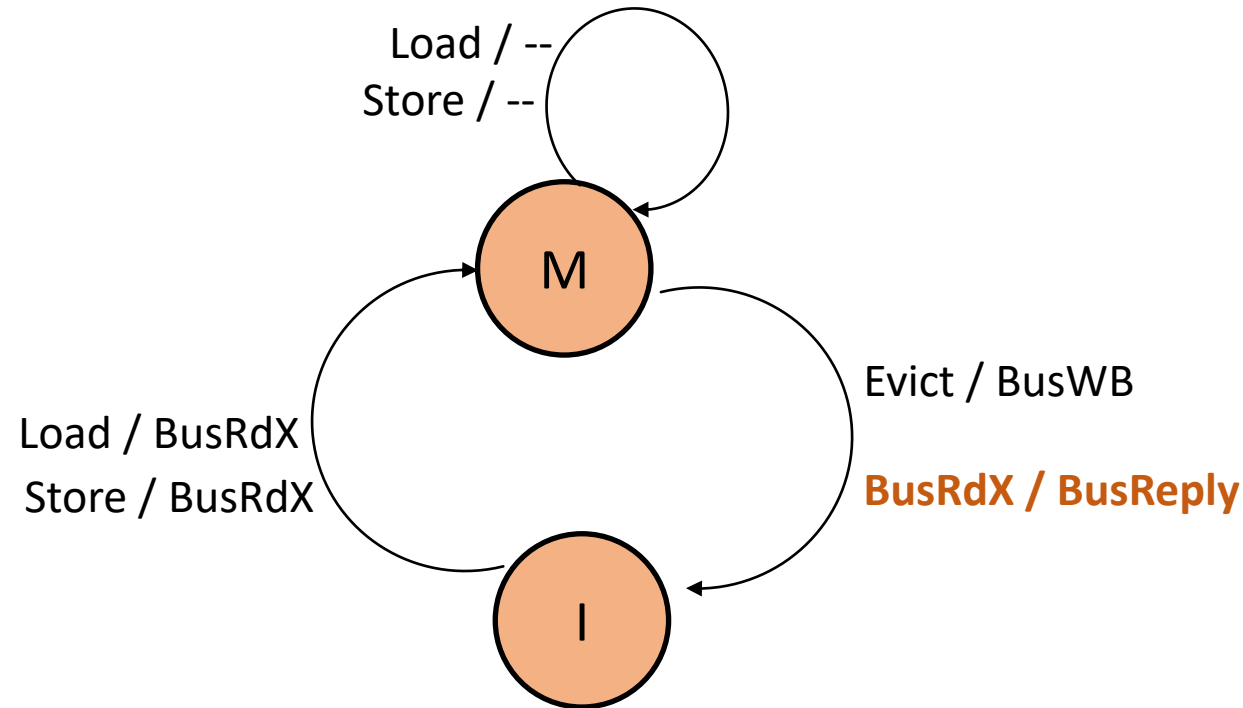
What happens on a BusRdX?



Snooping-Based Coherence

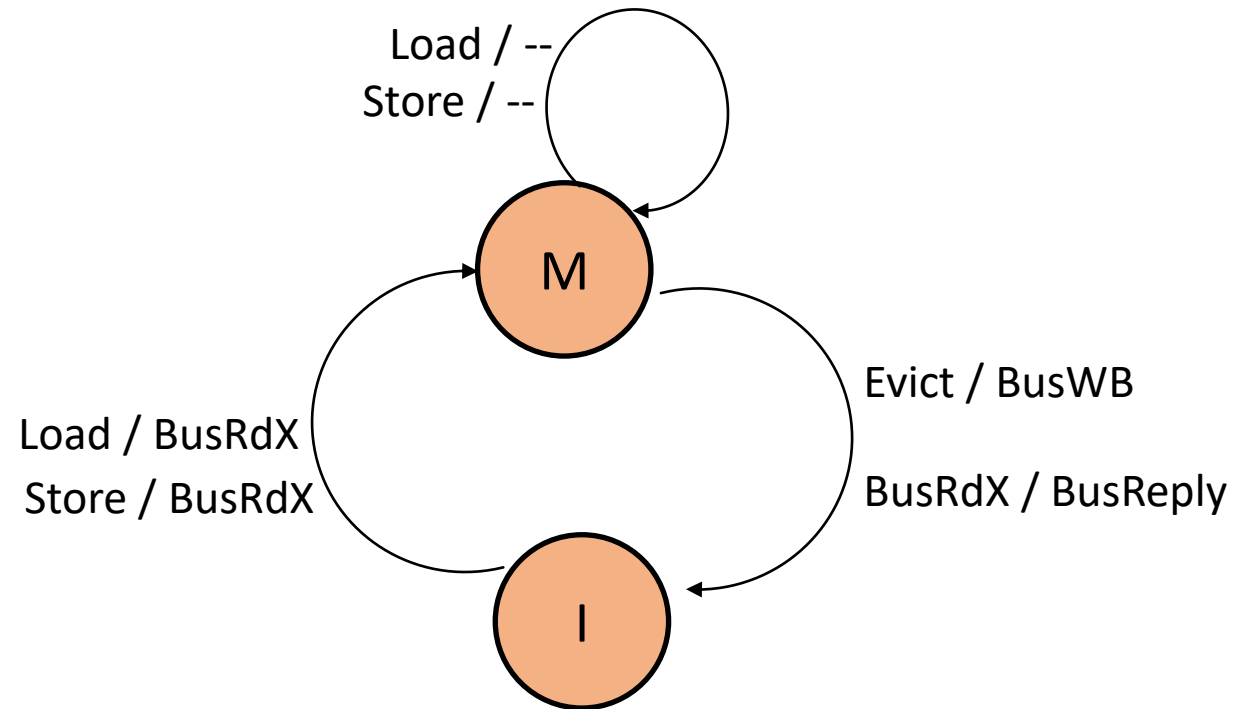
Modified-Invalid (MI) Protocol

What happens on a BusRdX?



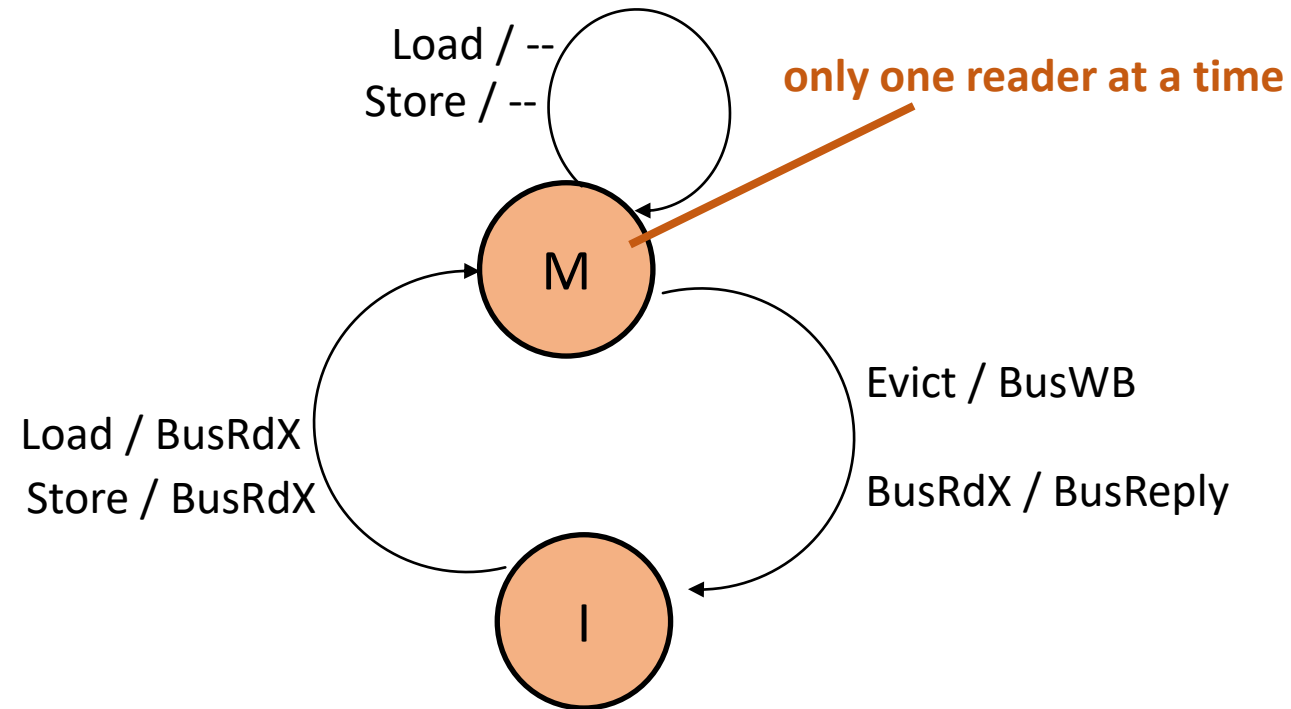
Snooping-Based Coherence

Modified-Invalid (MI) Protocol



Snooping-Based Coherence

Modified-Invalid (MI) Protocol



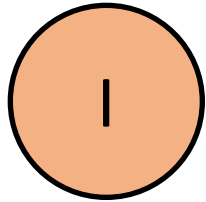
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol

- Write-back, write-allocate caches
- *Modified* state: Reads and writes allowed; cache has only copy of block
- *Shared* state: Reads allowed; cache has a copy of block
- *Invalid* state: Block is not cached
- *BusInv*: Request exclusive access to block, but already have copy of block

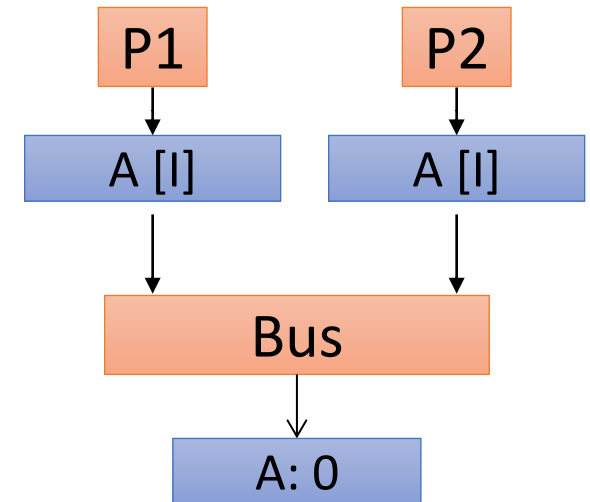
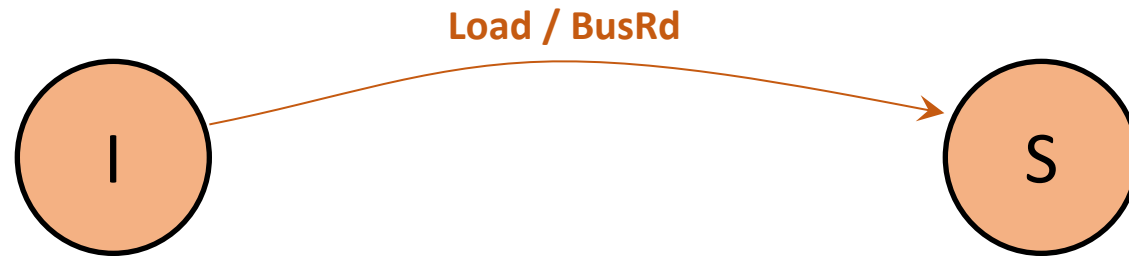
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



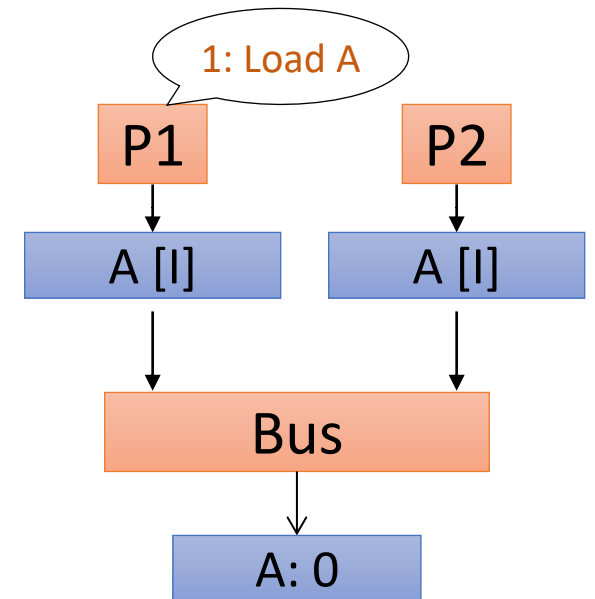
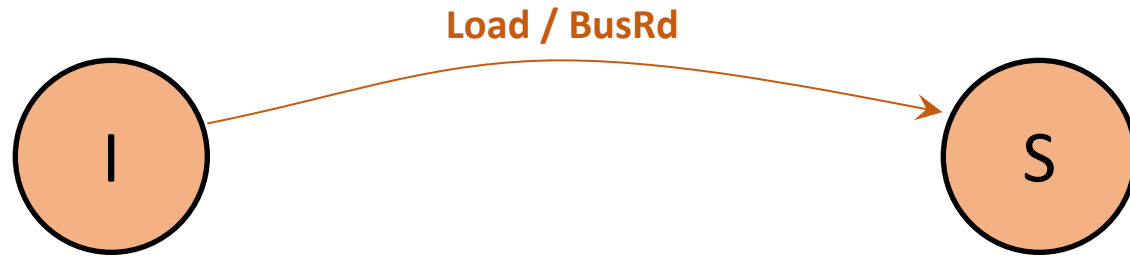
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



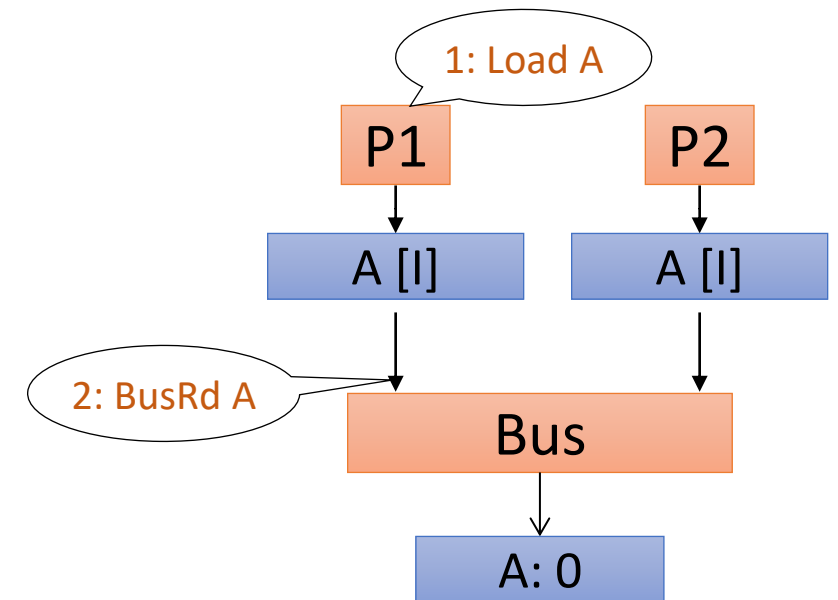
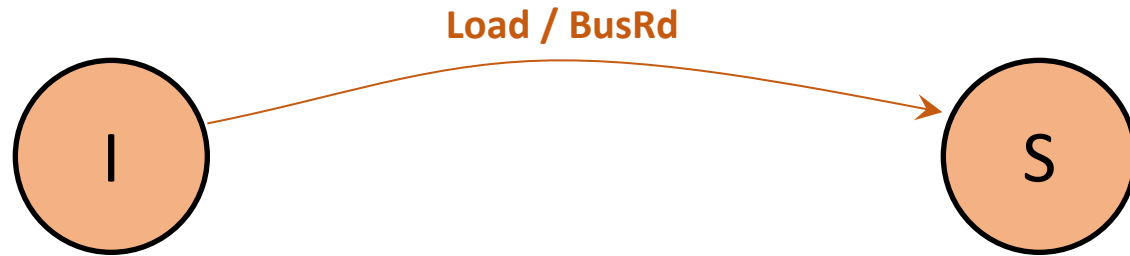
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



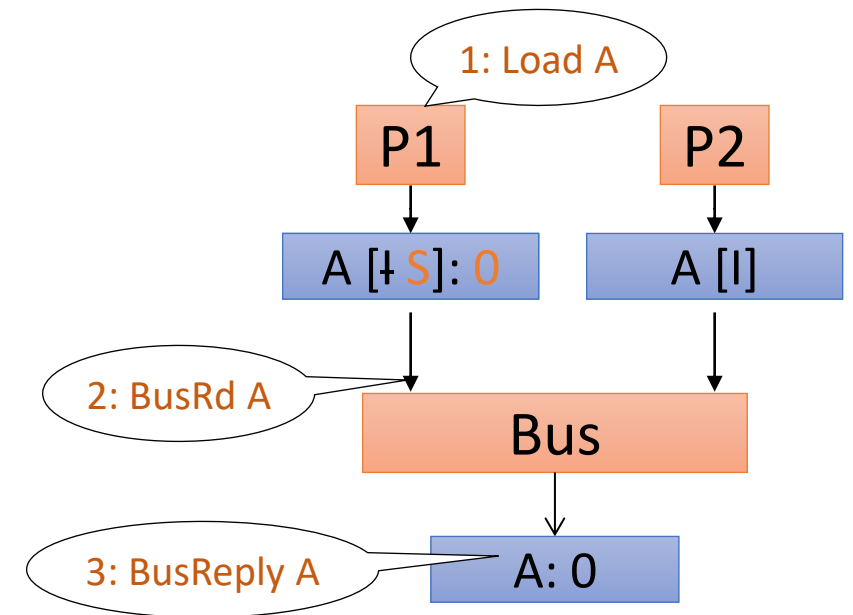
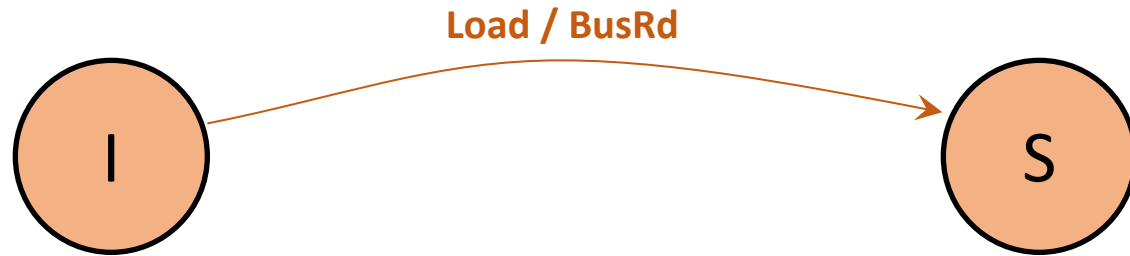
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



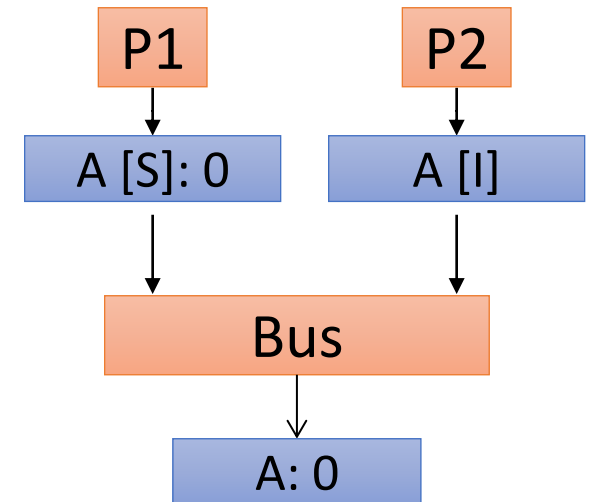
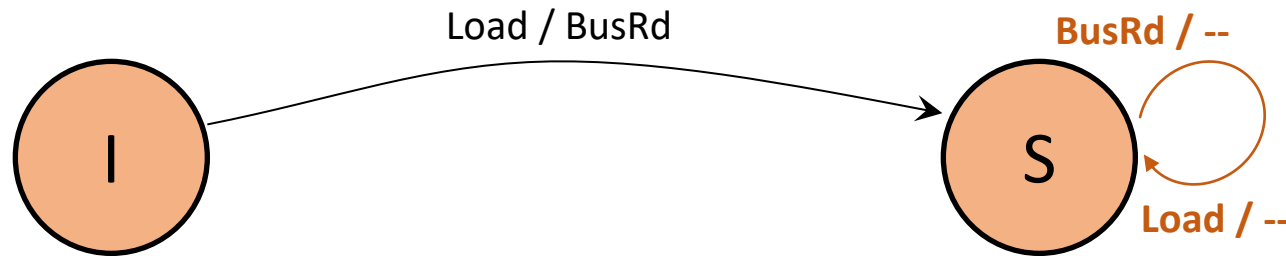
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



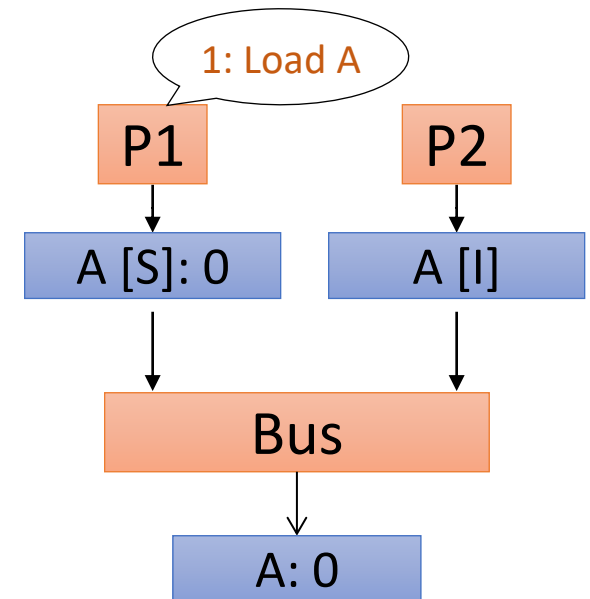
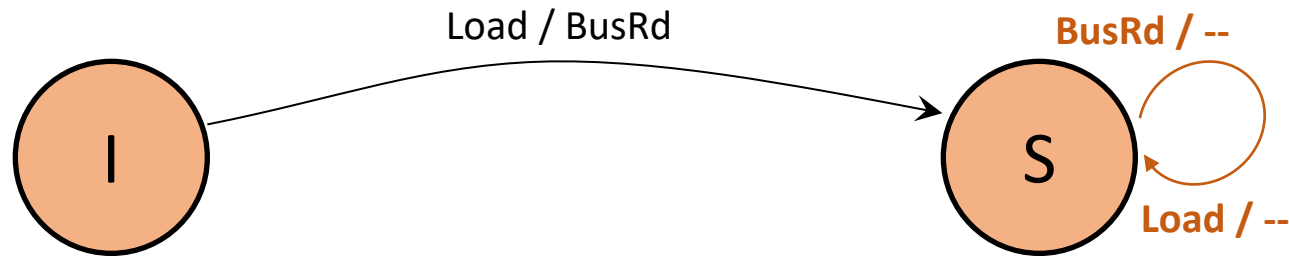
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



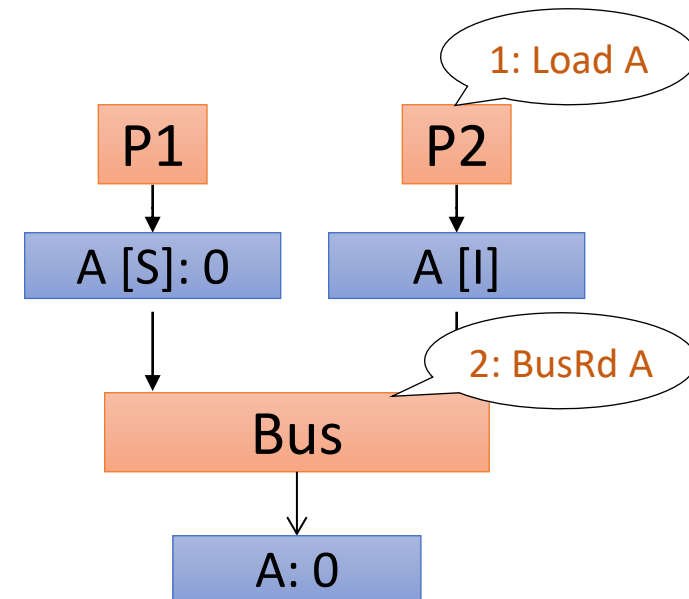
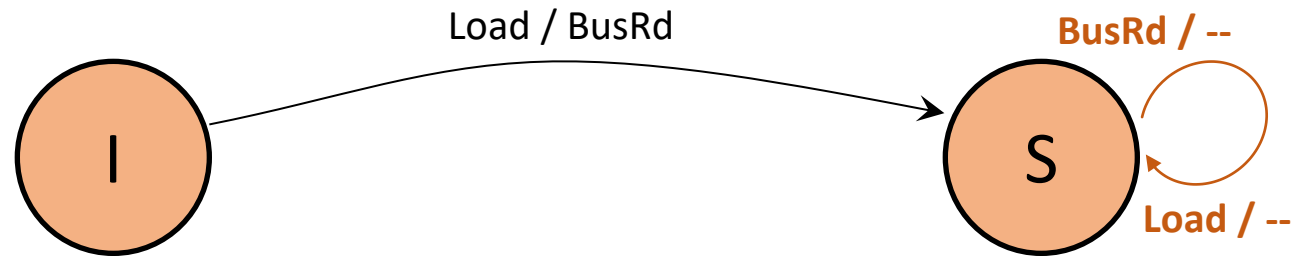
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



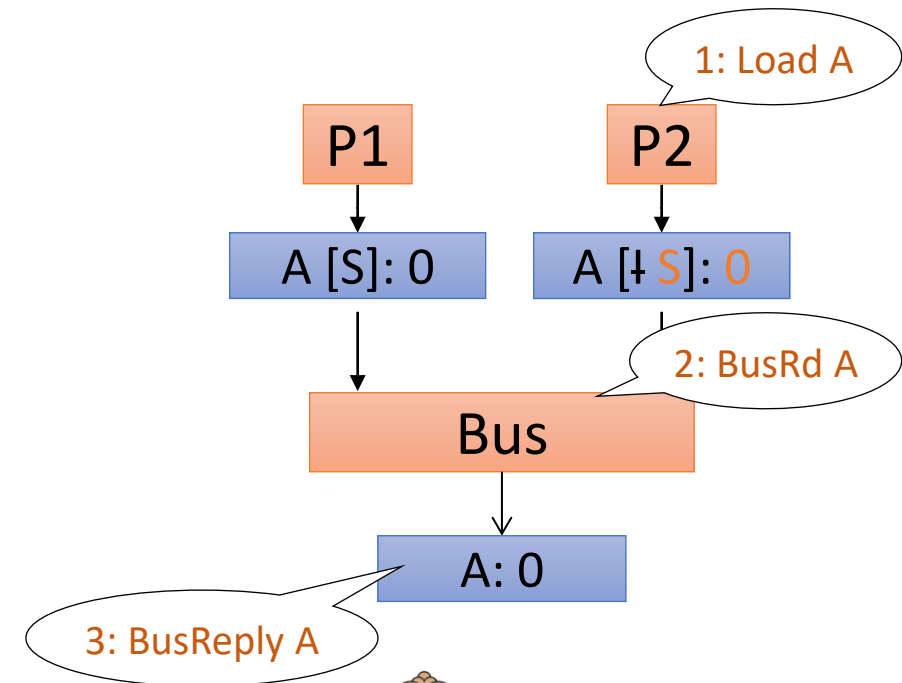
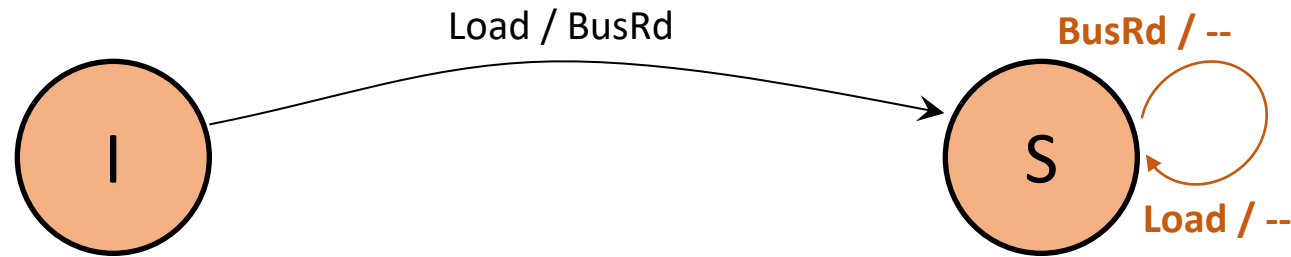
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



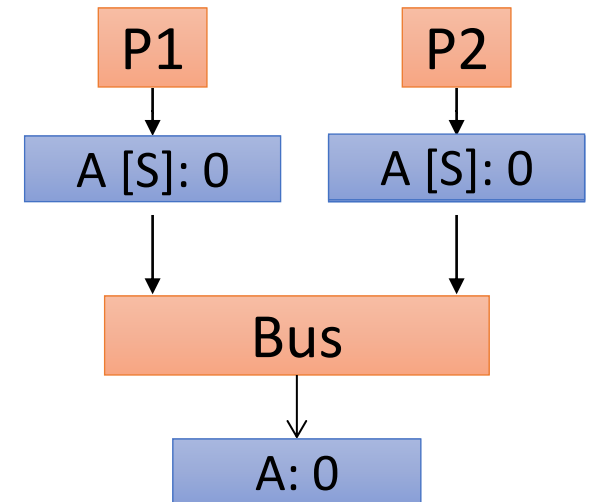
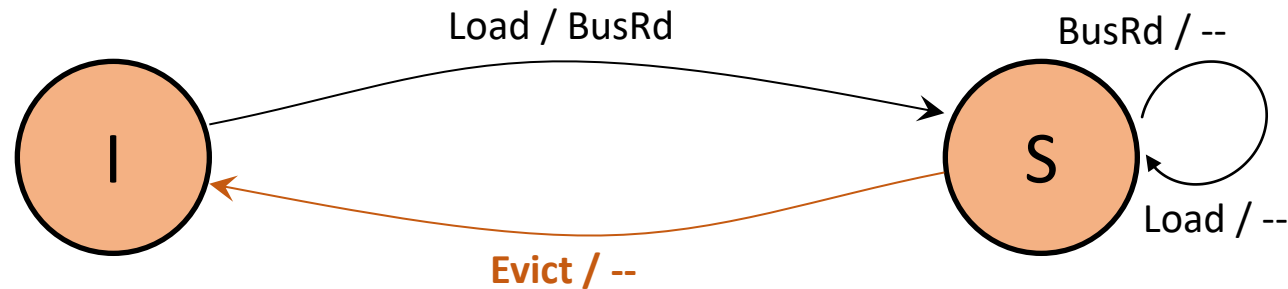
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



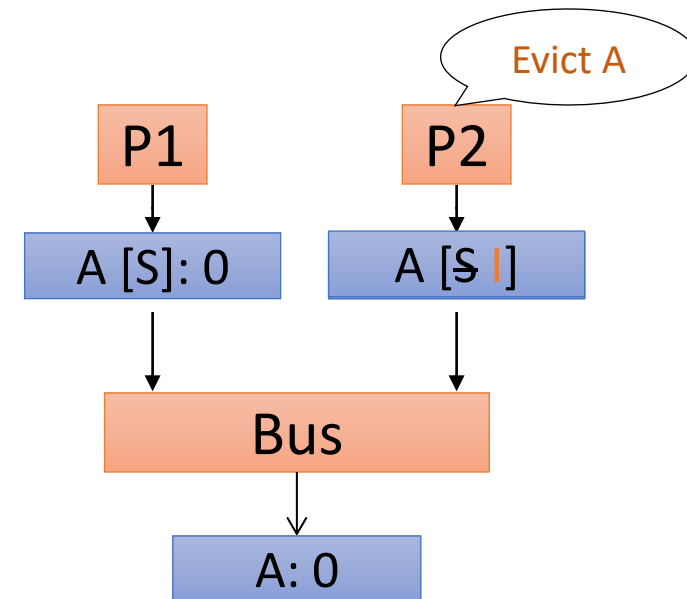
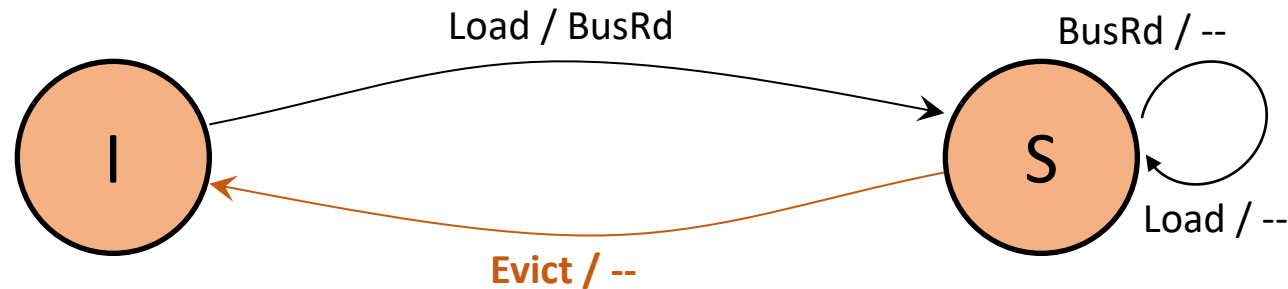
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



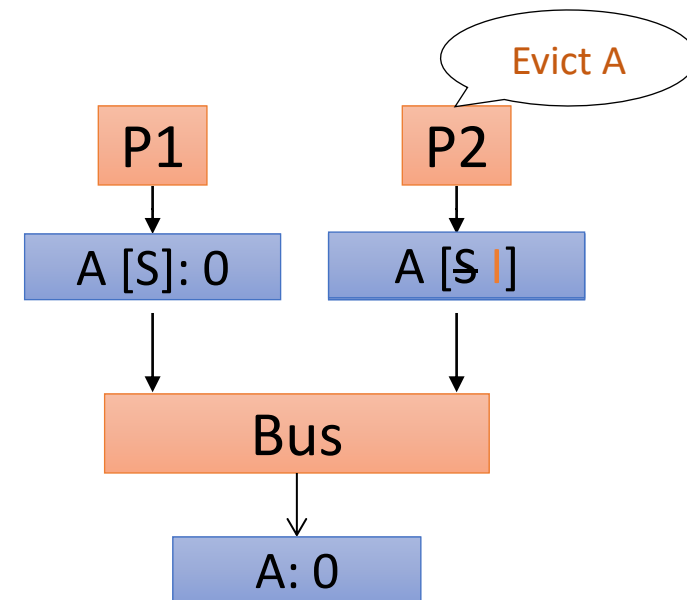
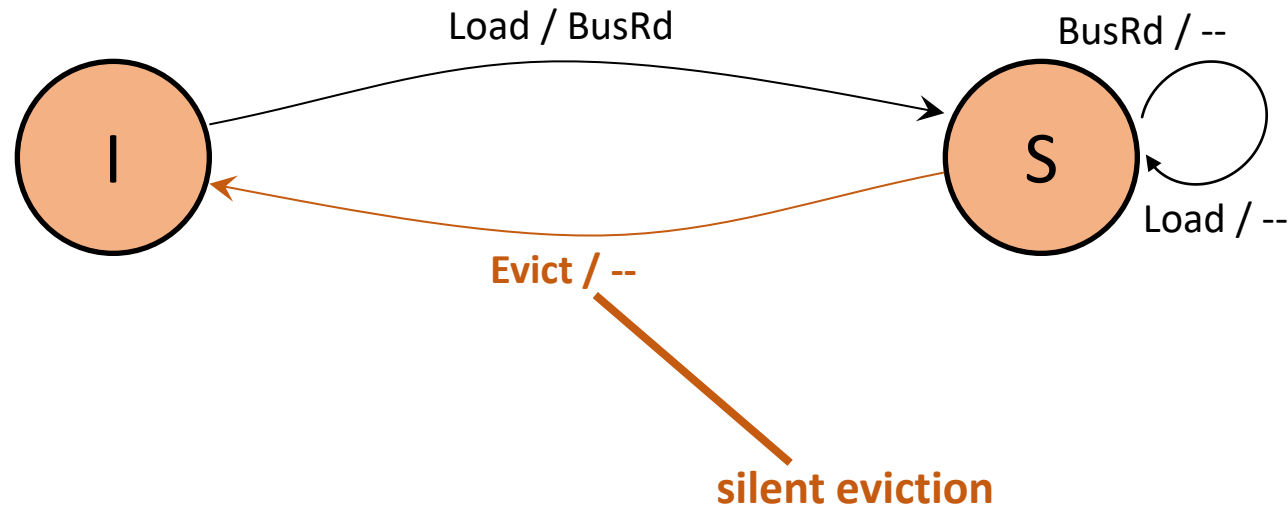
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



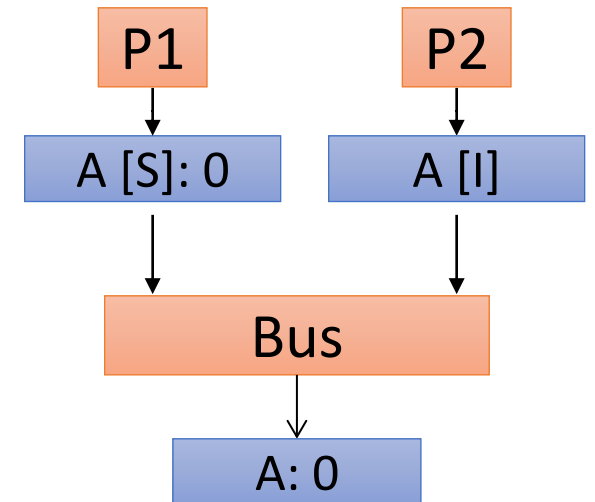
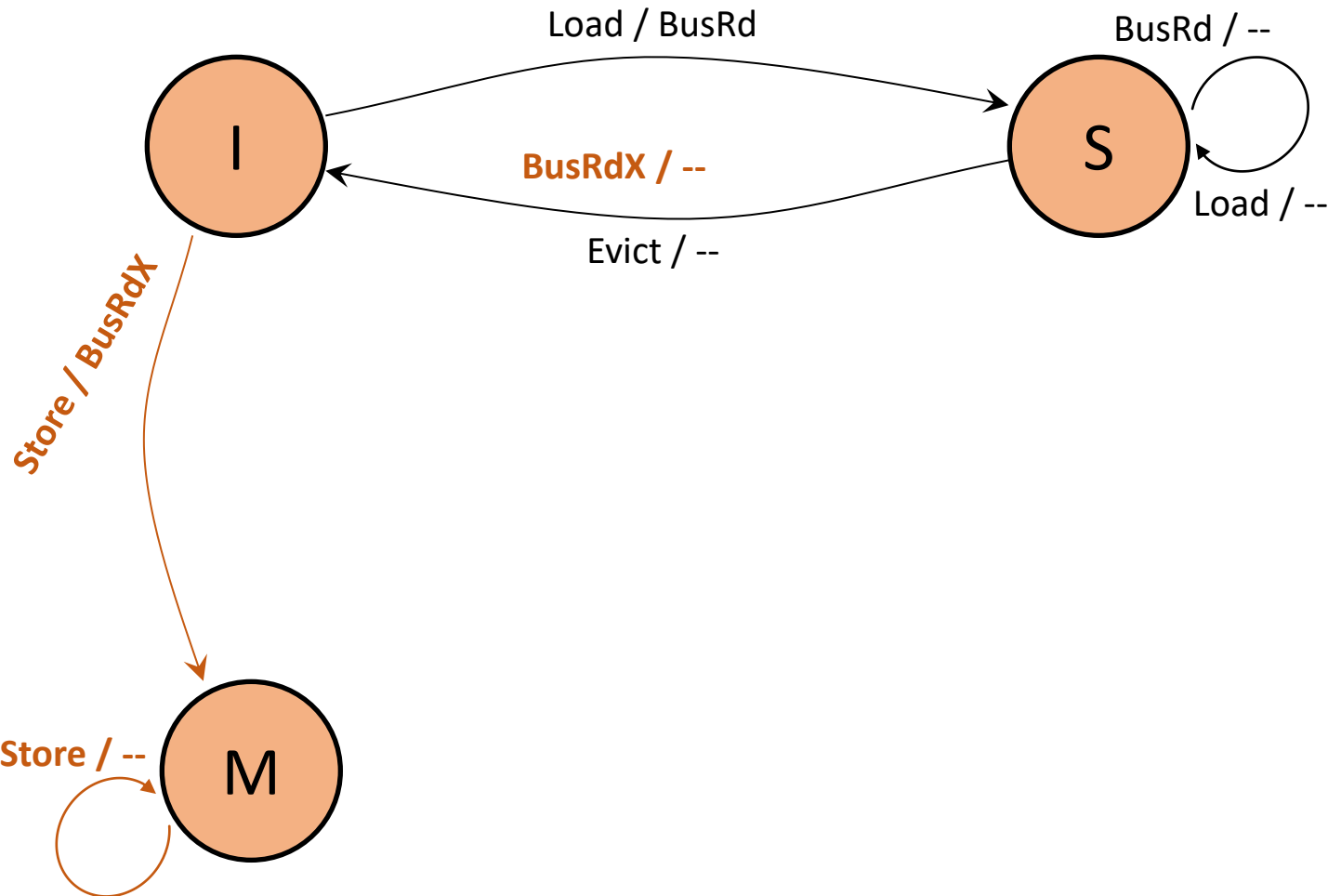
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



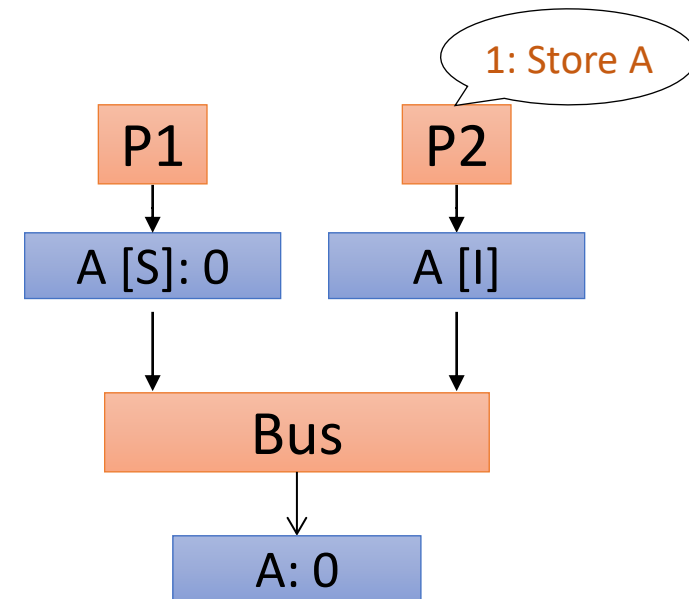
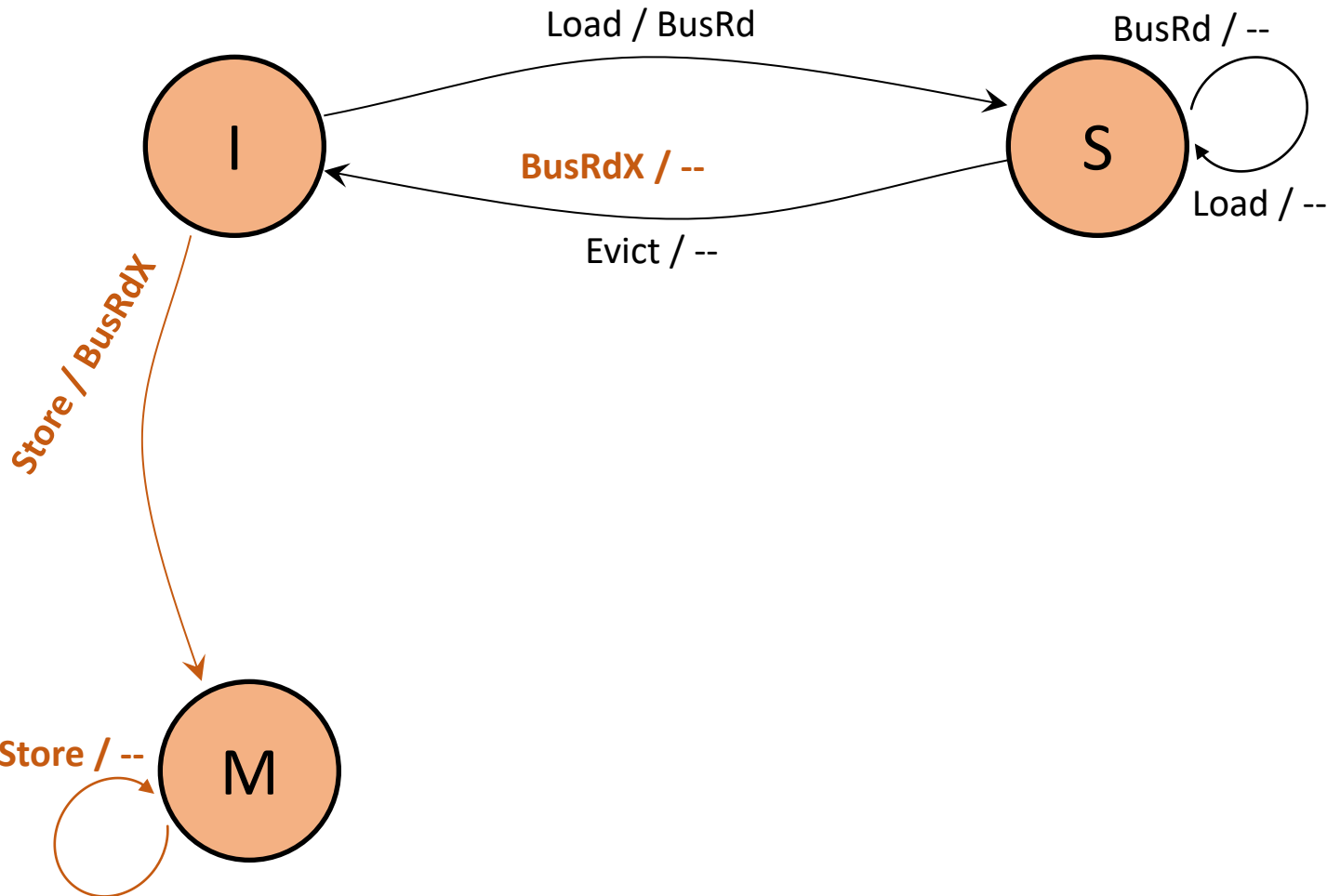
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



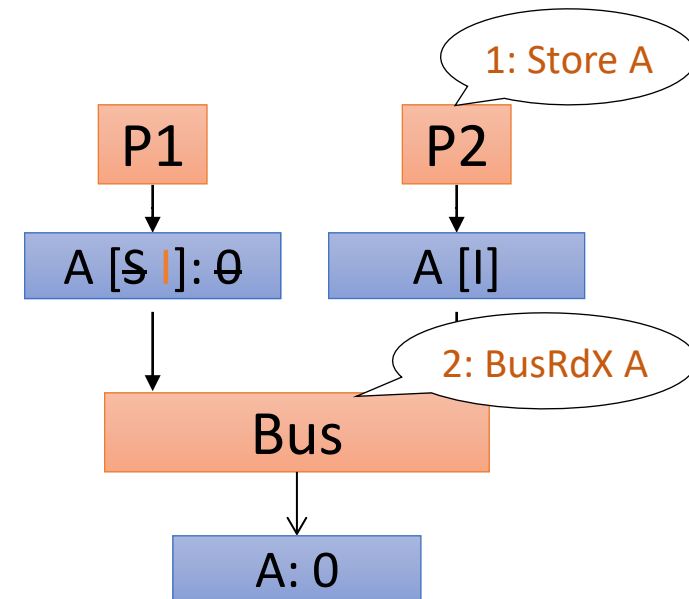
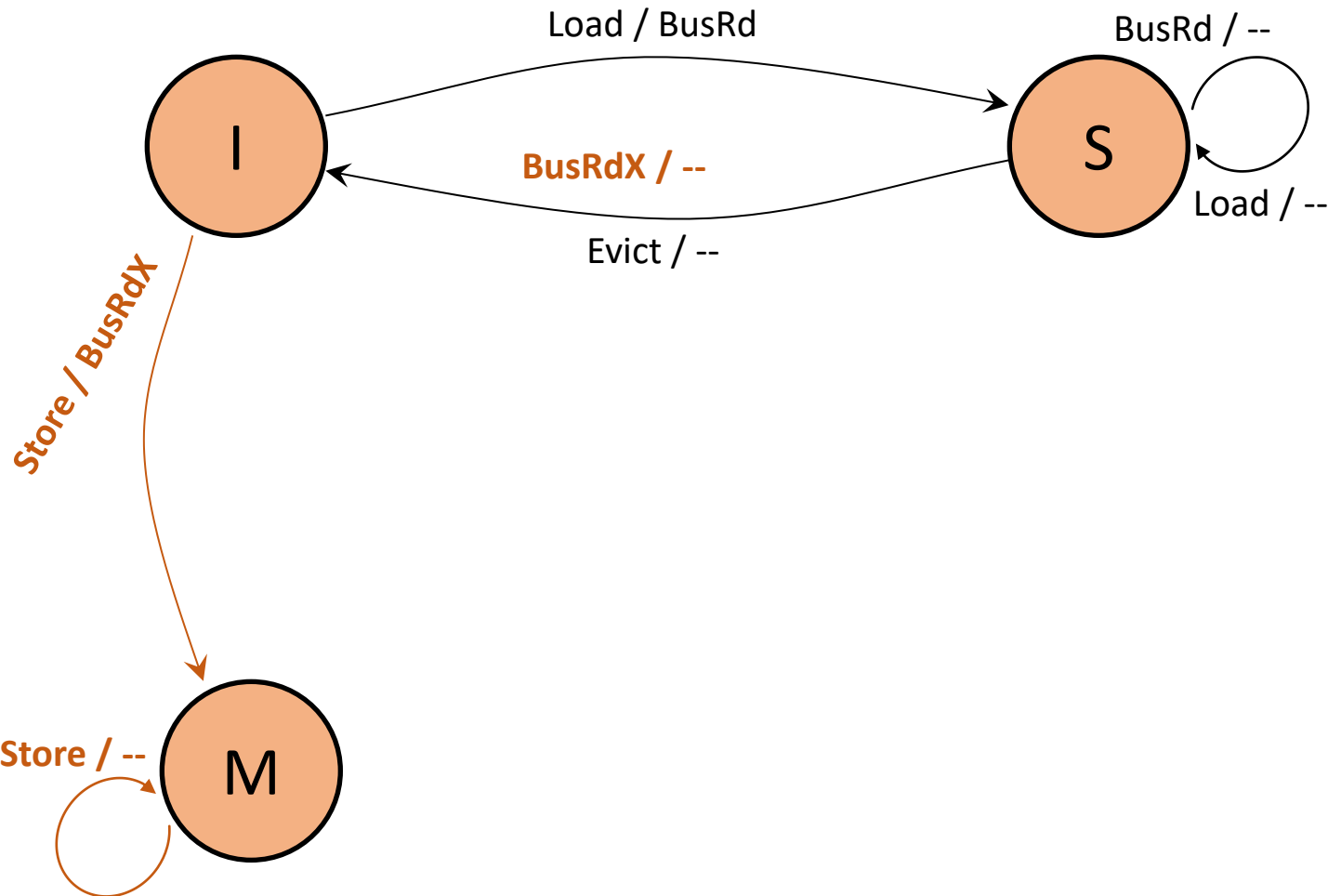
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



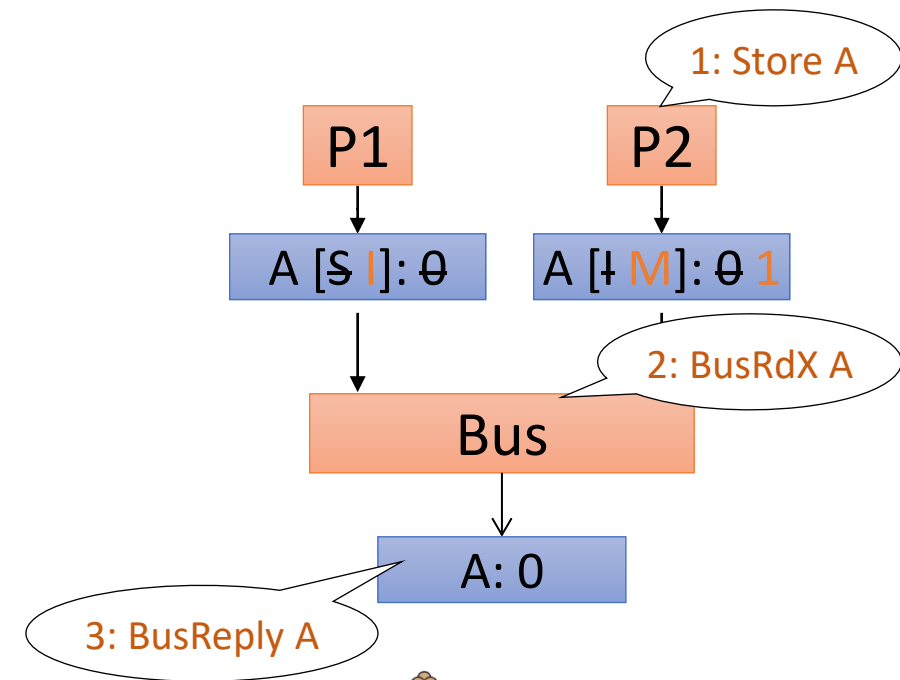
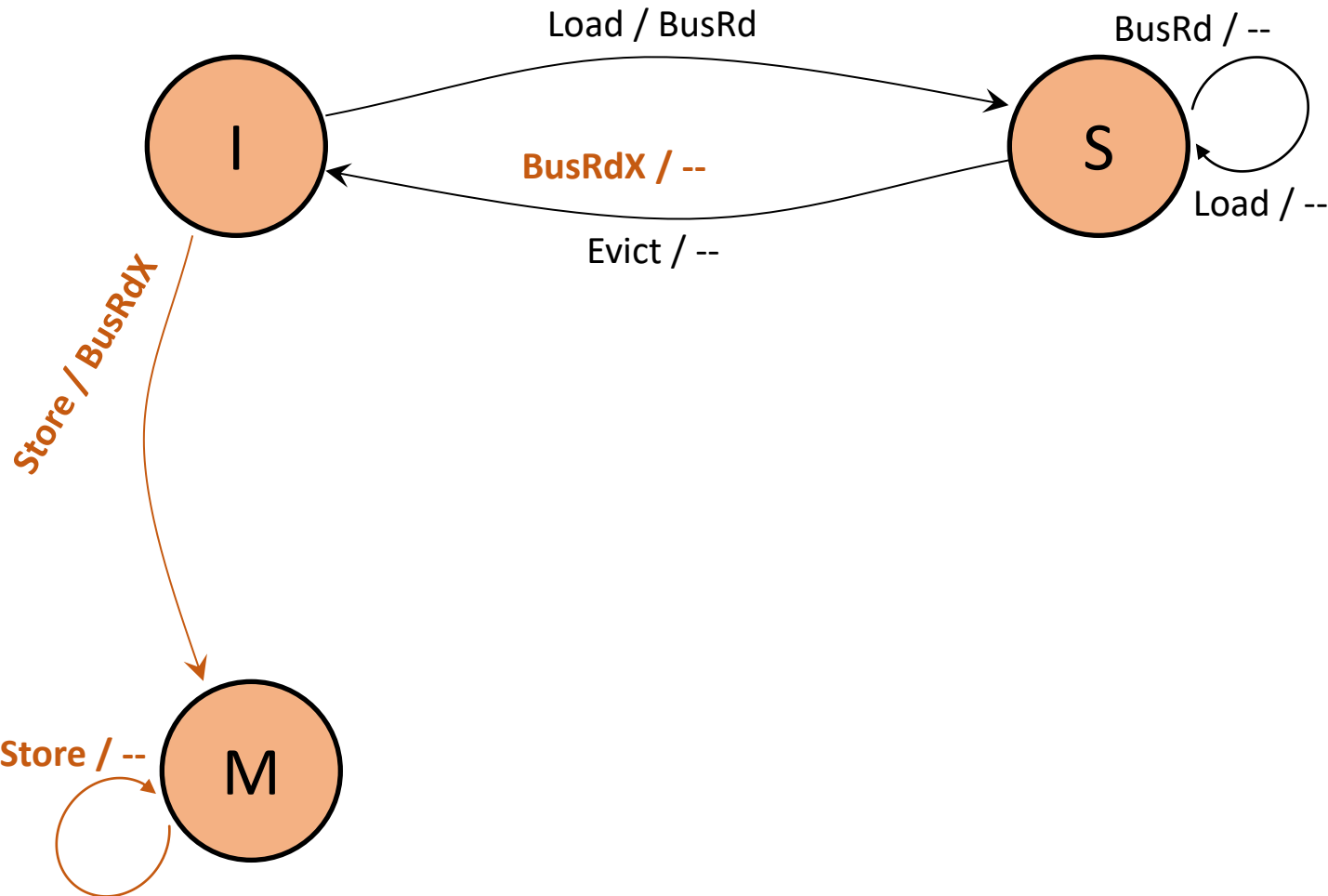
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



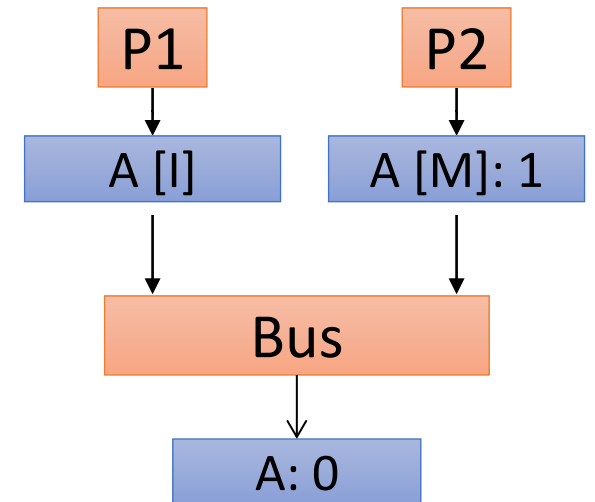
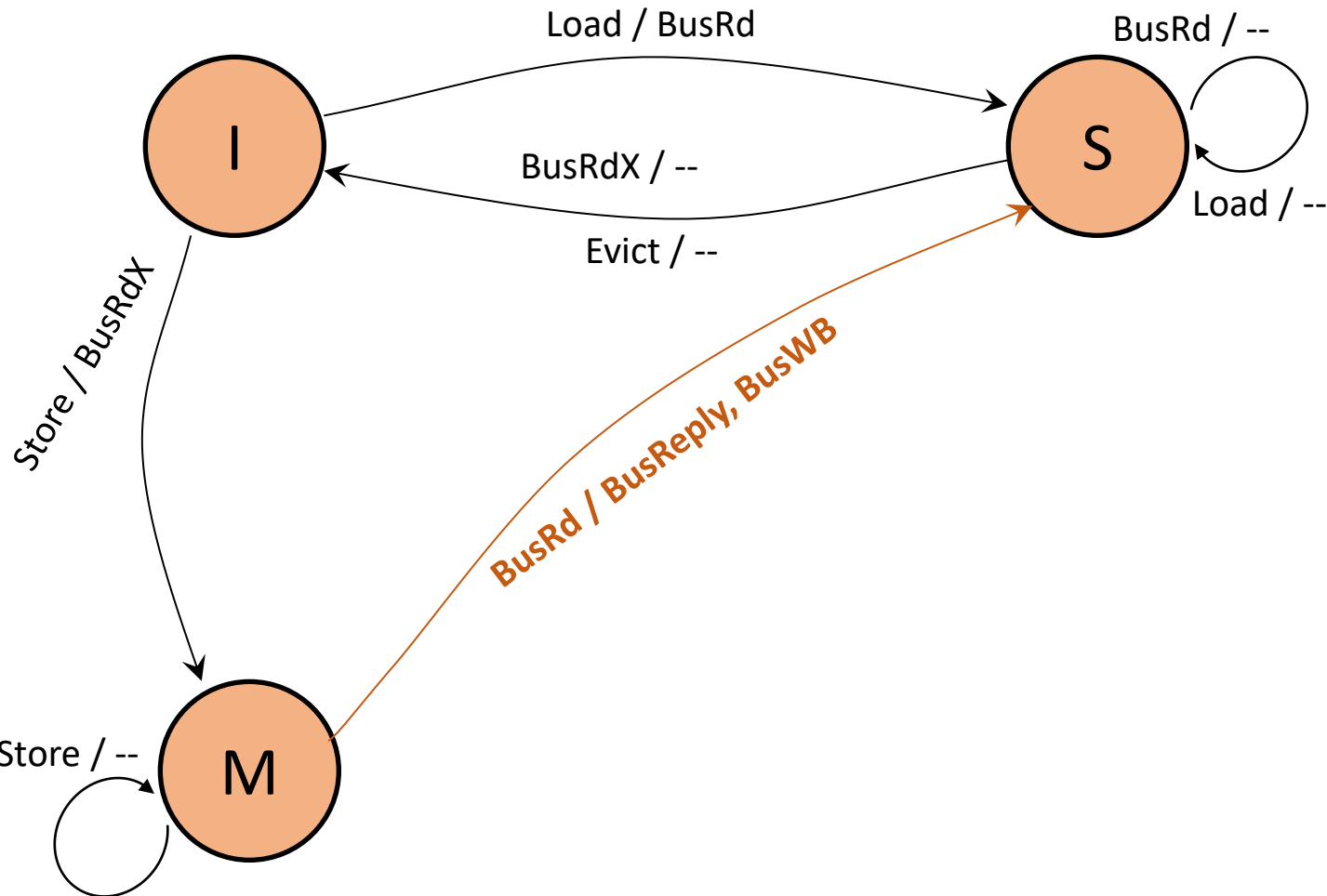
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



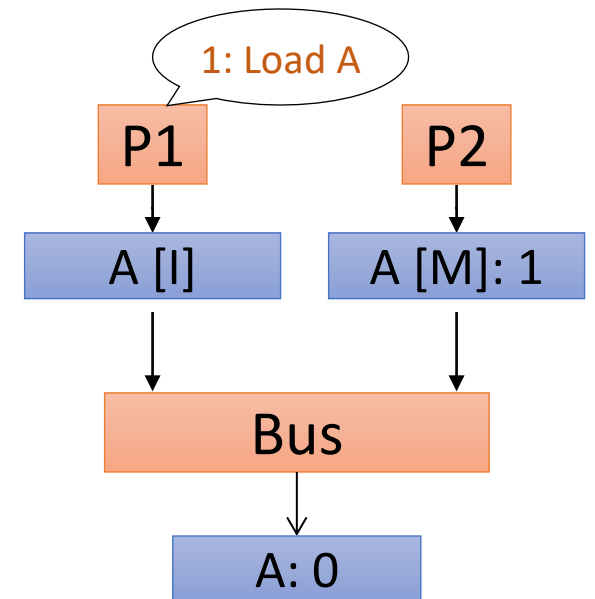
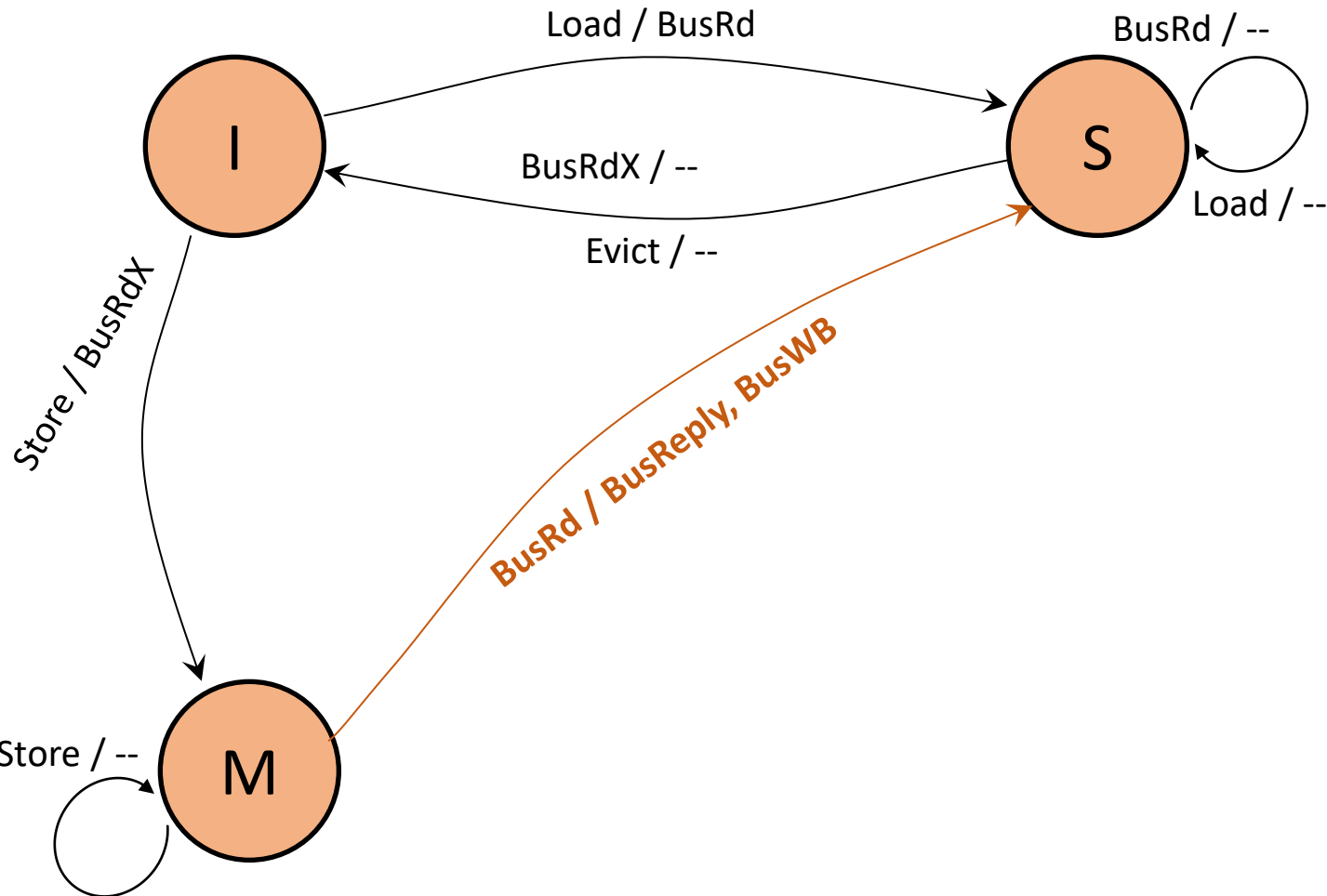
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



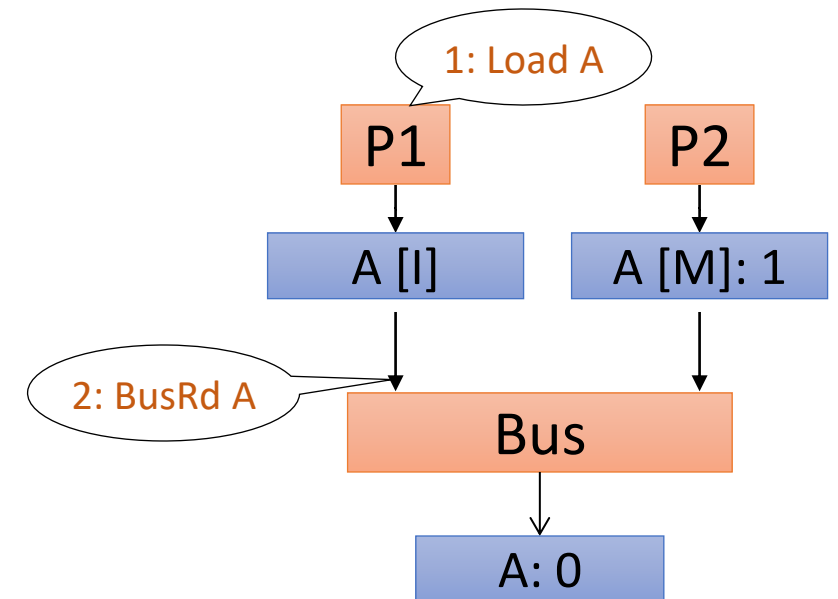
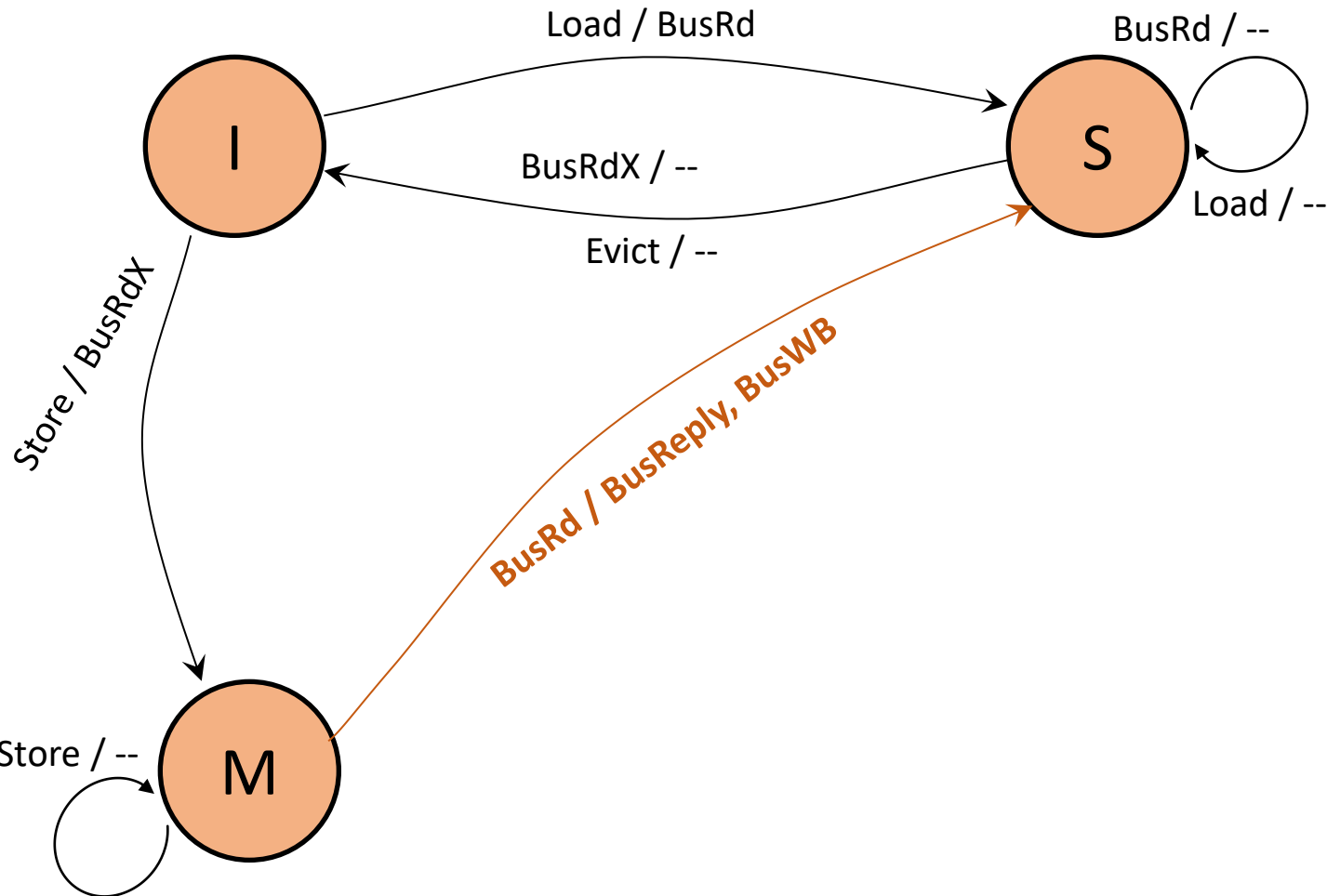
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



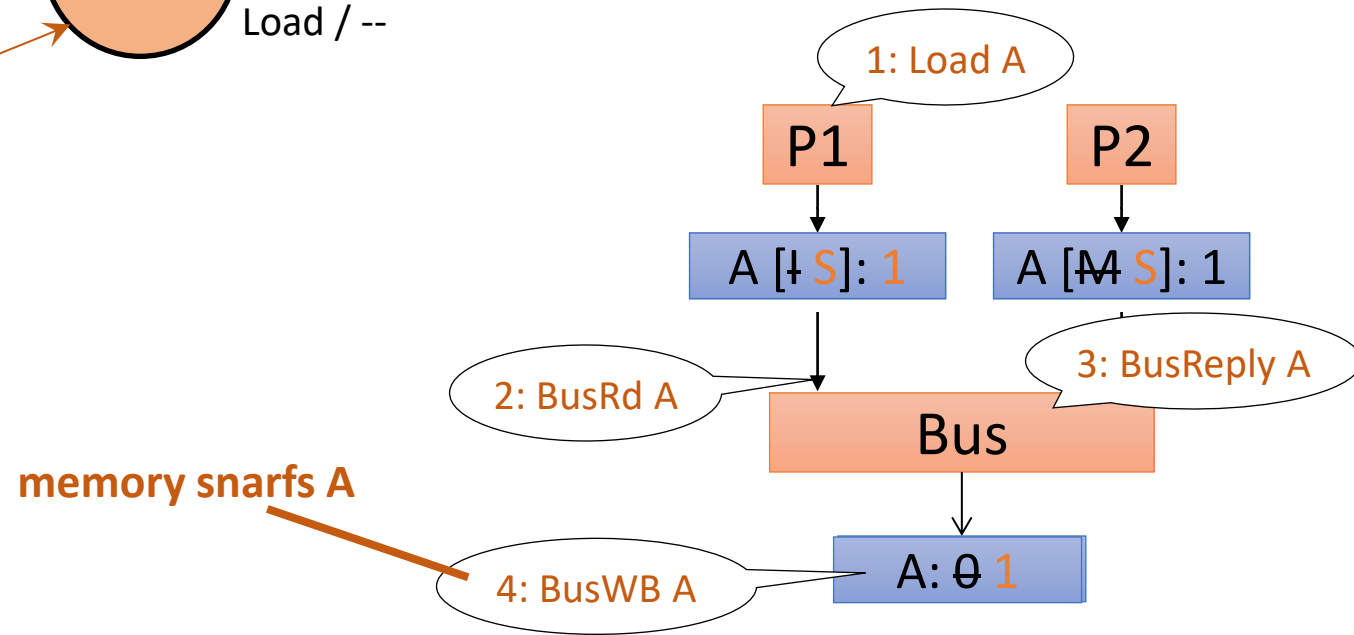
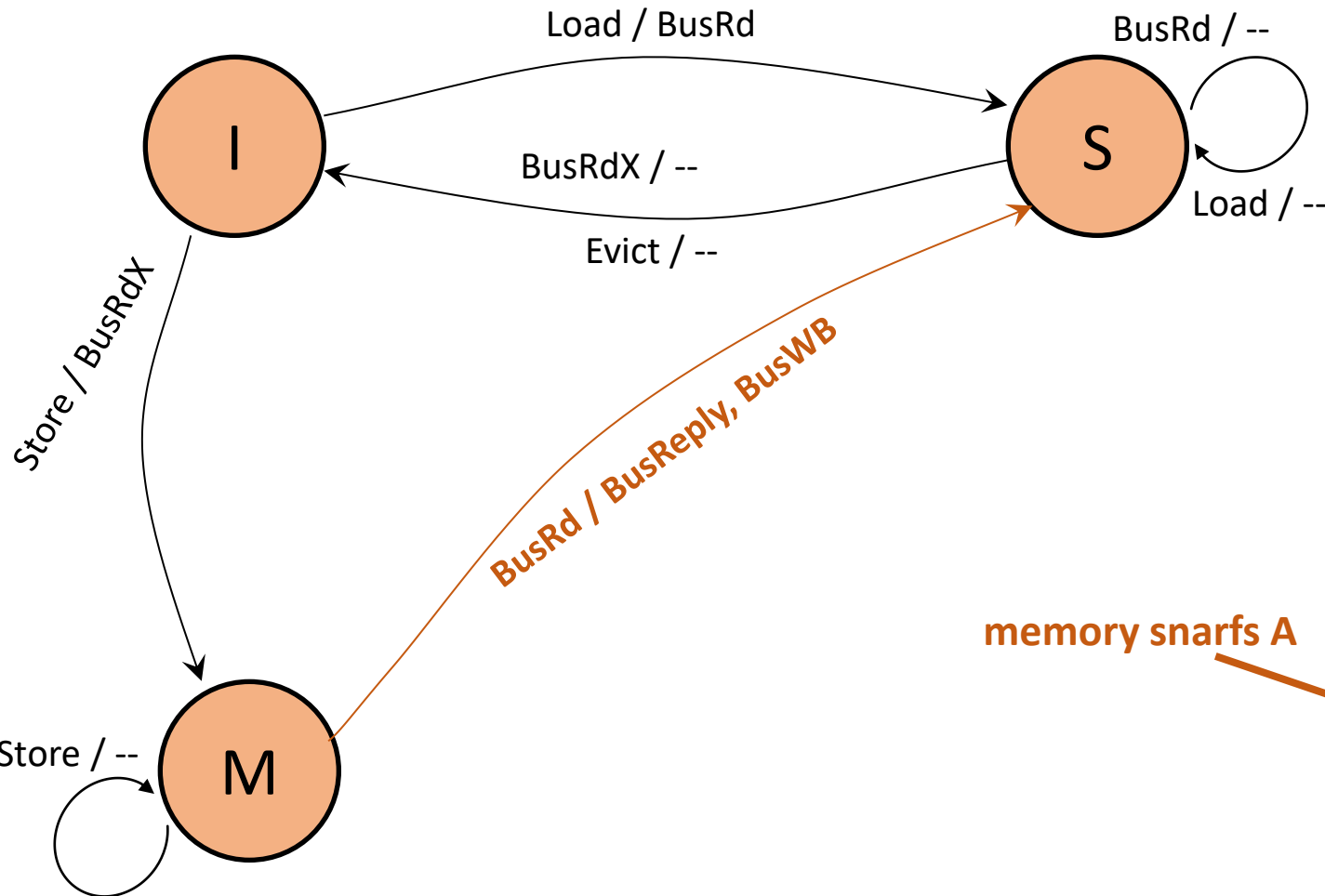
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



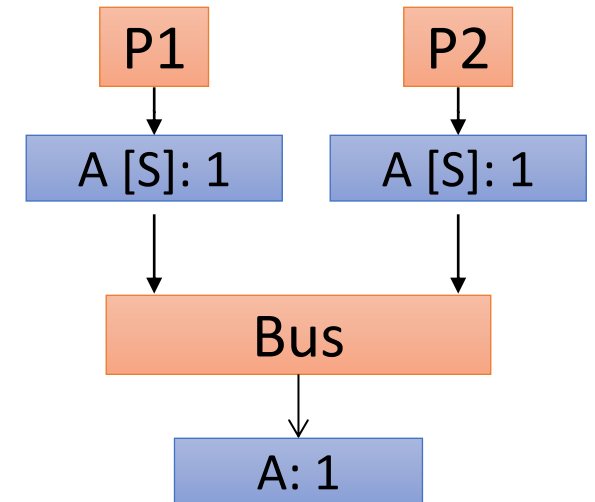
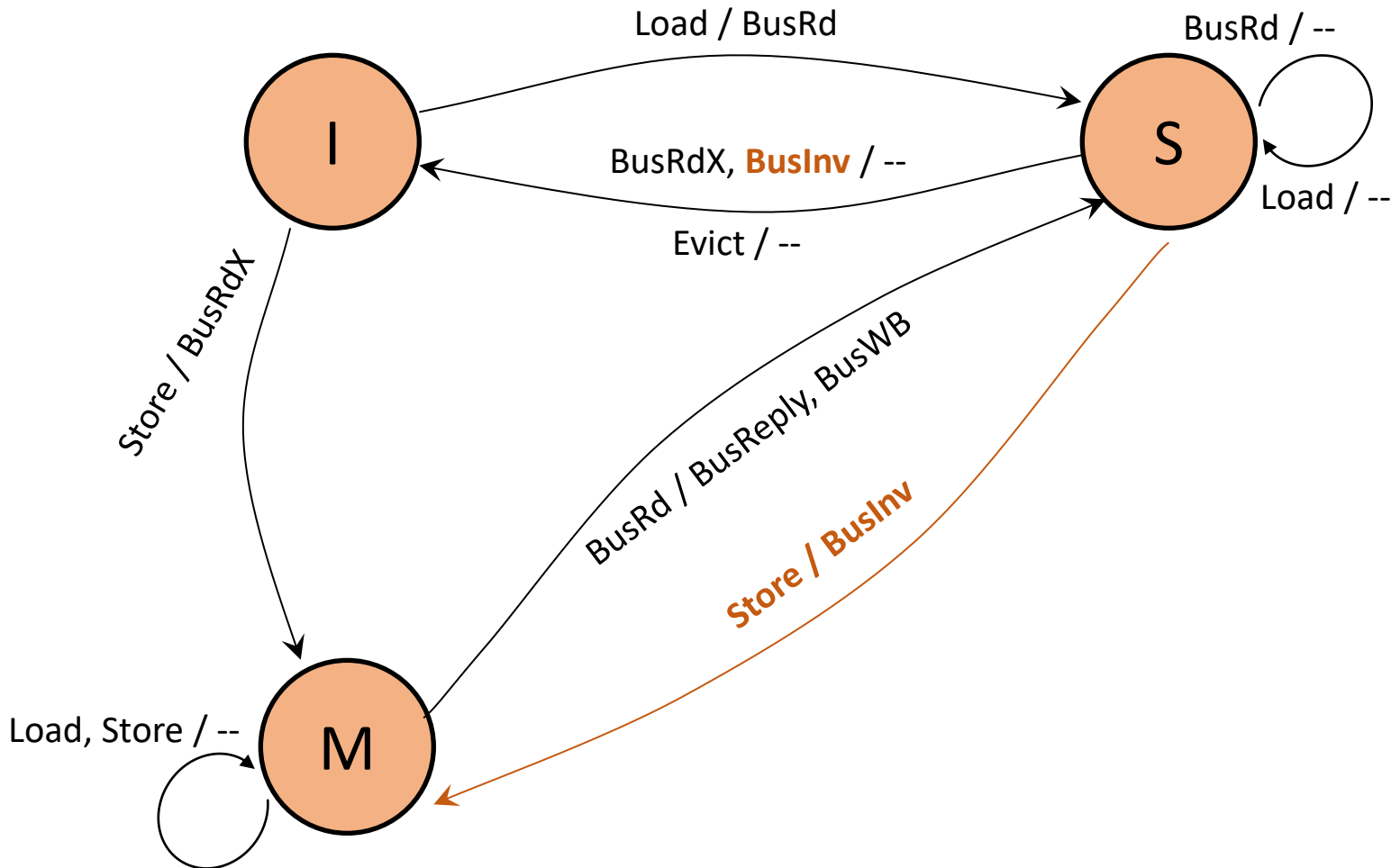
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



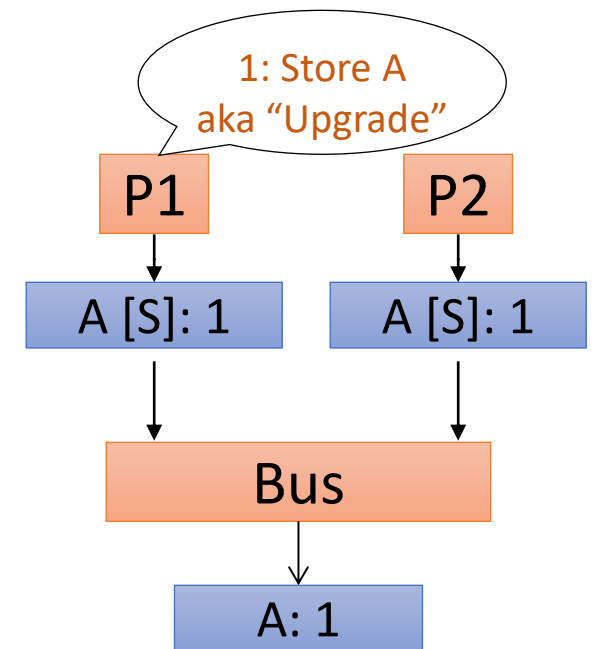
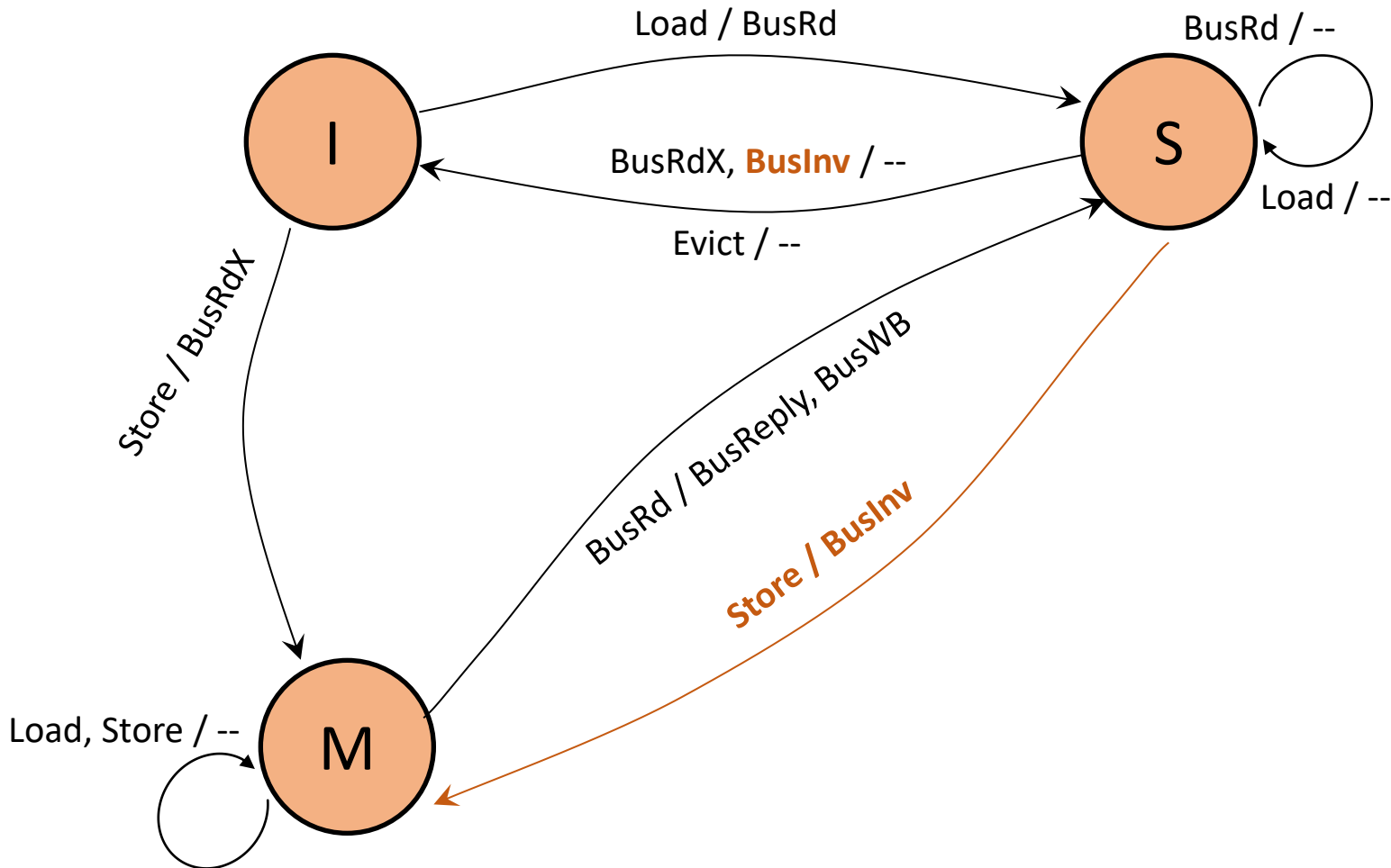
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



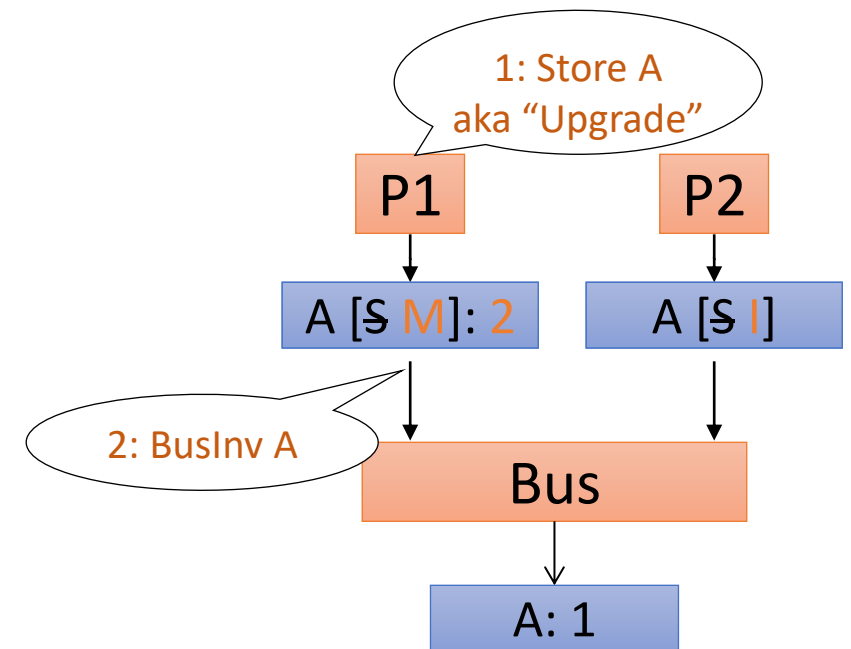
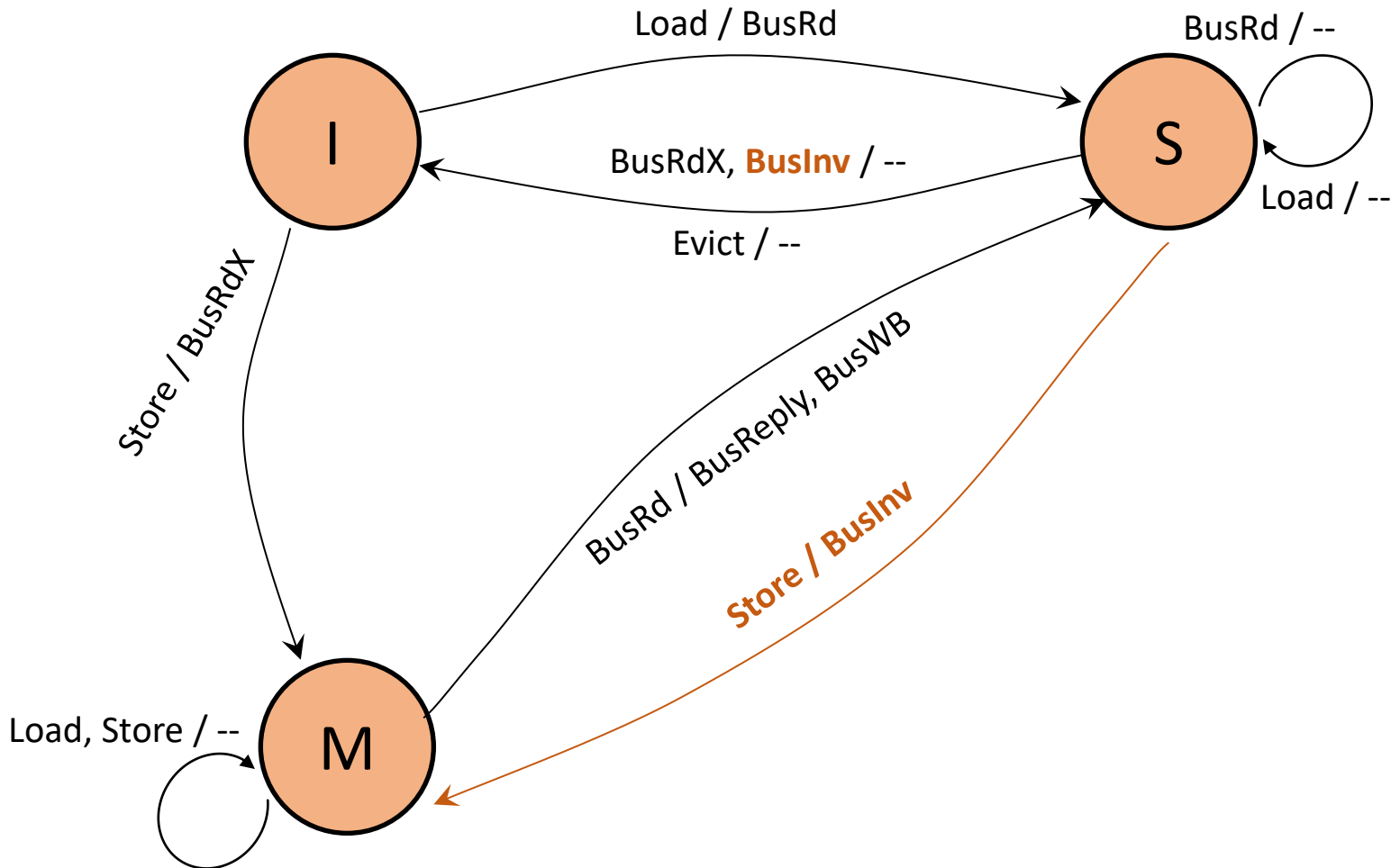
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



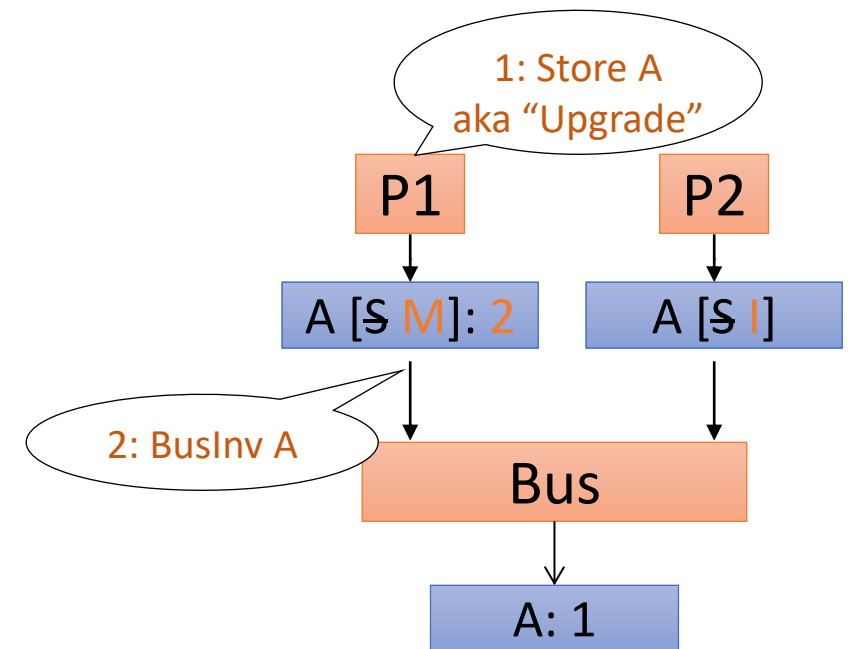
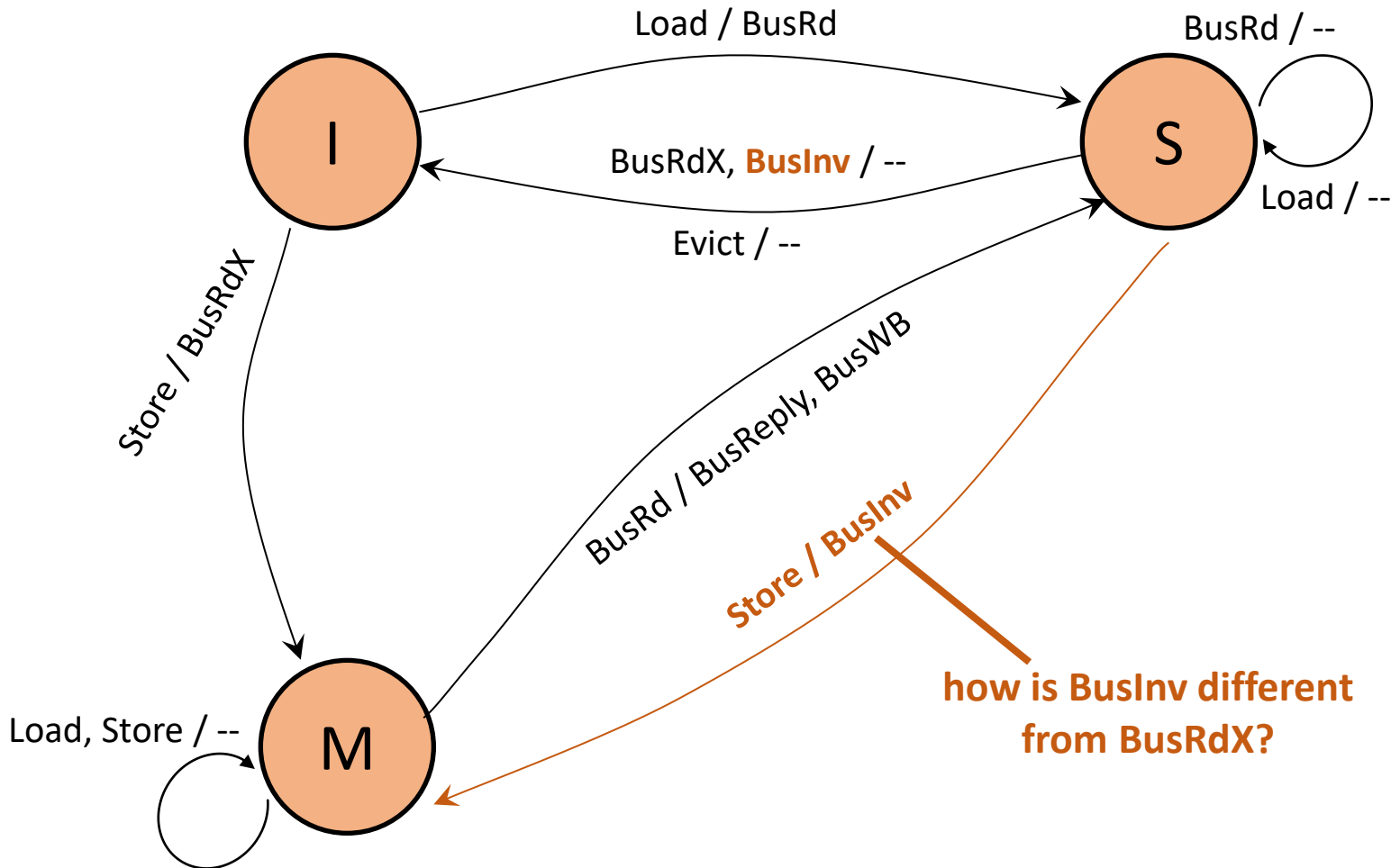
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



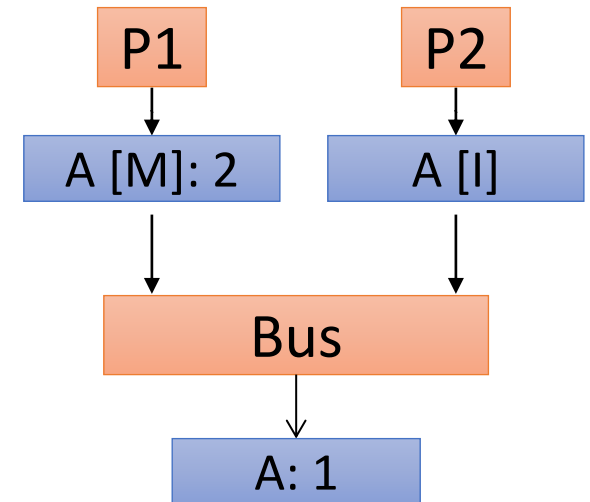
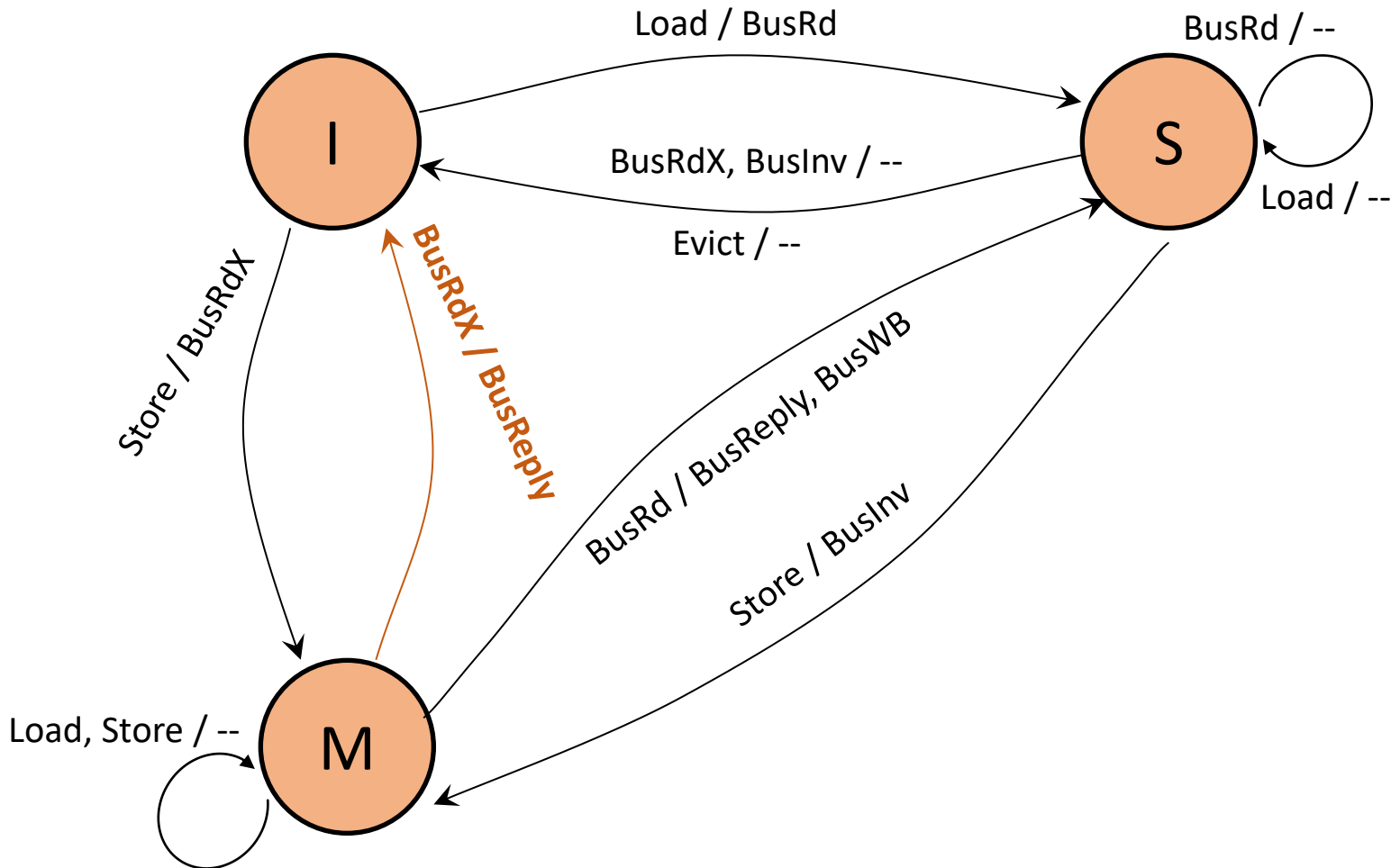
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



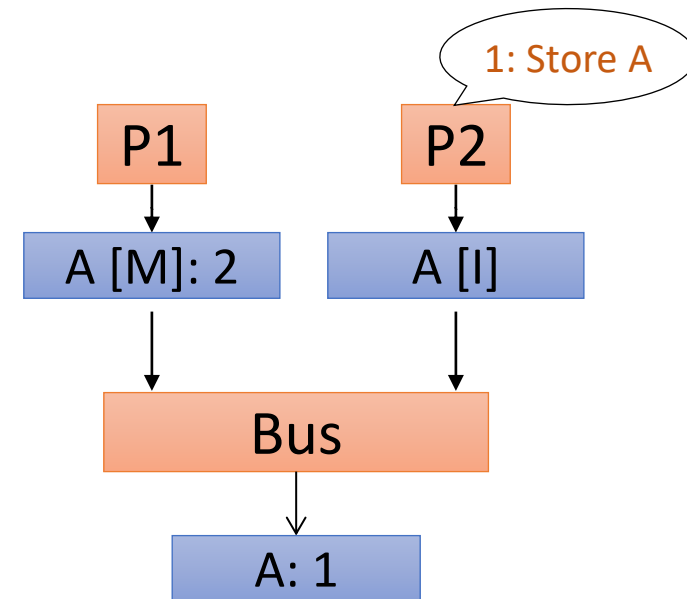
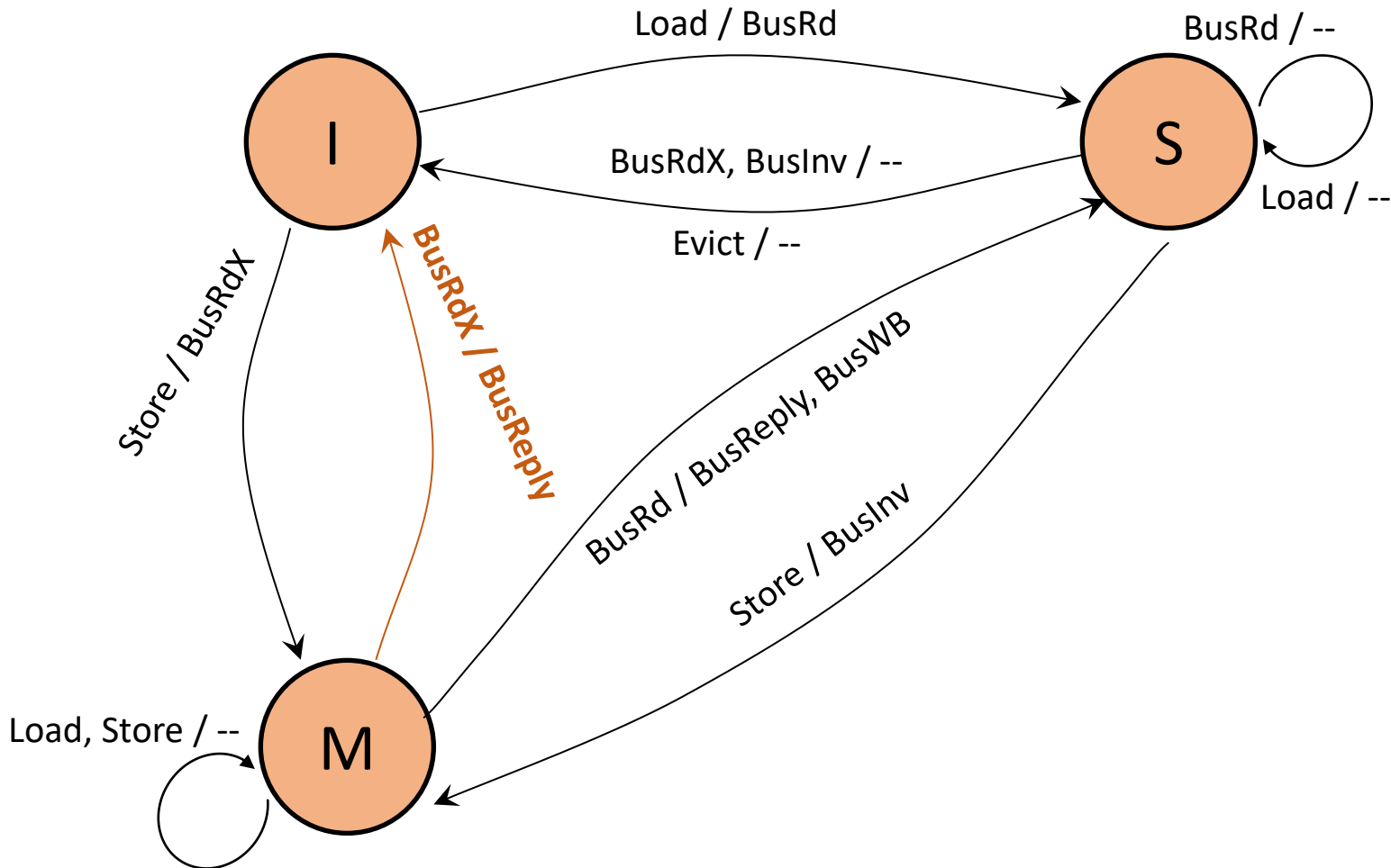
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



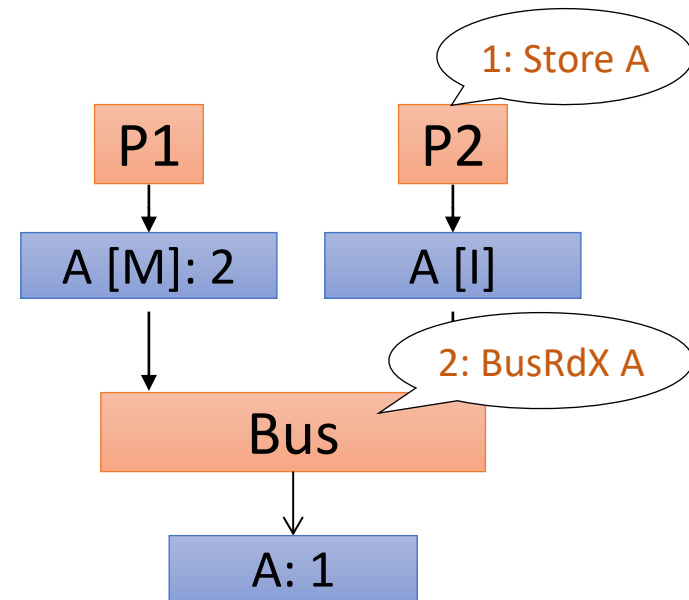
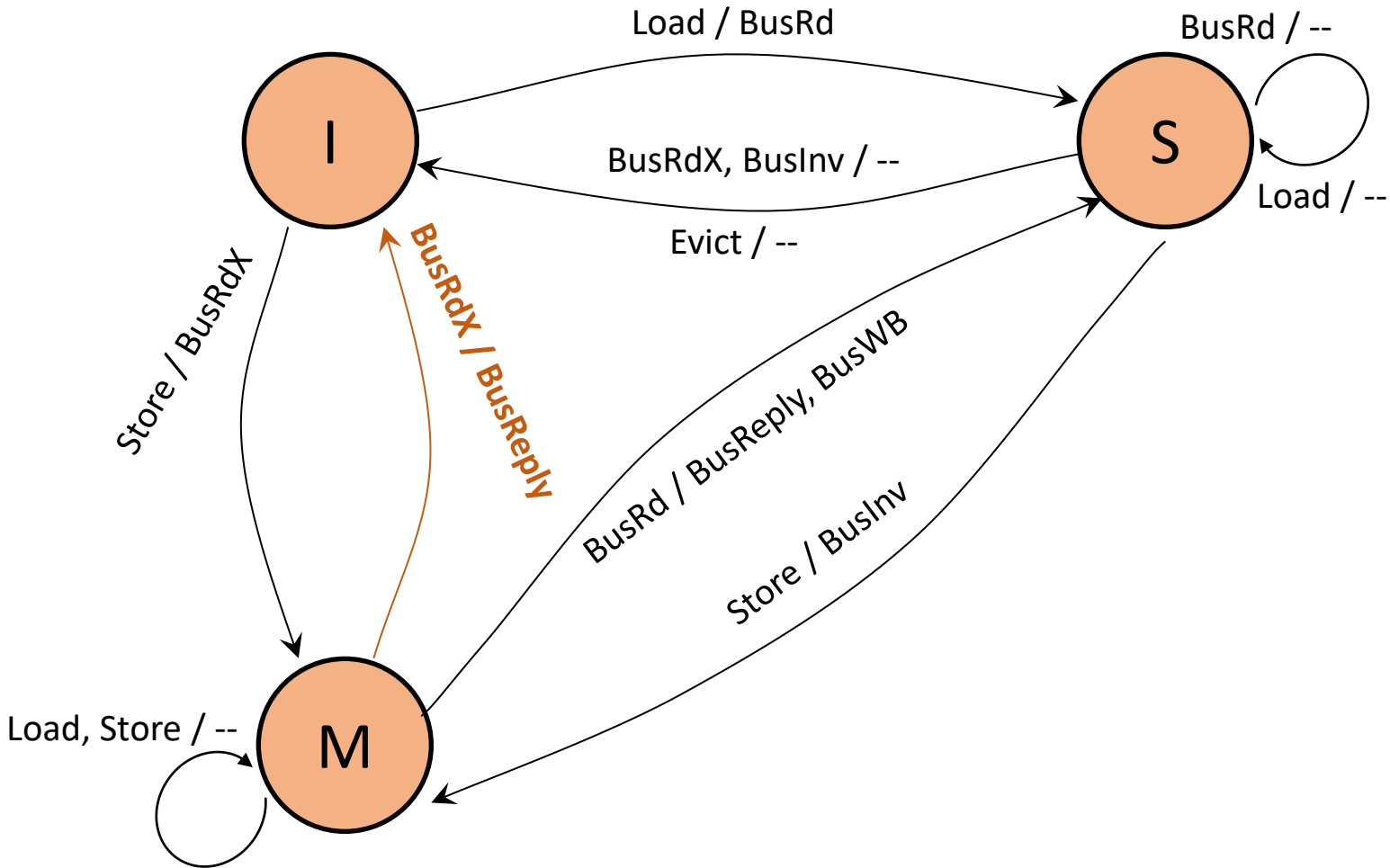
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



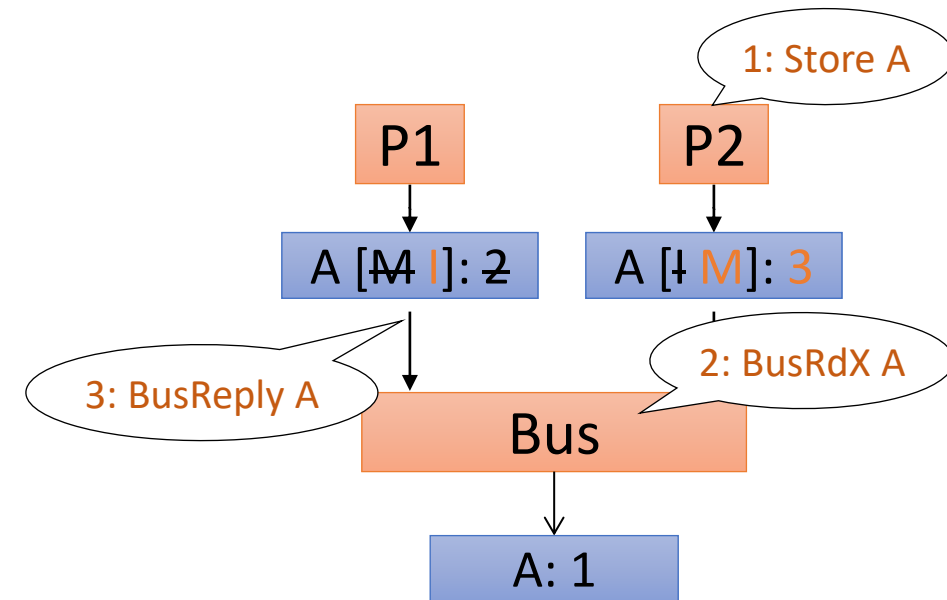
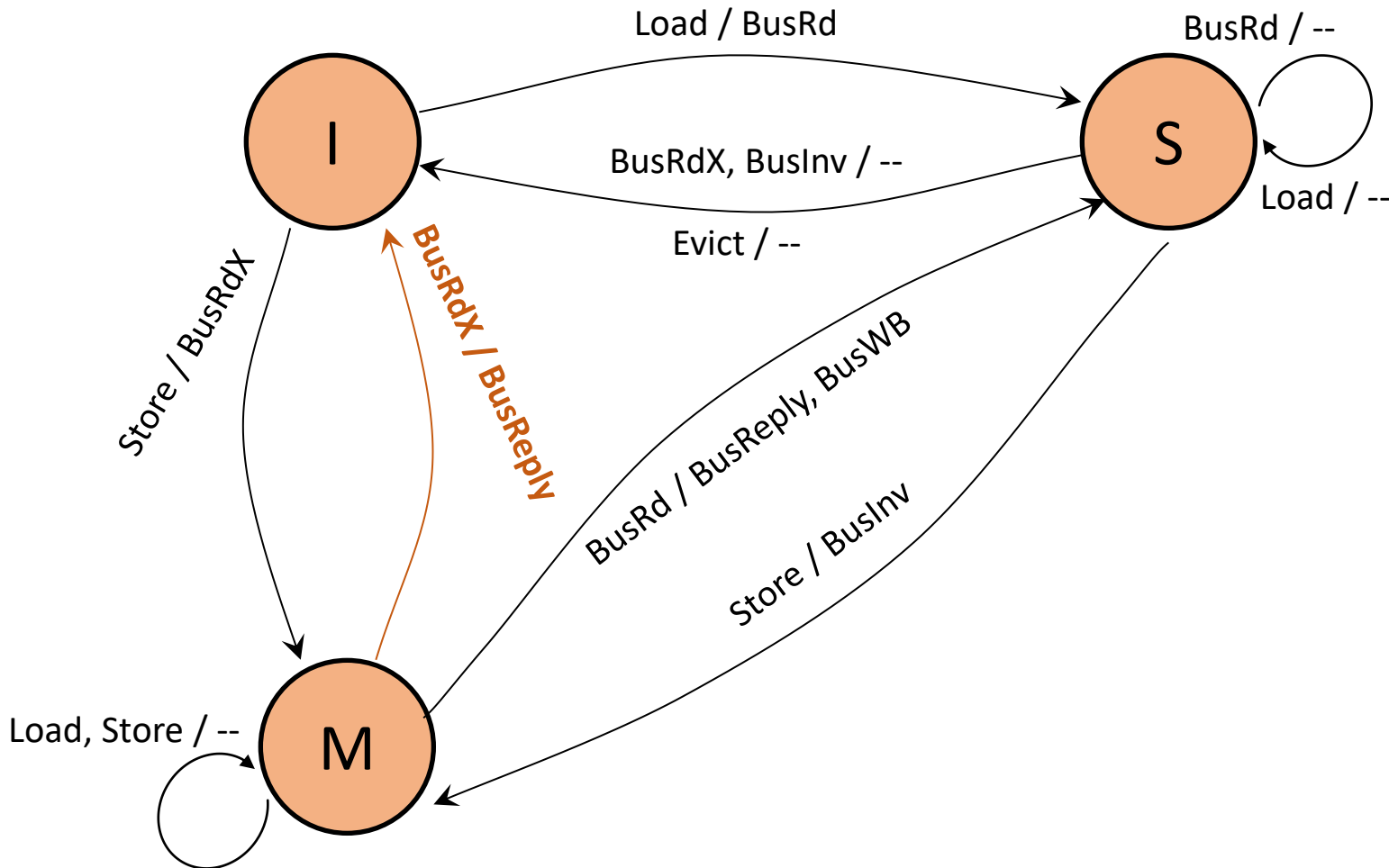
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



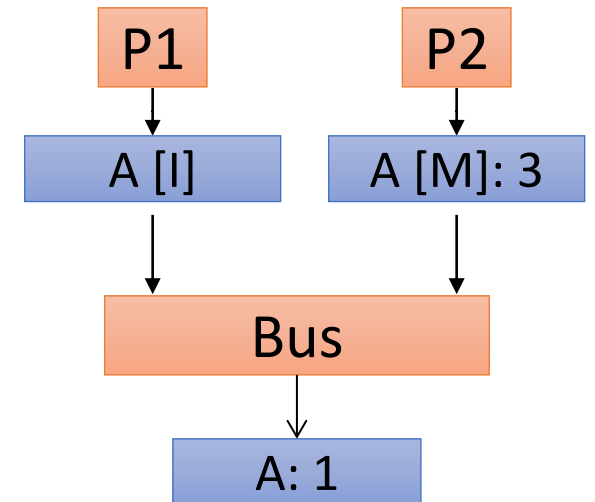
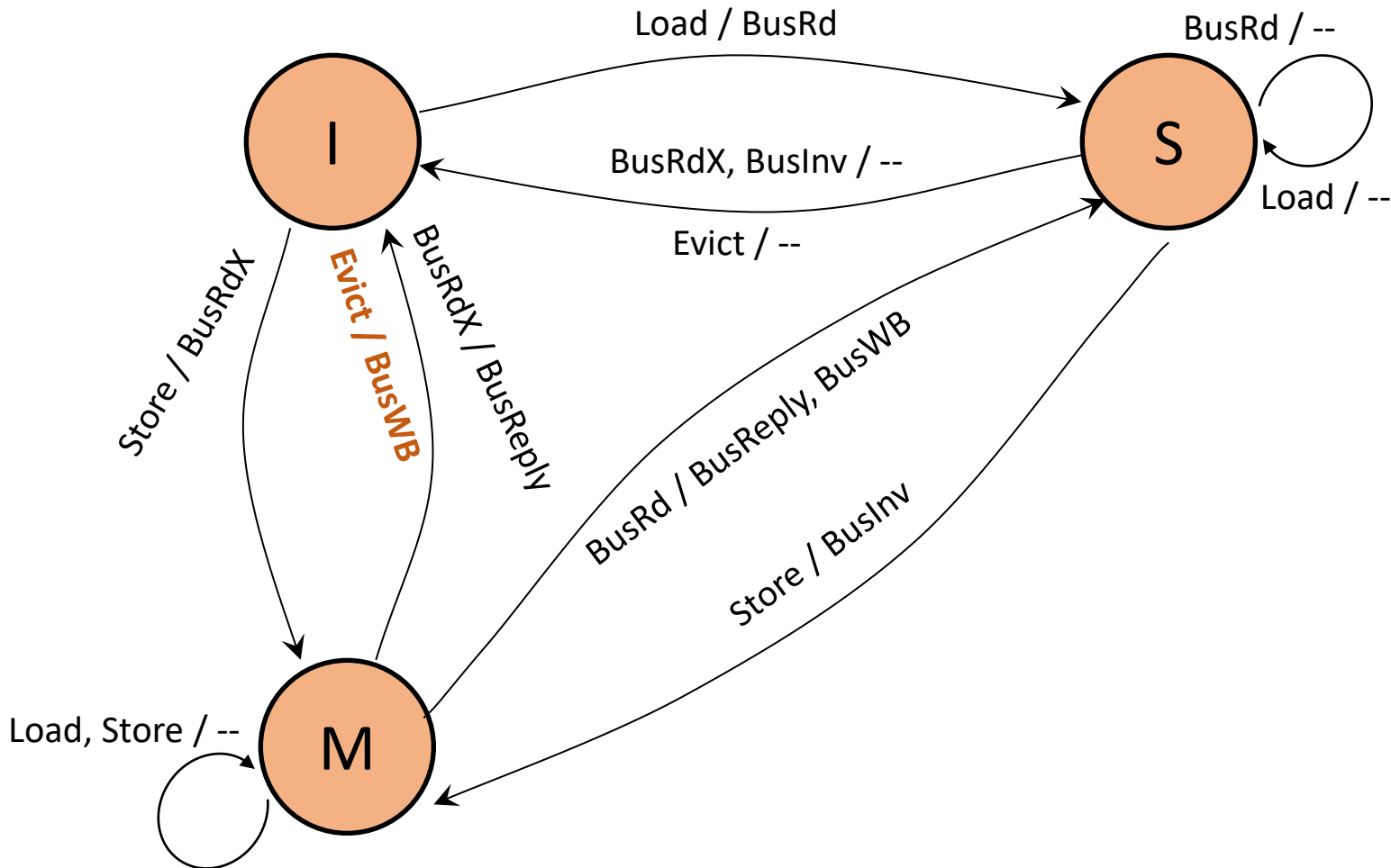
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



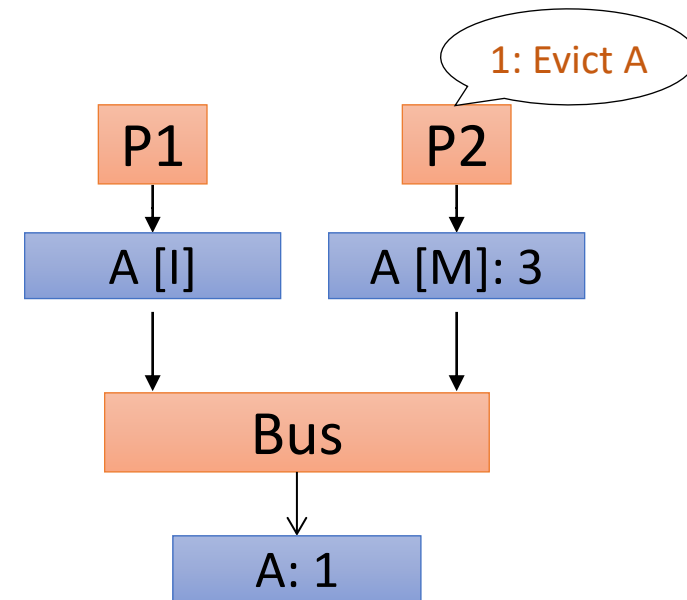
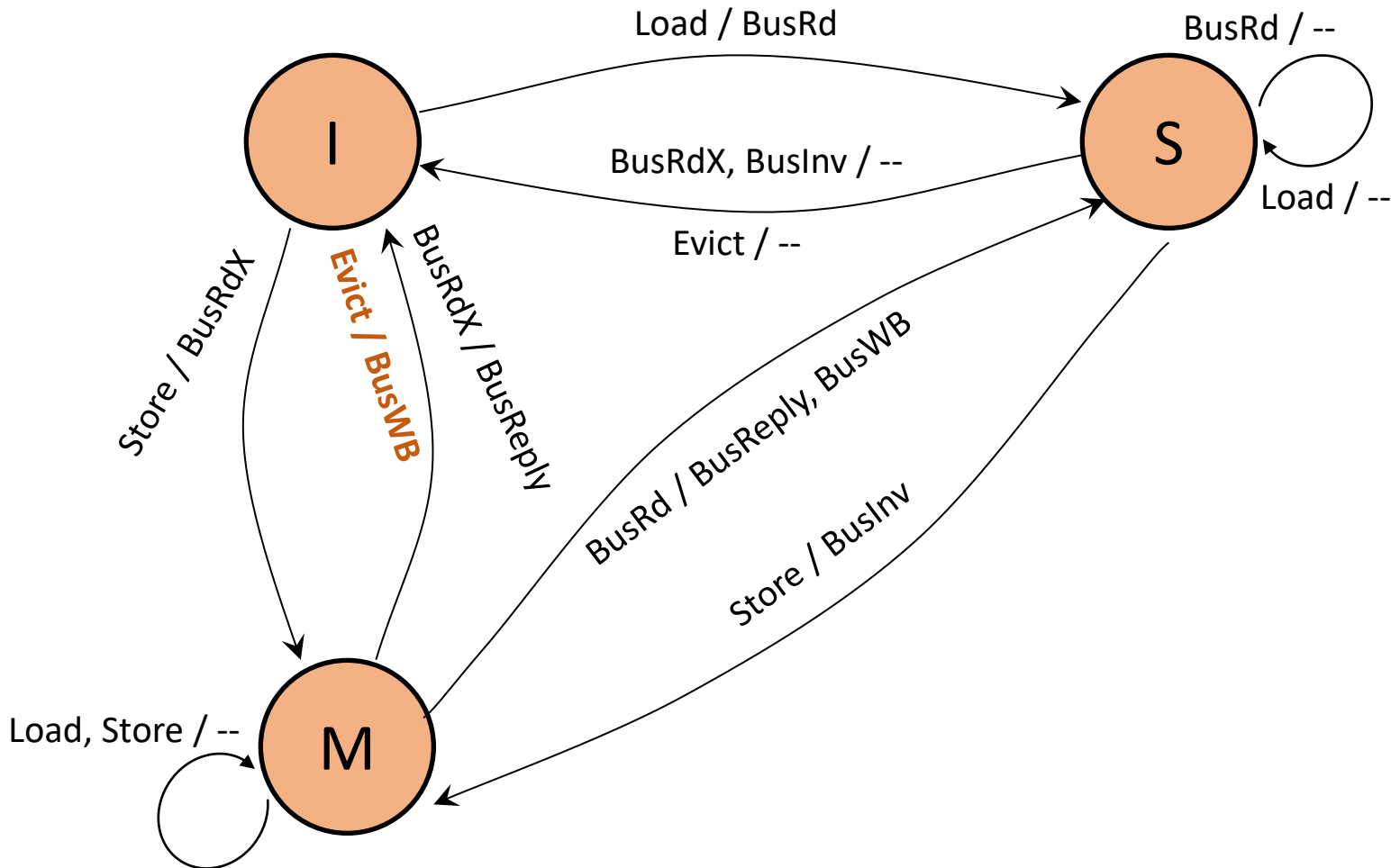
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



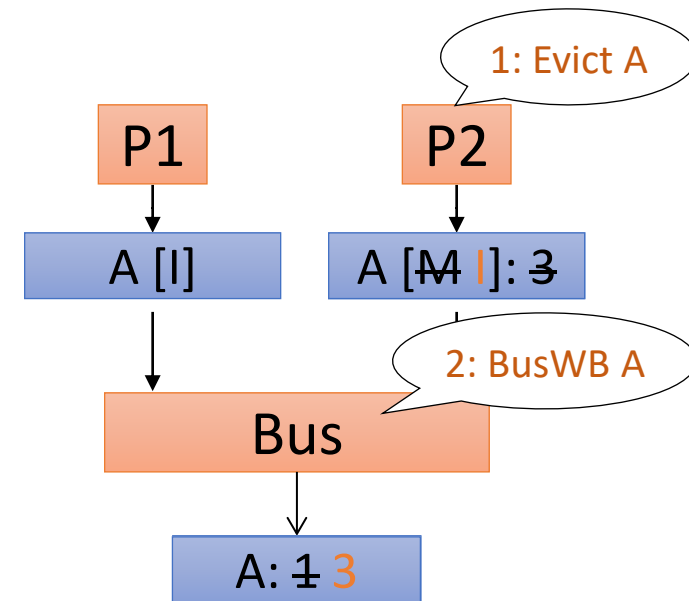
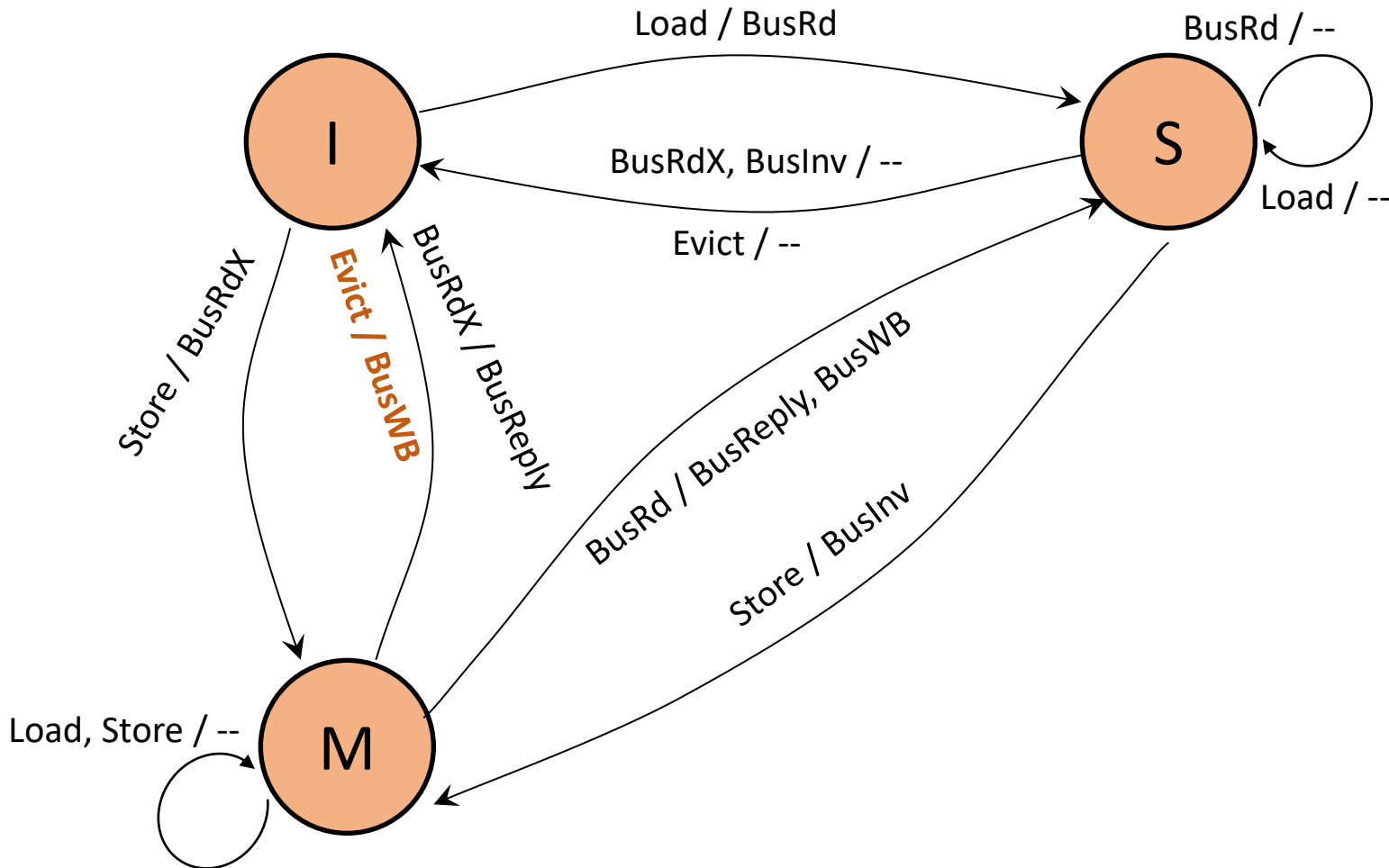
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



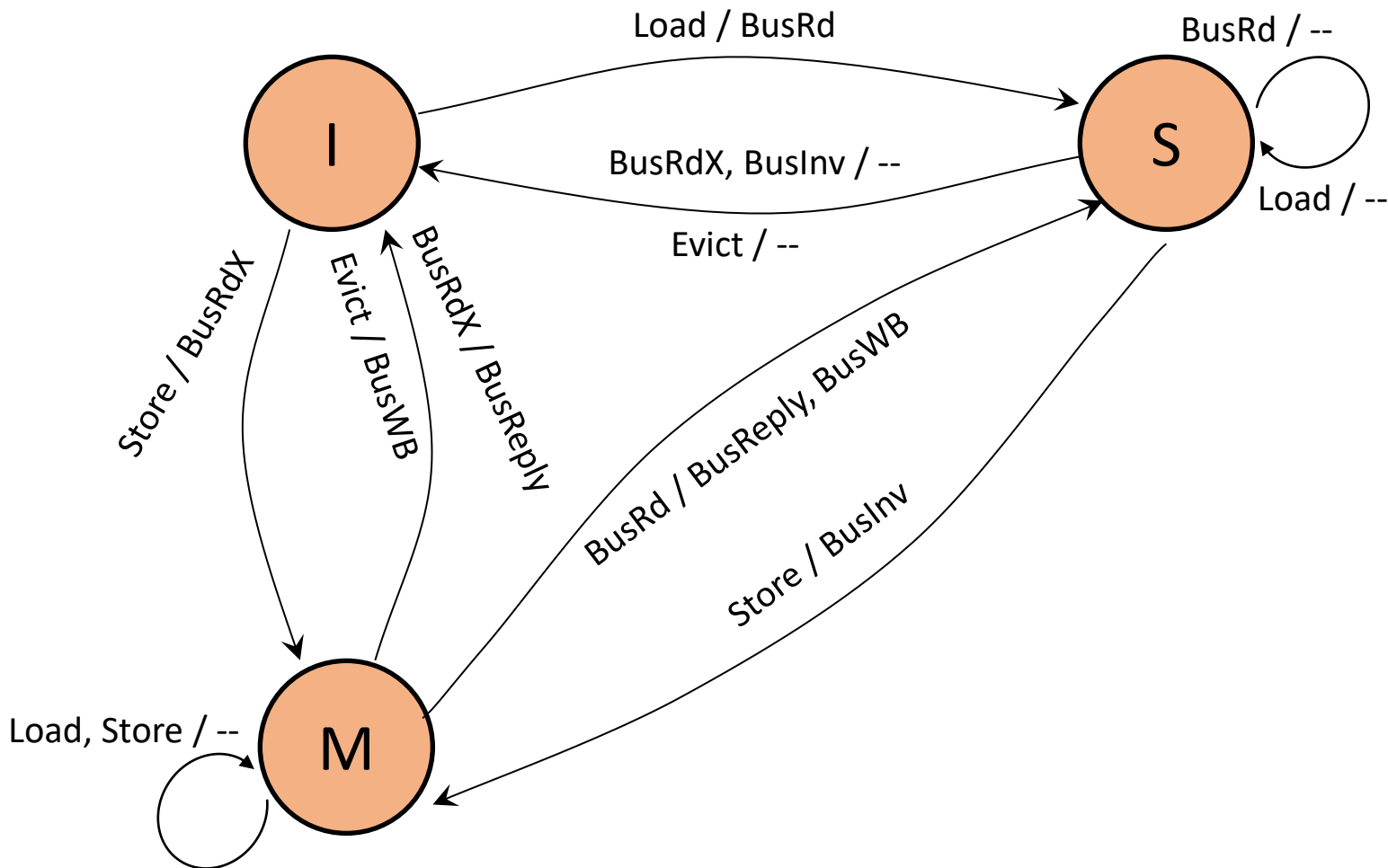
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



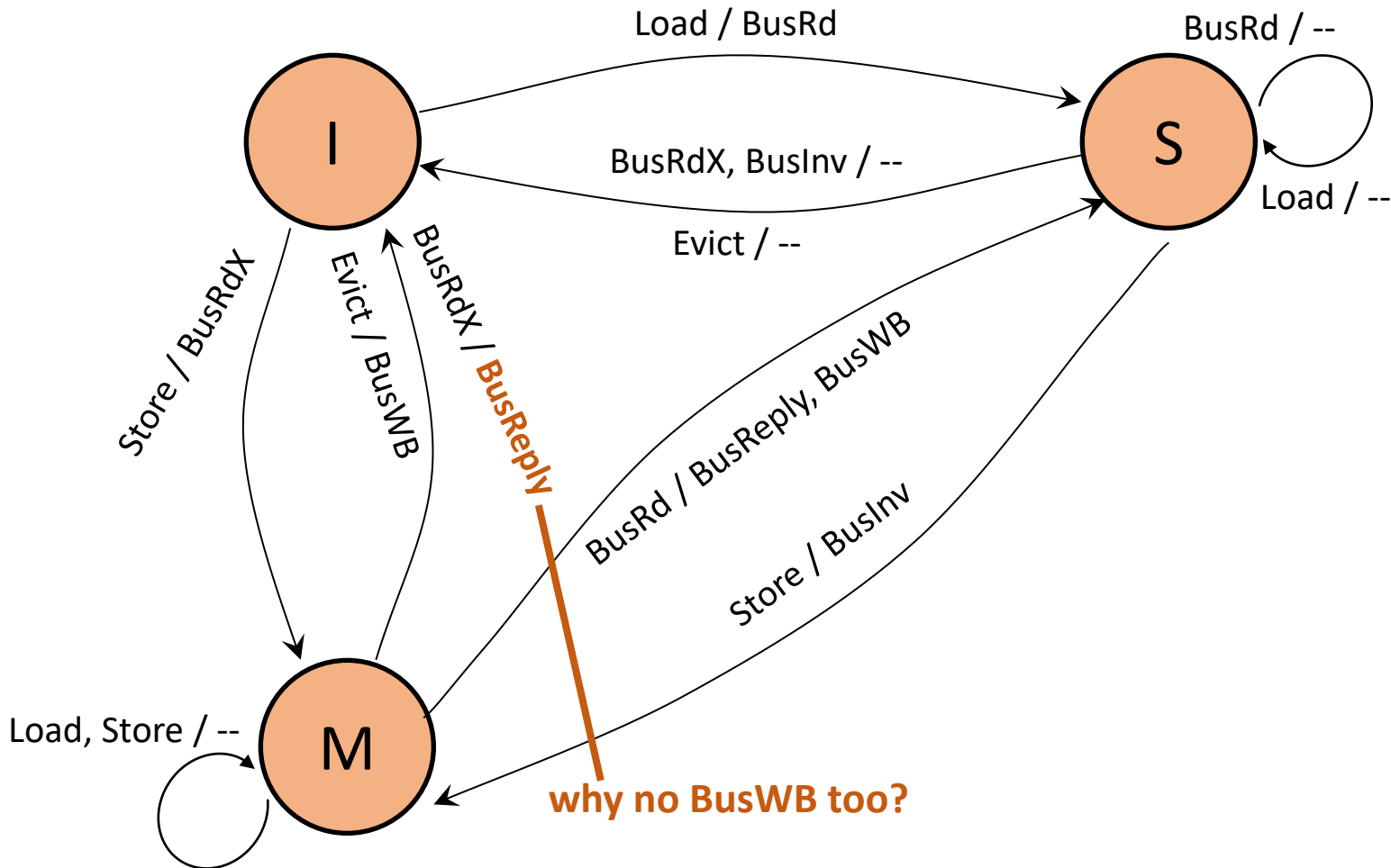
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



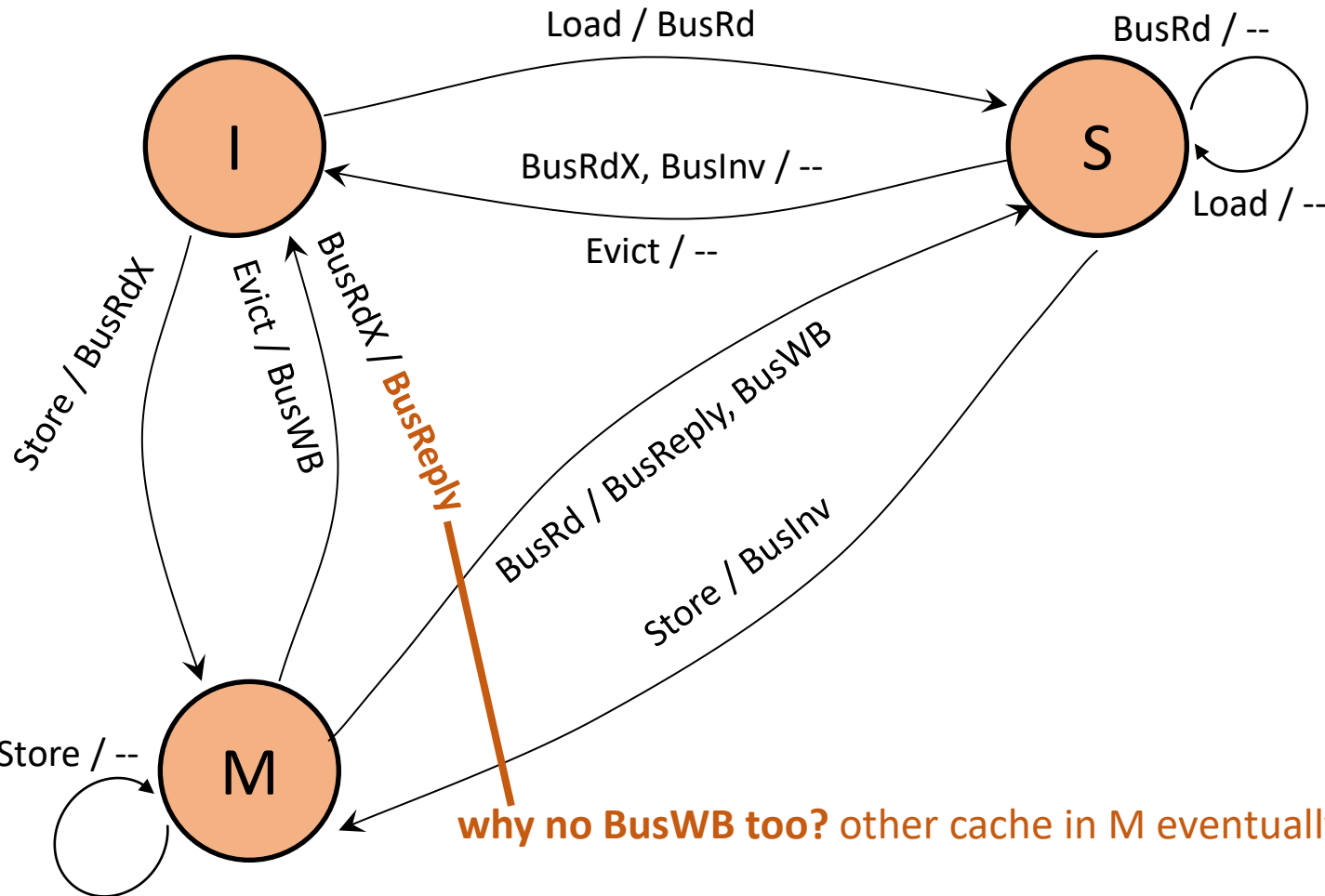
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



Snooping-Based Coherence

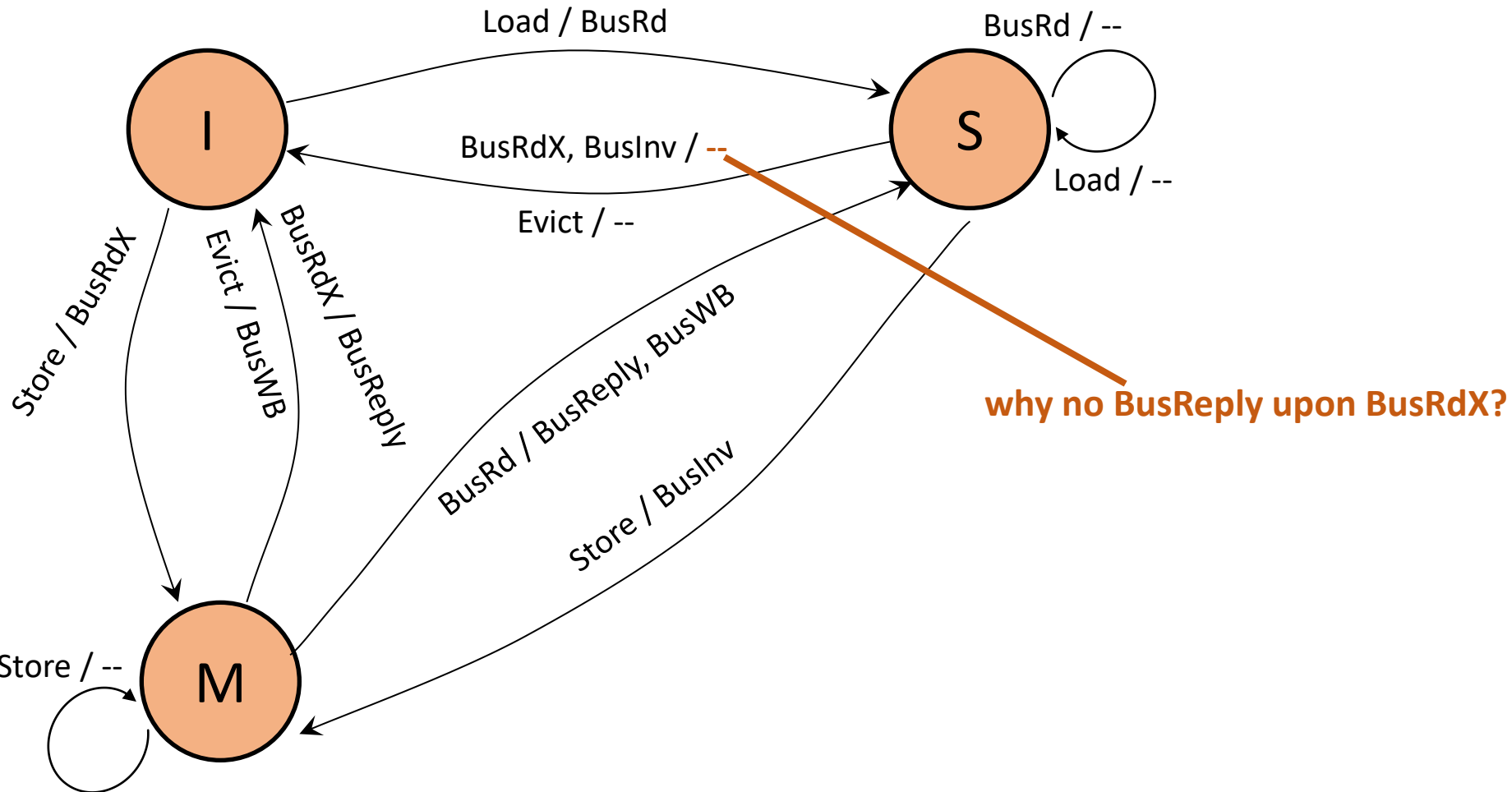
Modified-Shared-Invalid (MSI) Protocol



why no BusWB too? other cache in M eventually will write-back

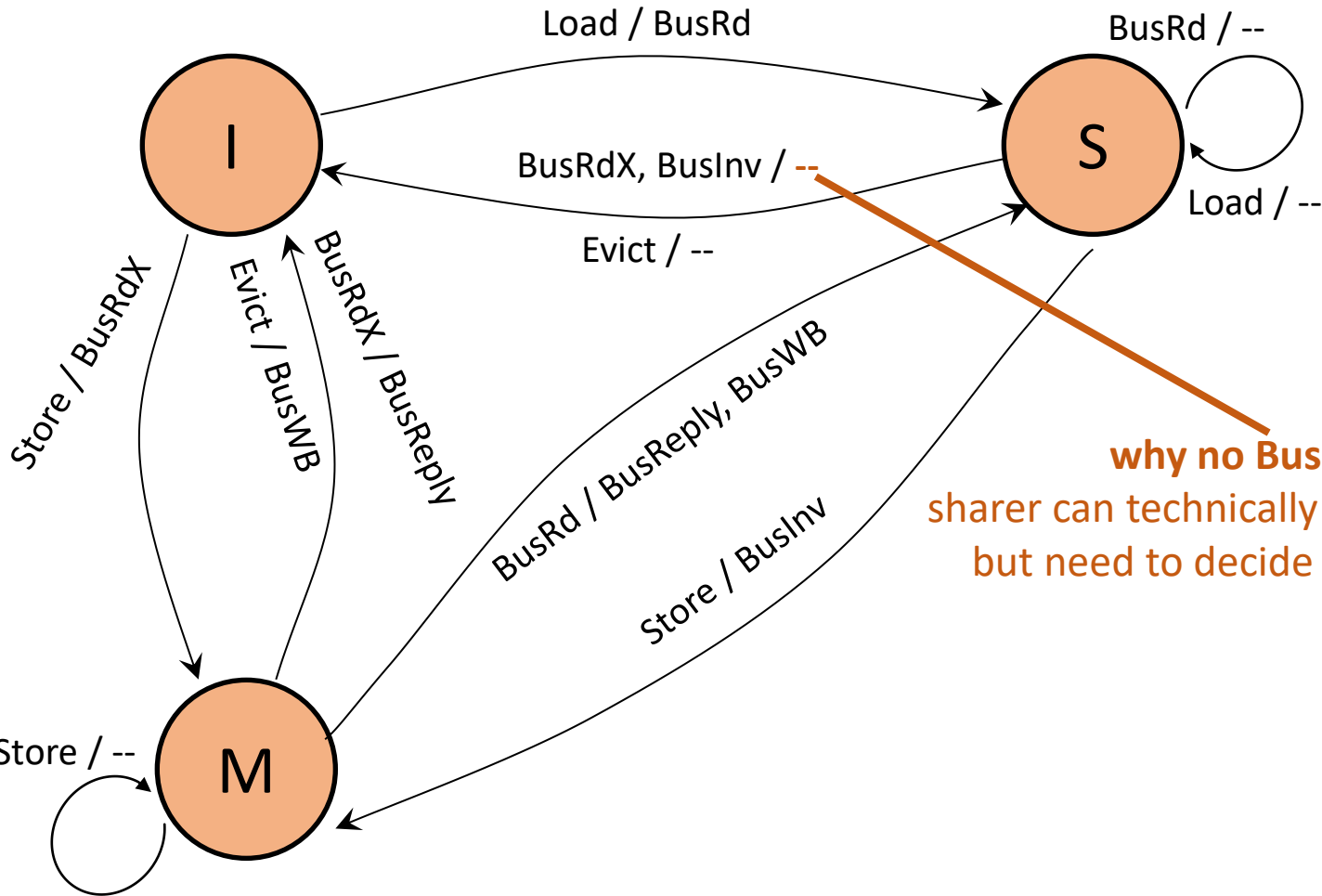
Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol



why no BusReply upon BusRdX?

sharer can technically send block instead of memory,
but need to decide which sharer (*forward* state)

Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol

Current State	Event					
	Load	Store	Evict	BusRd	BusRdX	BusInv
I	BusRd → S	BusRdX → M		→ I	→ I	→ I
S	→ S	BusInv → M	→ I	→ S	→ I	→ I
M	→ M	→ M	BusWB → I	BusReply BusWB → S	BusReply → I	

action
→ next state

Snooping-Based Coherence

Modified-Shared-Invalid (MSI) Protocol

Current State	Event					
	Load	Store	Evict	BusRd	BusRdX	BusInv
I	BusRd → S	BusRdX → M		→ I	→ I	→ I
S	→ S	BusInv → M	→ I	→ S	→ I	→ I
M	→ M	→ M	BusWB → I	BusReply BusWB → S	BusReply → I	

why is this impossible?

